

Aalto University
School of Science
Department of Applied Physics

Antonio David Bastida Zamora

Quantum Algorithms for Fluid Simulations: Quantum Lattice-Gas Automata and Alternatives

SCI Master's thesis

Supervisor: Ville Vuorinen

Instructors: Valtteri Lahtinen and Ljubomir Budinski

Author Antonio David Bastida Zamora

Title of the work Quantum Algorithms for Fluid Simulations: Quantum Lattice-Gas Automata and Alternatives

Degree programme Engineering physics, Master of science (Msc)

Major Materials Physics and Quantum Technology **Code of major** SCI3107

Supervisor Ville Vuorinen

Instructor(s) Valtteri Lahtinen and Ljubomir Budinski

Date April 2024 **Number of pages** 76 + 25 **Language** English

Abstract

Quantum computing has the potential to speedup simulations exponentially. Its applicability spans diverse fields such as biochemistry, cryptography and multiphysics simulations. In this work, we narrow our focus to computational fluid dynamics (CFD). We reviewed different quantum algorithms developed for fluid dynamics, discussing their advantages, disadvantages and specific characteristics. Furthermore, we have focused on the quantum lattice-gas automata (QLGA) algorithm, as it is one of the few existing models that implement nonlinear terms from Navier-Stokes equations in a natural way for a quantum computer. Positioning as the best alternative for a quantum model for CFD according to our research.

After conducting a thorough bibliographical review of various alternatives for CFD, we developed two novel quantum algorithms for fluid dynamics based on QLGA: D1Q3 (one-dimensional) and QFHP (two-dimensional). We delve into the structure, quantum circuits, and computational complexity of these algorithms. Our simulation results show the quantum noise resilience of QLGA and its capability to model Navier-Stokes equations in real use-cases such as a fluid flow between parallel plates, that we used as a benchmark. These results using QLGA are compared to the classical lattice-gas automata under different quantum noise levels and different number of measurements (shots) at the end of each time step. Given the scarcity of literature on the use of lattice-gas automata (LGA) to simulate CFD problems, we also compare the accuracy of LGA with respect to the predicted results from theoretical Navier-Stokes equations. Furthermore, we also suggested two new research directions to test and develop the model, such as using a Monte-Carlo based QLGA (MC-QLGA) and the first test using LGA and QLGA with an airfoil simulation. Although QLGA have problems to be solved, this work paves the way for a quantum-native algorithm for fluid dynamics capable of modelling Navier-Stokes equations at all scales with high Reynolds number with potential speedup.

Keywords LGA quantum algorithms LBM fluid dynamics

Contents

1	Introduction	1
2	Quantum computing and theory overview	2
2.1	Quantum physics	3
2.1.1	Kinetic postulates	3
2.1.2	Dynamical postulates	6
2.2	Quantum computing	8
2.2.1	One-qubit quantum gates	8
2.2.2	Two-qubit quantum gates and entanglement	10
2.2.3	Universal quantum gates	11
2.2.4	Real quantum devices	12
2.3	Quantum emulators	15
2.4	Quantum Algorithms	16
2.4.1	Quantum state preparation	16
2.4.2	Measurement	18
3	Quantum Lattice-Gas Automata	19
3.1	Introduction to Lattice-Gas Automata models	19
3.1.1	D1Q3	22
3.1.2	HPP	23
3.1.3	FHP	23
3.2	QLGA bibliographic review	26
3.2.1	Is it possible to build a quantum LGA?	26
3.2.2	Mesoscopic QLGA and factorisation	28
3.2.3	Recent algorithmic implementations on QLGA	31
4	Quantum Lattice Boltzmann Method	32
4.1	Lattice-Boltzmann method and the BGK Approximation	33
4.2	Quantum Lattice-Boltzmann Method bibliographic review	37
5	Other quantum algorithms for fluid dynamics	39
5.1	Quantum algorithm for linear system of equations (HHL)	40
5.2	Variational quantum algorithms	42
5.3	Quantum finite volume method (QFVM)	44
5.4	Quantum algorithm for nonlinear differential equations	45
6	Novel quantum models for fluid dynamics	45
6.1	Quantum Lattice-gas automata algorithm overview	46
6.2	Computational complexity	52
6.3	1D model D1Q3 -Numerical results	54
6.4	2D model QFHP and Numerical results	58
6.4.1	Introducing the 2D QFHP	58
6.4.2	QFHP Poiseuille flow: Numerical results	62

6.5	Further research and challenges	70
6.5.1	Monte-Carlo QLGA	70
6.5.2	QFHP with airfoil simulation	75
7	Conclusion	76
	References	78
A	From LGA to LBM using a multiscale expansion	94
B	Classical LBM example using D1Q3 diffusion equation	100

1 Introduction

In recent decades quantum computing has emerged as a pioneering field to develop more efficient algorithms taking advantage of the properties of quantum physics. This field has experienced a huge inflow of investments in recent years, with companies emerging and new university study programs being created [1, 2]. This boom is similar to the artificial intelligence spring we have seen in the last decade. This is the reason behind the rapid acceleration on quantum computers, with lower error rates and a higher number of qubits that have been released in recent years [3]. However, the practical use of quantum computers is limited for the moment, with some examples found in finance [4], chemistry [5] and cryptography [6]. Therefore, while quantum computing hardware is improving, the real usefulness of quantum computing is in question. Developing useful quantum algorithms that will provide a real advantage has become increasingly important and of interest in the industry.

In specific, we focus on quantum algorithms for computational fluid dynamics (CFD). The number of operations to model turbulence scales as $O(Re^3)$, as a result of the high nonlinearity of the equations. This means that to model an aircraft with precision, which stands at a $Re = 10^8$, we need around 10^{24} floating point operations. However, by making use of the quantum superposition, we could potentially compute these operations with $2^Q = Re^3$, or the same $Q = 3\log_2(Re)$ [7]. This is equivalent to around 80 qubits for an aircraft, which is in the scope of current quantum hardware. There are several quantum algorithms that have been developed for this purpose (among others). Originally, the research community gave special focus to linear solvers such as Harrow–Hassidim–Lloyd algorithm (HHL) [8] or variational quantum algorithms [9]. However, these methods cannot model nonlinearities from Navier-Stokes equations, relying on linearization and the usage of approximations to solve them. Additionally, their advantage with respect to classical algorithms disappears when considering key steps such as initialization or measurements. In the present work, we review each of them with special emphasis in lattice-based models, which are especially promising due to their ability to scale simulations using an exponential number of quantum states. Additionally, these models, based on cellular automata (CA), have the ability to transform simple microdynamical rules into a specific macroscopic behaviour. The first LGA models to appear in the literature were the Hardy-Pomeau-Pazzis (HPP) [10, 11, 12] and Frisch-Hasslacher-Pomeau [13] models. However, the study of LGA also led to lattice Boltzmann methods (LBM) [14], and LGA was largely abandoned due to the advantages presented by the former. Specifically the classical noise resilience and simulating directly at a mesoscopic scale, which added flexibility and reduced the computational time. However, in the context of quantum computing, LGA has been positioned as a very promising algorithm, as it is capable of modelling nonlinear terms with a simple framework and scale exponentially the lattice sites. The goal of this thesis is to review the most promising and well-studied quantum algorithms for fluid dynamics and their potential usage in simulations. The

special focus will be on LGA, developing new quantum algorithms and providing numerical results.

We will divide this work into five main sections. First, in Section 2, we will introduce quantum computing. We will provide an introduction to quantum physics postulates, quantum gates, real quantum devices and a bibliographic review of two of the most important parts of quantum algorithms (quantum state preparation and measurement). In Section 3, 4 and 5, we will review the several quantum algorithms which have been researched for CFD. Specifically, in the second chapter, we will focus on quantum LGA (QLGA), introducing the theory and a bibliographic review of the origins of the model and its recent implementations. Then, in the third section, we do the same with quantum LBM (QLBM). The fourth section is composed of other quantum algorithms (no lattice based models) and their recent advances towards efficiency and modelling nonlinear terms. Finally, in Section 6, we present a novel quantum algorithm for CFD using QLGA. We introduce two versions: The first of them is a one-dimensional model (D1Q3) and the second one a two-dimensional model (QFHP). We present the algorithm and quantum circuits, analyze their computational complexity, compare their resilience to noise with different error rates and number of measurements and test the model using a flow between parallel plates benchmark. We conclude the section with a brief discussion of possible future research and challenges to obtain quantum advantage and continue developing this algorithm.

2 Quantum computing and theory overview

This chapter aims to provide a simple but accurate review of quantum physics, quantum computing, and quantum algorithms. The goal is to settle the foundations for the following chapters, where we will cover state-of-the-art quantum algorithms for computational fluid dynamics. We will only assume previous knowledge of linear algebra.

Quantum mechanics is a mathematical framework consisting of a simple set of rules with counter-intuitive interpretations. It has been shown to accurately describe physical phenomena such as fundamental particles, matter interaction with light, superconductors, and others. One of the numerous fields that the theory of quantum physics opened up was quantum computing. This emerging field had, at a theoretical level, several advantages concerning classical computing. We will introduce them in more detail, but some of them are entanglement, superposition, and parallel computations, among others. Since the 1970s, more and more efforts have been directed to control quantum systems with the goal of several engineering applications, making quantum computing one of the most promising for its potential impact. Currently, in the 21st century, quantum computing is not only

a potential and emerging field but also a reality. We will divide this chapter into three parts: Quantum physics, quantum computing, and quantum algorithms.

2.1 Quantum physics

This subsection will be a comprehensive summary of quantum mechanics based on [15, 16]. We will introduce the postulates of quantum mechanics, that provide a link between the mathematical tools and the physical model. These are divided into kinetic and dynamical postulates, where the former introduces the mathematical framework associated with a physical system and its properties that quantum mechanics will use and the second describes the system's dynamical evolution of the mentioned mathematical framework associated with the physical system. The description to be provided will aim to be a basis for understanding quantum computing and information later on. Therefore, we will focus only on finite-dimensional quantum systems. A generalization can be found in [17].

2.1.1 Kinetic postulates

These postulates set a mathematical framework and clear rules to define quantum physics. It comprises three postulates: The state space, dynamical variables, and composite systems. Step by step, we will build the principles of quantum physics.

Postulate 1. *Any isolated physical system has a complex vector space with an inner product (Hilbert space $\mathcal{H}$) associated, known as the state space of the system. The mentioned system is completely described by its state vector, which is a unit vector in the system's state space.*

Before breaking down the previous postulate, we will introduce a widely extended notation, known as Dirac notation, due to the person who coined the term. To refer to an element from a vector space, we use $|\cdot\rangle$, known as ket, where we use the symbol $\cdot$ as a filler. In specific $|\cdot\rangle \in \mathcal{H}$ indicates that the vector is in a Hilbert space. Nevertheless, to refer to a certain vector, we use labels inside the ket such as $(\psi$ or $\phi)$. In the same way, we may refer to $\langle\cdot|$, called bra, for an element of a dual vector space. The elements of this space can be represented as a linear operator from the vector space associated to the complex

$$\phi^* : \mathbf{V} \longrightarrow \mathbb{C}$$

the set of all the linear maps ϕ^* is called dual space.

Definition 1. *A complex Hilbert space is a complex vector space $\mathcal{H}$ with an inner product $\langle\cdot|\cdot\rangle$, complete with respect to the norm induced by $\langle\cdot|\cdot\rangle$. That is, every Cauchy sequence converges to a unique limit within $\mathcal{H}$.*

From now on, we will use $\mathcal{H} \equiv \mathbb{C}^d$ with an inner product, where d is finite. Following the bracket notation introduced previously, let us now formally define the inner product.

Definition 2. *An inner product is a function $\mathbf{V} \times \mathbf{V} \rightarrow \mathbb{R}$ (or identifying the vector space already with the corresponding Hilbert space, $\mathcal{H} \times \mathcal{H} \rightarrow \mathbb{R}$), such that the following properties hold:*

1. Positive-definiteness: $\forall |\psi\rangle \in \mathcal{H}; \langle\psi|\psi\rangle \equiv ||\psi||^2 \geq 0$ and $\langle\psi|\psi\rangle = 0 \Leftrightarrow |\psi\rangle = 0$.
2. Linearity (in the second argument): $\forall |\psi\rangle \in \mathcal{H}, \langle\phi| \in \mathcal{H}^*$ and $z_i \in \mathbb{C}$;

$$\langle\psi| \left(\sum_{i=1}^n c_i |\phi_i\rangle \right) = \sum_{i=1}^n c_i \langle\psi|\phi_i\rangle$$
3. Anti-linearity (in the first argument): $\forall |\psi\rangle \in \mathcal{H}, \langle\psi| \in \mathcal{H}^*$ and $z_i \in \mathbb{C}$;

$$\left(\sum_{i=1}^n c_i |\phi_i\rangle \right)^\dagger |\psi\rangle = \sum_{i=1}^n c_i^* \langle\phi_i|\psi\rangle$$
4. Skew symmetry: $\forall |\psi\rangle \in \mathcal{H}, \langle\phi| \in \mathcal{H}^*; \langle\phi|\psi\rangle^\dagger = \langle\psi|\phi\rangle$

Notation: Conjugate transpose of an operator $\mathbf{A}$ is denoted as $\mathbf{A}^\dagger$. State vectors are written in a given basis, which in our case is finite. The most simple example is the basis associated to a qubit. A qubit is a two-state quantum system, with a basis usually denoted with vectors $|0\rangle$ and $|1\rangle$. Therefore $|\psi\rangle = a|0\rangle + b|1\rangle$, with $a, b \in \mathbb{C}$. Due to the first postulate, $||\psi||^2 = 1 \rightarrow |a|^2 + |b|^2 = 1$.

Postulate 2. *For each dynamical variable $\mathcal{A}$ of a quantum system, there is a corresponding self-adjoint operator $\mathbf{A}$. This operator assigns probabilities for different outcomes depending on the state of the system. In specific, the eigenvalues of $\mathbf{A}$ specify the values of $\mathcal{A}$.*

The operator $\mathbf{A}$ can be decomposed into a weighted sum of other operators called projectors, which are the effective decomposition with respect to the eigenspaces of the operator associated with the dynamical variable $\mathcal{A}$. This is because $\mathbf{A}$ is a self-adjoint operator, which means that $\mathbf{A} = \mathbf{A}^\dagger$ and therefore we can apply the spectral theorem [18]. Notice that this is not the case for every operator in quantum mechanics but for those associated with an observable variable. In simple terms,

$$\mathbf{A} = \sum_i \lambda_i \mathbf{P}_{\lambda_i} \quad (1)$$

with $\mathbf{P}_{\lambda_i}$ projector to the eigenvector associated to the eigenvalue λ_i , in specific, $\mathbf{P}_{\lambda_i} |\psi_i\rangle = |v_{\lambda_i}\rangle$ if $|\psi\rangle$ is an eigenvector of the operator.

This is called the discrete spectral decomposition of an operator (if it exists) with λ_i the eigenvalues of $\mathbf{A}$. Due to the spectral theorem, we know that $\lambda_i \in \mathbb{R}$ and that the eigenvectors form a complete orthonormal basis and, therefore, the spectral decomposition is as seen in (1). We will not cover operator algebra in this article but more can be read in [19].

Postulate 3. *When the state space of a physical system is composed of several distinguishable subsystems, the total system is the tensor product of its constituents.*

Definition 3. *The tensor product $\otimes$ of two quantum state vectors $|x\rangle$ and $|y\rangle$ of length n and m respectively, is a $n \times m$ -matrix such that $(|x\rangle \otimes |y\rangle)_{ij} = x_i y_j$.*

This postulate is directly related to the superposition principle in quantum mechanics, a crucial concept that allows for parallel computations in quantum computing when combined with entanglement. The idea of using tensor product for describing the total system is due to the tensor product algebra. If we have two subsystems described in their basis as $|\psi\rangle = a|0\rangle + b|1\rangle$ and $|\phi\rangle = c|0\rangle + d|1\rangle$ (using qubits), the composed system would be $|\psi\rangle \otimes |\phi\rangle \equiv |\psi\rangle |\phi\rangle \equiv |\psi\phi\rangle = (a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle) = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle$. This means that the postulate indicates how we compose quantum systems and how such composition creates a superposition of outcomes as a weighted sum. We will soon see the relevance that these complex weights have for a quantum state. This also means that if we want to apply an operator to a composed system, we must use the operators of their constituents such as $\mathbf{A} = \mathbf{A}_1 \otimes \mathbf{A}_2$, given that $\mathbf{A}_1$ and $\mathbf{A}_2$ are operators of the subsystem. This is not the case though if the states were originally entangled or will be entangled later.

In quantum computing, we will often encounter states that cannot be described by its constituents, this is called an entanglement. An entangled state has “entity” only by itself. Mathematically, for two qubits this translates as given a composed state $|\psi\rangle$, there exist no single qubit states $|a\rangle$ and $|b\rangle$, such that $|\psi\rangle \neq |a\rangle |b\rangle$. In this context, operators cannot be decomposed in the subsystems as before. Note that this is an essential part in achieving quantum advantage with a quantum computer. While superposition can be achieved classically having a system of n strings in two vibrational modes at the same time encoding 0 and 1, the strings will not interact, and there will not be entanglement. Therefore, superposition can be achieved classically but all the combinations of results obtained at the same time by applying certain algorithm will not prevail. This is because entanglement is described by correlated tensor products, giving structure to our data that we can use [20].

2.1.2 Dynamical postulates

Having provided the kinematic postulates, which set the basis of quantum physics and the mathematical framework in which it is described, we will introduce the dynamical postulates. These postulates describe the evolution of quantum systems: how the states evolve and how to extract results from the superposition, i.e., measurement.

Postulate 4. *The evolution of a closed system (without interaction with any other systems) is described by a unitary operator $\mathbf{U}$. This means that the state vector $|\psi\rangle_{t_1}$ and $|\psi\rangle_{t_2}$ are related by a unitary operator which depends only on t_1 and t_2 .*

The corresponding mathematical description of the postulate is

$$|\psi\rangle_{t_2} = \mathbf{U}_{t_2 t_1} |\psi\rangle_{t_1} \quad (2)$$

In quantum computing, we will often see that the evolution of qubits is described using such operators. To understand this, we will briefly describe the properties of the unitary operators and their relationship with the Schrödinger equation, which is the governing dynamical equation of quantum mechanics.

Properties 1. *The time-evolution operator $\mathbf{U}$, $\forall t_i : i \in [0, 1, \dots, n]$, where t_n is the last time step, satisfies the following properties*

1. Composition: $\mathbf{U}_{t_{i+2} t_i} = \mathbf{U}_{t_{i+2} t_{i+1}} \mathbf{U}_{t_{i+1} t_i}$
2. Inverse: $\mathbf{U}_{t_{i+1} t_i}^{-1} \equiv \mathbf{U}_{t_{i+1} t_i}^\dagger = \mathbf{U}_{t_i t_{i+1}}$
3. Identity: $\mathbf{U}_{t_i t_i} = \mathbf{I}$

The evolution of a quantum system is usually described using Lie algebra and its generators, this mathematical description will not be introduced here but we recommend [21, 22]. Briefly, the unitary operators can describe what is called the Hamiltonian operator $\mathbf{H}$, which generates the unitary evolution. This means that we can express the evolution as

$$|\psi\rangle_t = e^{-i\mathbf{H}t} |\psi\rangle_{t_0} \quad (3)$$

in the case the generator is time-independent. For time-dependent generators, the expression is more complex. This is related directly to the Schrödinger equation which reads as

$$i\frac{d}{dt}\psi_t = \mathbf{H}_t\psi_t. \quad (4)$$

Postulate 5. *There exists a non-unitary and irreversible dynamical evolution of the state function using experiments $\mathcal{M}$, which can give different random outcomes. Measurements are described by a collection of operators $\mathbf{M}_m$, and each possible measurement outcome is noted by subindex m .*

Using this and the rest of the postulates, we can derive what is called as the Born rule that states as following

Proposition 1. *When an observable associated with a discrete spectrum self-adjoint operator $\mathbf{A}$ is measured in the context of quantum mechanics then the measured result will be one of its eigenvalues with probability*

$$p(m) = \langle \psi | \mathbf{M}_m^\dagger \mathbf{M}_m | \psi \rangle \quad (5)$$

As the measurement operators are not unitary, the probability does not preserve unitarity but only positivity. As the total probability is 1

$$1 = \sum_m p(m) = \sum_m \langle \psi | \mathbf{M}_m^\dagger \mathbf{M}_m | \psi \rangle \rightarrow \sum_m \mathbf{M}_m^\dagger \mathbf{M}_m = \mathbf{I} \quad (6)$$

This means that an instant after the measurement, the state vector will not be any more linear combination of states but only one of them. This process is known as the collapse of the wavefunction. This can be represented as

$$|\psi\rangle_t \rightarrow |\psi\rangle_{t+dt} = \frac{\mathbf{M} |\psi\rangle}{\sqrt{p(m)}} \quad (7)$$

Before finishing the chapter, it is important to mention the no-cloning theorem [23]. This is an important result and very transcendent in quantum computing and especially when developing quantum algorithms.

Theorem 1. (No-cloning theorem): *It is not possible to produce several copies of a quantum system, each having the same state as the original.*

This means that it is not possible to create a non-entangled copy of a quantum state, which makes quantum algorithm development especially difficult. In parallel, we can introduce a similar result which is the time-reversed dual of the no-cloning theorem [24]

Theorem 2. (No-deletion theorem): *Given two identical states, it is impossible to delete one of the copies.*

2.2 Quantum computing

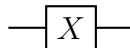
Having given a formal introduction to quantum physics, and defined the postulates, we will now focus on quantum computing. This overview is a practical and qualitative guide to quantum computing to understand its benefits, limitations, and notation that will be necessary for the next chapters. Specifically, we introduce how the computation is done; what are the basic operations to create algorithms, universal set of quantum gates, the significance of noise, and transpilation, among other things. The introduction is somewhat superficial, but an appropriate bibliography is recommended to the reader when necessary. The main sources are [16, 25].

As said in the previous subsection, a qubit (quantum bit) is one of the most simple quantum states, described by a linear combination of $|0\rangle$ and $|1\rangle$. Qubits are broadly used in quantum computing due to the obvious analogy with classical bits that acquire either a state 0 or state 1. However, qubits have a particularity: they can be in a complex superposition of $|0\rangle$ and $|1\rangle$. This means that we can algorithmically adjust the probability with which we will obtain a certain measurement outcome and a complex phase. As a convention, and without lack of generality, we will use $|0\rangle$ as the state at the start of any computation.

2.2.1 One-qubit quantum gates

A quantum algorithm consists of a set of quantum gates applied to qubits, which allows to transform the initial $|0\rangle^{\otimes n}$ into a desired state $|\psi\rangle$. If the algorithm is based on a single qubit, it will be sufficient to use a single quantum gate to obtain the desired result. In classical computing, this usually means either keeping the state as it was (trivial operation) or flipping it from 0/1 to 1/0. In the quantum world, this is a bit more complex. We will introduce now the most used quantum gates and their outcomes. We will show both the equation that reflects the transformation and the circuit notation, which uses a line to note the qubit on which it is applied, and a box with a symbol to note which transformation is applied. This is similar to a classical computational circuit.

- **NOT or X:** This gate flips the state of the qubit from $|0\rangle$ to $|1\rangle$ or vice-versa. The operation to arbitrary states is the linear extension of this.



$$\mathbf{X}(a|0\rangle + b|1\rangle) = a|1\rangle + b|0\rangle \quad (8)$$

- **H** or Hadamard: It creates an equal superposition of $|0\rangle$ and $|1\rangle$, with the phase depending on the input.

$$\text{---} \boxed{H} \text{---}$$

$$\mathbf{H}(a|0\rangle + b|1\rangle) = \frac{a}{\sqrt{2}}|0\rangle - \frac{a}{\sqrt{2}}|1\rangle + \frac{b}{\sqrt{2}}|0\rangle + \frac{b}{\sqrt{2}}|1\rangle \quad (9)$$

- **P**(θ) or Phase: It adds a relative phase between quantum states. This means a phase difference between two or more states in the computational basis.

$$\text{---} \bullet^\theta \text{---}$$

$$\mathbf{P}(\theta)(a|0\rangle + b|1\rangle) = (a|0\rangle + e^{i\theta}b|1\rangle) \quad (10)$$

- **R_x**(θ), **R_y**(θ) and **R_z**(θ): These gates perform different complex rotations, which allow flexibility to create a given state. Furthermore, they are 1-qubit universal quantum gates, meaning that a combination of them can result in any 1-qubit state. They all have a similar circuit notation. In the case of **R_z**(θ), this gate has the same effect as **P**(θ) as only the relative phase of quantum states is distinguishable.

$$\text{---} \boxed{R_y(\theta)} \text{---}$$

$$\begin{aligned} \mathbf{R}_x(\theta)(a|0\rangle + b|1\rangle) = & \left(a \cos\left(\frac{\theta}{2}\right) - ib \sin\left(\frac{\theta}{2}\right) \right) |0\rangle + \\ & \left(b \cos\left(\frac{\theta}{2}\right) - ia \sin\left(\frac{\theta}{2}\right) \right) |1\rangle \end{aligned} \quad (11)$$

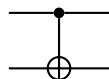
$$\begin{aligned} \mathbf{R}_y(\theta)(a|0\rangle + b|1\rangle) = & \left(a \cos\left(\frac{\theta}{2}\right) - b \sin\left(\frac{\theta}{2}\right) \right) |0\rangle + \\ & \left(b \cos\left(\frac{\theta}{2}\right) + a \sin\left(\frac{\theta}{2}\right) \right) |1\rangle \end{aligned} \quad (12)$$

$$\mathbf{R}_z(\theta)(a|0\rangle + b|1\rangle) = e^{-i\theta/2}a|0\rangle + e^{i\theta/2}b|1\rangle \quad (13)$$

2.2.2 Two-qubit quantum gates and entanglement

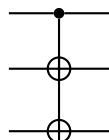
Qubits can create a composed state of different subsystems. We refer to the specific composite state that encodes or stores certain information in a quantum register as $|r\rangle$. If we want to use several qubits, we make use of conditional gates. These gates perform different manipulations conditioned on the state of another qubit. For reasons that we will see later in Subsection 2.2.4, gates with more than one control qubit (multi-controlled gates) are generally hard to implement in real quantum devices, and therefore, two-qubit gates are the largest we will see. Nevertheless, every quantum computation can be performed using a combination of two-qubit and one-qubit quantum gates. This is called universality, more in Subsection 2.2.3. Additionally, we will use matrix notation to write the operators to avoid cumbersome expressions in some cases. In quantum computing, we will call computational basis to orthogonal states $|0\rangle$ and $|1\rangle$, as this is the natural basis we will use for our algorithms. For composed states it is usually used with binary order. For example, for two qubits this is $B = \{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$.

- **CNOT**: Controlled-gate that applies a **NOT** gate to a target qubit when the controlled qubit is $|1\rangle$. This is the most simple controlled-gate and can be translated to an “if” in classical computing. It translates as “apply a **NOT** gate to the target qubit (first) only if the controlled qubit (second) is $|1\rangle$ ”. Such a one-control gate can be generalized to any other gate, applying any gate when a quantum register is in a given basis state.



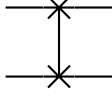
$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

- **CCNOT** or Toffoli: It is a **CNOT** gate with an additional conditional. It is used to do an operation only if both controlled qubits are $|1\rangle$. In this sense, this operation can be seen as a classical AND for a third qubit (target).



- **Swap**: It exchanges the states between two registers, allowing therefore to “transport” information from one register to another. It can be also

implemented with an additional control qubit to make this operation only under certain circumstances.



$$\text{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2.2.3 Universal quantum gates

In classical computation, we can prove that every operation can be performed using just **NAND** and **NOR** gates [26]. Something equivalent can be proved in quantum computation in general and for different hardware technologies in particular [27, 28, 29, 30, 31], having several sets of gates that can perform every computation. When creating a quantum algorithm, different gates or strategies using quantum languages such as Qiskit [32] or Cirq [33] are used, nevertheless, only a few different kinds of gates are necessary. Therefore, real quantum devices only use a universal set of gates implemented in the hardware called native gates when running the algorithms. For instance, this is a list of the native gates (universal) of different real quantum devices, which include more gates than necessary, as this is often more optimal in real computations. The gates we have not seen here are explained in detail in the references, but we will not delve into them here.

- IBM (superconductor): **ECR, I, R_z, SX, X** [34, 35, 36].
- IONQ (trapped-ion): **GPI, GPI2, P, MS, ZZ** [37].
- Quantinuum H2 (trapped-ion): **U_{1q}, R_z, ZZ, RZZ** [38].

The native gate set used in a real device is an engineering decision. For example, certain gates as two-controlled gates are hard to achieve physically. The most common set of universal quantum gates used for quantum algorithms is $U_1 = \text{CCNOT}, \text{H}$, $U_2 = \text{CNOT}, \text{H}, \text{T}$ [39], on the other hand for some applications related to quantum machine learning such as variational quantum circuits a different set is often used $U_3 = \text{R}_x, \text{R}_y, \text{R}_z, \text{CNOT}$ [9].

2.2.4 Real quantum devices

In this subsection, we will briefly discuss additional considerations when using real quantum devices. When a quantum algorithm is constructed using quantum computing languages, these must first be transformed into an assembly language designed to be an intermediate representation for compilers. The most used assembly language is OpenQASM [40]. Once the quantum circuit is described using an assembly language, the transpiler optimizes the code. This optimization includes parameters such as the connectivity of the quantum computer, the native gates, the number of gates, and error correction [41]. In the case of qiskit, the steps are

1. Transform circuit into one and two-qubit gates.
2. Mapping of virtual qubits to physical qubits in the device. As different qubits have different error rates, this process also involves an optimization to avoid noise. Qubits are not usually all-to-all-connected, and therefore, some qubits cannot condition others, as shown in Figure 1 with an example of a 27-qubit connectivity graph.
3. Add additional gates such as swaps to translate information from some qubits to others to match the qubit connectivity.
4. Transform gate set into native gates.
5. Optimize the number of gates using transformations and gates identities which need a lower number of gates giving the same result. Some research in this direction can be found in [42, 43]. Another alternative is to obtain the same results using variational quantum circuits, which can be optimized using parameters [44, 45]. These are active research areas that aims to automate the process in an optimal way .

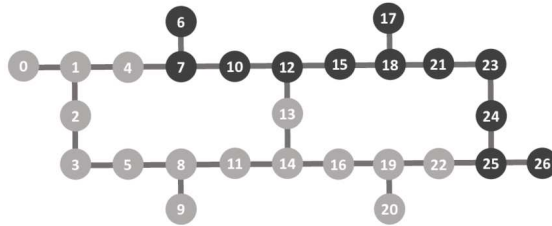


Figure 1: Connectivity of the 27-qubit IBM Quantum device ibmq dublin. The different colors correspond to different error rates.

Indeed, the process of communicating a quantum circuit to quantum devices is more intricate than it might initially appear, as illustrated in Figure 2. This figure provides a detailed view of the quantum stack, shedding light on the complexity involved in this process. One of the most challenging steps in this procedure is noise

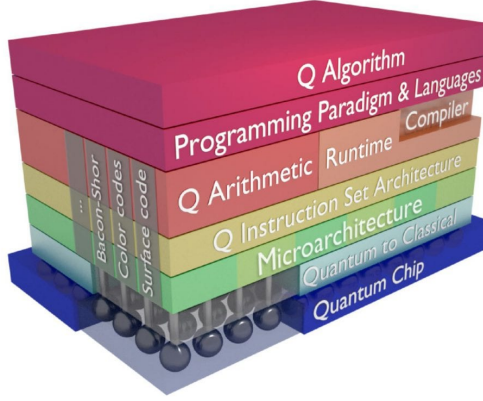


Figure 2: Quantum computing stack to process gates to real operations in the hardware. Figure extracted from [46]

management. Quantum operations in real devices are only sometimes performed as intended. Until now, we have described an idealization. While performing basic operations, noise accumulates as impreciseness in the amplitude and phases, which build up for every computation we make. As incomplete qubit connectivity further increases the number of two-qubit gates, this is a typical measure of the complexity of quantum algorithms: two-qubit gates have higher error rates than single-qubit ones. Errors can be corrected with a relatively high physical qubit overhead (using several physical qubits that act as a logical one), but introducing the theory of quantum error correction is beyond our scope. However, the reader is referred to [47, 48]. It is important to notice that quantum errors can be seen as additional operations on the qubits, disturbing the quantum states. Therefore, if high enough, such accumulating errors can lead us to obtain a completely distorted result. For instance, if a quantum algorithm is represented by an operator $\mathbf{A} : |0\rangle^{\otimes n} \rightarrow |\psi\rangle$, the quantum noise can be seen as $\mathbf{B} |0\rangle^{\otimes n} = |\phi\rangle$. The fidelity (metric to measure the closeness of different quantum states) between the ideal quantum state $|\psi\rangle$ and the noisy one $|\phi\rangle$ is

$$F(\rho, \delta) = \left(\text{Tr} \sqrt{\sqrt{\rho} \delta \sqrt{\rho}} \right)^2 \quad (14)$$

with ρ and δ the density matrices of the states. A density matrix is defined as

$$\rho = \sum_i p_i |\psi_i\rangle \langle \psi_i| \quad (15)$$

where p_i is the probability of having $|\psi_i\rangle$ as output in the computational basis. The fidelity provides information about the error rates of quantum algorithms in a real device. In the ideal case, in the absence of noise fidelity is equal to 1. As quantum

computing resources are scarce and limited, quantum simulators¹ are used along with noise models to replicate the quantum noise behaviour of real devices. This is a very important feature to take into account while building algorithms. However, errors are usually expressed as error rates, which differ not only for different qubits but also for different gates. It is an active research area to reduce the error rates in different technologies without dramatically affecting the speed or number of qubits needed [50].

There is an accelerating trend of decreasing error rates and increasing number of qubits. A useful metric to measure this progress is the quantum volume and its generalizations [51]. In this context, it is crucial to clarify and standardize certain terminologies for the coherence of this thesis, in specific on the terms *noisy intermediate-scale quantum* (NISQ) and *fault-tolerant*, which are frequently confused due to our current position in an intermediate stage of quantum computing development. Let's take a moment to clearly delineate these concepts. The term NISQ was originally defined by Preskill [52] and it is defined as

Definition 4. *A noisy intermediate-scale quantum computer is a quantum computer that is intermediate (its number of qubits ranges from 50 to a few hundred) and noisy (with imperfect control over those qubits).*

This definition was established in 2018, five years before the present thesis and since then, new quantum computers have been designed and built. Since then, works redefining with more precision the terms NISQ has been done [53, 54]. As they did, we will distinguish quantum computers by their number of qubits and error rates in terms of two-qubit gates. These concepts are very related to each other as the number of physical qubits required to form a logical qubit² will reduce the effective noise of the model. Therefore it is important to distinguish between individual qubits errors and logical qubits. In particular small quantum computers refer to computers that can be simulated by a supercomputer. This definition will change according to classical capabilities, the current limit being around 60-65 qubits [55]. Intermediate will be used to refer to quantum computers from the mentioned lower bound up to a scale where we can do simulations with real applications with logical qubits without noisy. In this regard, the number will depend on the algorithm used, and therefore, it is better to use an alternative definition. The number of physical qubits needed to have low error rates (10^{-6}) logical qubits for real industrial applications is estimated to be 100-1000 [56] with physical error rates of (10^{-2} to 10^{-3}). A good definition for large quantum computer could be the number of physical qubits for having 60 logical qubits with high fidelities for

¹Quantum simulators are software programs that allow you to use a classical computer to run quantum circuits as if they were being run on a quantum computer. These simulators also allow to replicate at to some extent quantum noise from real devices, connectivity and transpilation. More information in [49].

²A logical qubit, utilized in quantum algorithms, is a computational qubit unit whose noise is mitigated through the collective use of multiple others ancillary qubits.

industrial applications, overpassing supercomputer capabilities. Therefore we will use 6000 qubits to name a large quantum computer opposite to Preskill’s definition. In terms of noise, we can define three categories: Noisy for nonreliable qubits with less than 99.9% fidelity. Most algorithms need a high number of gates and these noise levels are considered unreliable. We will define near-perfect quantum computers for those with noise ranging between 10^{-3} to 10^{-6} and fault-tolerant for quantum computers with less than 10^{-6} error rates. Specifically, if both large and fault-tolerant are satisfied, we will call such a quantum computer a practical error-corrected quantum computer (PQC).

Currently, the largest quantum device is IBM Condor with 1121 qubits. On the other hand, the quantum computer with the lowest error rates is Quantinuum H2 with a two-qubit gate error rate of 99.8% fidelity, and therefore not near-perfect (but close) according to our definitions. Therefore according to our definitions, we are yet in a NISQ era, which is compatible with the current difficulties to obtain real advantage quantum algorithms. However, error rates are being reduced drastically, and the close relationship between error rates and the number of qubits (as the number of physical qubits can reduce the logical qubits using methods such as surface code [57]). We can conclude that we are in a moment of transition from intermediate and noisy quantum computers to large and near-perfect qubits in the following few years. More information about other quantum processors mentioned and others is available [3]. Finally, it is important to mention that when discerning the capabilities of a quantum computer, we must take into account the connectivity. Not all qubits have the same error rates and the effective fidelity to perform an algorithm is decreased by poorly-connected architectures with a low average degree of the associated graph.

2.3 Quantum emulators

Executing algorithms in a quantum computer is currently expensive and for most users unaccessible due to the reduced number of machines yet made. However, we can still test and benchmark quantum algorithms using classical computers. To do so, we use a process called quantum emulation or quantum simulator, where the classical computer, simulates/emulates the behaviour of a quantum computer.

Many simulators are available in the quantum ecosystem, with open-source tools like Qiskit and Cirq. From a classical point of view, classical emulators compute quantum algorithms mimicking the algebra of a quantum computer. In order to clarify this point, we will present here few examples:

A quantum computer is based on unitary operations applied on an exponential space. First, a qubit is a quantum computational unit representing two possible states. Using Dirac notation, we can write $|0\rangle$ and $|1\rangle$. If we want to manipulate

an state

$$|\Psi\rangle = (c_1 |0\rangle + c_2 |1\rangle)$$

, we need to apply an unitary operator which gives as a result

$$\mathbf{U} |\Psi\rangle = |\phi\rangle = (d_1 |0\rangle + d_2 |1\rangle)$$

. This is done classically using $2^N \times 2^N$ matrices, with N the number of qubits. These computations can also be applied to superposition of states. For example, given the previous example if we have instead N qubits with a superposition of states,

$$|\Psi\rangle = \sum_{i=0}^{N-1} 2^N (c_i |q_i\rangle)$$

, such that c_i are coefficients that make the vector unitary, we can obtain another vector

$$|\phi\rangle = \sum_{i=0}^{N-1} 2^N (d_i |q_i\rangle)$$

, where the probabilities of obtaining each vector have changed. In this sense, the classical computer is giving us the same results as the quantum computer. However, notice that as the number of qubits increases, the matrix we computer get exponentially larger, limiting the capabilities of the calculations. Usually, the maximum number of qubits we can simulate in a classical computer is $N = 20$, corresponding to square matrices with a dimension of one million. In contrast, real quantum computers could do these operations only using linear amount of resources with the number of qubits, instead of computing linearly with the possible states we generate. In the case of a classical simulator, given a quantum gate (unitary operator), all the possible states are computed.

2.4 Quantum Algorithms

Quantum algorithms consist of three key steps: Quantum state preparation (to load data into quantum registers), the main algorithm, and measurements (to extract the results). In this subsection we will briefly describe how data is encoded into quantum registers and extracted, exploring the algorithms for computational fluid dynamics in the following sections.

2.4.1 Quantum state preparation

To make quantum algorithms, we often use classical data that we want to represent in a quantum register. This means we need to encode classical information into given quantum states in a computational basis, which depending on the application

may include, a specific amplitude and phase. In general, we can prepare states (also called initialization) by first distributing the probability and then marking the states on a computational basis if necessary. To distribute the probability, we can perform an algorithm based on a binary tree, with each node being a different state and the depth encoding each gate used. A naive approach consists of $\mathbf{CR}_y$ gates to achieve this, conditioning every node in the tree to generate new nodes distributing the probability. Then we can mark states using $\mathbf{CNOT}$ and $\mathbf{X}$ gates. There are two main strategies to encode quantum information: computational basis and amplitude. The first approach is based on storing information in the basis regarding the amplitude, being able to measure a state at least once. On the other hand, in the second case, the amplitude is important, and we need to measure it many times to have a good approximation of its exact value. For example, we can prepare a quantum circuit with two qubits as $|\psi\rangle = \alpha_1 |00\rangle + \alpha_2 |01\rangle + \alpha_3 |10\rangle + \alpha_4 |11\rangle$ and then use a third qubit which is either $|0\rangle$ (junk state³) or $|1\rangle$ for each of the basis states described before. In the computational basis encoding, the information would be stored in the third qubit describing which of the basis states are junk states. However, if we are also interested on the probability of each state and not only in the third qubit, then we would be using an amplitude encoding (amplitudes encode information).

Quantum state preparation is a bottleneck in quantum algorithms pursuing quantum advantage producing $\mathcal{O}(2^n)$ number of $\mathbf{CNOT}$ gates, where n is the number of qubits [58]. This is a problem as quantum algorithms have the advantage of producing $N = 2^n$ computations using an entangled superposition of states and operators acting on it. This is especially a problem if an algorithm needs several calls to initialization primitives, destroying the possible advantage. The standard approach consists of $\mathbf{R}_z$, $\mathbf{R}_y$ and $\mathbf{CNOT}$ gates to create a desired superposition. This is the case of Shende, Bullock, and Markov (SBM) method [59], improving earlier studies by a factor of 2. However, reducing the computational complexity of this critical step is ongoing research [60, 61]. Several studies offer algorithms and procedures to solve this issue, at the expense of an exponential number of qubits, sacrificing space for reducing the time [62, 63]. In the latter case, the algorithm shows to provide non-exponential dependencies for the case of sparse quantum states, which have a super-polynomial number of non-zero entries [61, 64]. Depending on the algorithm, this could provide an advantage (reduce computational complexity compared to classical algorithms) for special cases of quantum states. Recent studies also show that advantage is possible without using an exponential number of ancilla qubits⁴ A footnote that explains ancilla qubits and their distinction from working qubits would be a nice addition. for structured data like functions [65, 66]. Finally, another subset of states that have shown to offer a quantum advantage is basis-encoded states with equal superposition [67, 68],

³Junk state refers to every state created during a quantum algorithm whose information is not useful but cannot be erased from the superposition due to the no-deletion theorem

⁴Ancilla qubits are quantum memory units being used to implement irreversible logical operations on the working qubits registers, otherwise not possible to perform

reducing the time and number of gates greatly.

2.4.2 Measurement

Measurement is a necessary step to obtain results in quantum algorithms. While superposition offers a potential quantum advantage by encoding $m = 2^n$ basis states into n qubits, these results are not accessible directly. In specific, the amplitudes and phases cannot be extracted precisely, that is, the state of the system can be read only projectively and only once. As we saw, if we have m possible states, the squares of the amplitudes of these states will give us the probability of obtaining a particular result. The measurement operator $\mathbf{M}$ could differ from the computational basis⁵, but we will stick with it as it is the standard, used in quantum programming languages like Qiskit.

Once the measurement is performed, we will only obtain a single result, whose probability is given by (5). Therefore, only one result is obtained after a measurement, needing more measurements to retrieve the rest of the information. This means that the circuit must be executed several times to obtain more results following a probability. The number of times a state is obtained over the total number of executions will give the probability of obtaining each state. Regarding how the state is encoded, in general, a binary string $b = \{c_1, c_2, \dots, c_{2^n}\}$ corresponding to decimal $1, 2, \dots, 2^n$ would be stored as

$$|\psi\rangle = \sum_{i=0}^{2^n-1} a_i |b_i\rangle$$

with $c = 1, 0$ and a_i amplitudes associated to state $|b_i\rangle$ summing over all binary states described. Estimating a quantum superposition of entangled states with arbitrary precision is possible, nevertheless, this method needs an exponential number of measurements, destroying quantum advantage. This is called quantum (state) tomography [70]. However, extracting particular information of interest such as measuring statistics of certain subsets of qubits is possible with fewer measurements [71]. Finally, another strategy usually made to reduce the number of measurements is the use of expectation values based on operators to extract useful information only using in certain qubits. In the context of computational fluid dynamics (CFD), this allows us to obtain variables of interest such as velocity, density, or pressure of small regions without the need to measure every qubit. This makes CFD a field where quantum algorithms are especially useful as we can extract large-scale variables without resolving small scales, using expectation value operators. Several studies focus on efficient estimation of these operators [72, 73].

⁵In quantum computing, quantum basis are usually defined as eigenstates of an operator over a qubit. In the case of the computational case, this is the Z pauli operator. Other common states such as $|+\rangle$ and $|-\rangle$ states can be used. More information over quantum base states in [69]

3 Quantum Lattice-Gas Automata

The first quantum algorithm we will discuss and the main focus of this thesis is the Quantum Lattice-Gas Automata (QLGA). This chapter will briefly define Lattice-Gas Automata (LGA) in its classical version and comment on the rules and properties of the two principal models: Hardy–Pomeau–Pazzis (HPP) and Frisch–Hasslacher–Pomeau (FHP). Then, we will do a bibliographic review of QLGA, describing the theory and what is yet to be done.

3.1 Introduction to Lattice-Gas Automata models

This chapter will introduce the basic notions of LGA and their properties. This will help us understand the quantum models we will present later. The overview will be based on [74, 75]. We will focus specifically on the microscopic evolution of the system.

Lattice-gas automaton is a special kind of cellular automaton (CA). We can describe CA as a set of synchronized identical finite automata⁶ with the same evolution rules, which exchange information with a subset of automata according to a graph that establishes their connectivity. Another essential feature is synchronization, meaning every automaton simultaneously updates its state. Therefore, the sense of past and future is the same for every automaton. More on CA can be found in [76]. In the following section, we will define the specific characteristics of LGA and note their difference from general CA. The LGA is composed of

- A regular lattice of cells, in general, a Bravais lattice, covering a k -dimensional space. The importance of the Bravais lattice stems from its specific lattice symmetries, which are often crucial for the simulation of partial differential equations. Specifically, every automaton will be geometrically tied to each of these cells, having a one-to-one mapping between the cells and the automata. Therefore, we will refer to the automata cells as nodes.
- A set of m boolean variables $n(r, t)$, called channels such that $n \in \mathbf{C}$ with $\mathbf{C}$ the set of all channels, $r \in \mathbb{R}^k$ the regular lattice of cells and t the time. The channels provide information on the direction and velocities of propagation. Therefore, the local state of each automaton can be represented by m channels with 2^m possible states. We will note these states from zero to $m - 1$ in binary encoding. Since every node has the same amount of channels, their notation will be node-independent, while their occupation state may

⁶A finite automaton is a mathematical model of computation where the abstract machine can only be in a finite number of states at a given time

be different. We will note by 1_i to the i th channel when occupied or 0_i when unoccupied.

- The elementary evolution process of the LGA consists of two parts: Collision and propagation separated by a regular time increment Δt . During the collision, the information of each automaton will change depending on its internal state, and no information will be exchanged with their neighbours (the nodes suffer isolated transformations). This transformation will be node-independent except for the boundaries, which can include special rules. For the second step, a transfer of partial information between nodes will occur depending on the specific geometry and connectivity defined by the lattice. From the node perspective, their information at time t is lost, transferring it to their neighbourhood, with the information of each channel being sent in different directions. We will interpret the channels' information as particles with extra momentum.

One of the principal differences between LGA and CA is the former geometrical structure due to particle velocities (one per channel) and their definition over a Bravais lattice, which allows modelling of the behaviour of real fluids due to their space-and-time-independent behaviour.

There are different variations of this model. The boolean nature of the algorithm (exclusion principle) is imposed to guarantee that two indistinguishable particles cannot reside in the same node at the same time. Still, this consideration can change by counting how many particles of specific channels we have per node. Therefore, from particle physics perspective, the model can be fermionic or bosonic. As commented by Rivet and Boon [74], non-local interactions can be modelled as local if more channels are used, as proven in their book Lattice Gas Hydrodynamics (Appendix A.1). They also found that the order of collision-propagation or propagation-collision does not affect the model, provided that enough time steps are taken at a macroscopic level (chapter 1.3.6 Lattice Gas Hydrodynamics). Finally, even though the collision rules are generally defined deterministically for LGA, this is not necessarily the case. For example, in FHP some collision rules are probabilistic, introducing stochasticity in the model. However, non-deterministic dynamics can be simulated with deterministic regulations at the expense of more time steps to reach a stationary state. Before introducing the collision and shift rules, it is essential to highlight a few properties that we want when discussing the quantum version.

Property 1: The collision probability over all possible states $\mathbf{A}$ must follow detailed-balance. This means that $\forall n, n' \in \mathbf{C}, \mathbf{A}(\mathbf{n} \rightarrow \mathbf{n}') = \mathbf{A}(\mathbf{n}' \rightarrow \mathbf{n})$. Noting the transformation from n to n under the collision operator $\mathbf{A}$ as $\mathbf{A}(\mathbf{n} \rightarrow \mathbf{n}')$.

Property 2: The collision probability over all possible states $\mathbf{A}$ must follow semi-detailed balance. Meaning that $\forall n \in \mathbf{C}$, $\sum_{n' \in \mathbf{C}} \mathbf{A}(\mathbf{n}' \rightarrow \mathbf{n}) = 1$.

Property 3: An observable q will be a collisional invariant at a time t and site r defined by the Bravais lattice if the amount of q does not change after the collision step. This means that the amount does not change even though the information is propagated over the neighboring b . We can write this mathematically as

$$\sum_{i=1}^b q_i n_i(r + c_i, t + \Delta t) = \sum_{i=1}^b q_i n_i(r, t) \quad (16)$$

Property 4: The set of channels $\mathbf{C}$ with the same value for a specific conserved variable q_i form an equivalence class $E = \{e_1, e_2, \dots, e_s\}$ with s the amount of channels within the class. In this way, the collision step always consists of binary permutations within E following the conservation laws of a certain physical model. For example, if the mass encoded in every channel is the same ($m = 1$), they will form an equivalence class. Another typical example with these models is the velocity, if two channels have the same velocity in the x-axis v_x , they will form an equivalence class. This enormously restricts the possible collision rules of the model.

From these microdynamical rules, we can derive macroscopic variables. Non-boolean macroscopic variables can be obtained by averaging over a subset of lattice sites. For example, we can define, respectively, the average density and particle current as

$$\rho(\bar{\mathbf{x}}, t) = \sum_{i=1}^C \langle n_i(\bar{\mathbf{x}}, t) \rangle \quad (17)$$

$$\bar{\mathbf{v}}(\bar{\mathbf{x}}, t) = \sum_{i=1}^C c_i \langle n_i(\bar{\mathbf{x}}, t) \rangle \quad (18)$$

with $\langle n_i \rangle$ noting the average over an ensemble (lattice sites or different experiments) and c_i velocity of channel i . The connection between the microscopic boolean dynamics and the macroscopic continuous world must be established to assess the model's validity. However, the derivation is lengthy and complicated and will not be introduced here. See [75] for details. Using a statistical model called Chapman-Enskog expansion, we can derive quantities such as the viscosity or solve the lattice Boltzmann equation that exactly describes the kinetic transport at the mesoscopic scale in a quantum lattice gas. This last equation is derived by averaging boolean microdynamics as stated in Lattice-Gas Hydrodynamics book. variables over an ensemble and reads as

$$\langle n_i(\bar{\mathbf{x}} + \bar{\mathbf{e}}, t + \Delta t) \rangle - \langle n_i(\bar{\mathbf{x}}, t) \rangle = \langle \Omega \rangle \quad (19)$$

where $\langle \Omega \rangle$ is the total collision term. More details about this in the introduction of Appendix A A. In the following section, we will introduce the three main models for LGA: D1Q3, HPP and FHP.

3.1.1 D1Q3

The most simple non-trivial lattice-gas automata model is the D1Q3, which is one-dimensional and with three channels in every lattice site.

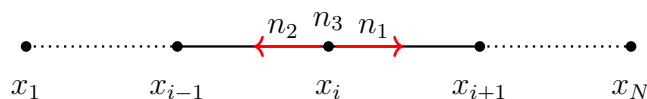


Figure 3: D1 lattice used in D1Q3 model with their corresponding channels.

The 1D was thought originally to have two different channels, the first with a particle with right velocity and the second with left. However, this leads to a probabilistic profile with different values on even and odd lattice sites. In order to smooth the results, the rest particle was added. The collision rule is based on three channels and includes an effective collision rate of 25% as only two states undergo a collision of the eight total number of channels. Using the notation $(c_1, c_2, c_3) = (\rightarrow, \leftarrow, \cdot)$ referring to the right, left and resting particles the collision rule will consist of a transformation from $(1, 1, 0) \rightarrow (0, 0, 1)$ and $(0, 0, 1) \rightarrow (1, 1, 0)$. The transformation is reciprocal as both states has the same total velocity and mass and therefore form an invariance group. This means that the mass of the resting particle $m_3 = m_1 + m_2$.

Table 1: D1Q3 lookup table

Initial State		Final State
000	$\rightarrow$	000
001	$\rightarrow$	110
010	$\rightarrow$	010
011	$\rightarrow$	011
100	$\rightarrow$	100
101	$\rightarrow$	101
110	$\rightarrow$	001
111	$\rightarrow$	111

3.1.2 HPP

HPP was the first LGA introduced (1973) as a model with applicability for statistical physics [11, 12]. The model consists of a square grid in two dimensions with four different channels representing four velocities with directions $c_1(1, 0)$, $c_2 = (-1, 0)$, $c_3 = (0, 1)$, $c_4 = (0, -1)$. This is because the connectivity of the nodes B in the Bravais lattice must be the same as the number of channels in the set $\mathbf{C}$. The collision operator only affects two of the possible 2^B combinations,

$$\begin{aligned}(1, 1, 0, 0) &\rightarrow (0, 0, 1, 1) \\ (0, 0, 1, 1) &\rightarrow (1, 1, 0, 0)\end{aligned}$$

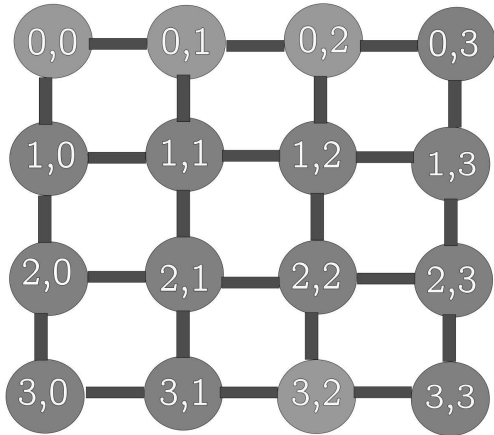
therefore exchanging incoming left and right particles to up and down and viceversa. There are three quantities conserved: mass m , and momenta p_x and p_y .

In lattice-gas models, microscopic collisions give rise to macroscopic non-linear terms, such as the convection term of the Navier-Stokes equations. The main disadvantage of the model is its anisotropy due to the limited number of channels and low effective interactions [77]. This makes the model limited compared to more complex ones such as FHP.

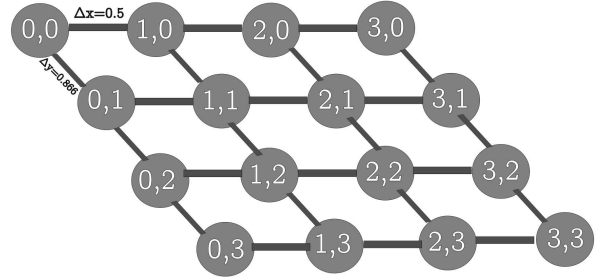
The HPP simulation literature is scarce as more complex models are usually preferred for real applications. Reference [78] explores the possibility of simulating the propagation of a sound wave in a pipe with obstacles and wall absorption, using a modification of HPP with rest particles. In another work [79], several simulations in electromagnetics using HPP, and HPP with rest particles, are studied. The research found that non-normal incident pulses behave poorly, while incident ones correspond closely with analytical results. In a third work [80], a course-grained procedure is studied with a modified HPP with non-conservative momentum rules to simulate turbulence without the smallest scale resolution efficiently. The possible applications of HPP in medical applications have also been explored [81] finding a promising algorithm for image visualisation in ophthalmic research and practice.

3.1.3 FHP

Developed by Frisch, Hasslacher and Pomeau, FHP is based on the target of producing proper isotropy in the large-scale dynamics [82]. FHP constitutes a family of models, all based on the same triangular Bravais lattice consisting of six channels. The FHP models include, for example, FHP-I, FHP-II, FHP-III, CFHP [83], GBL [84], and BIO-LGCA [85], among others.



(a) Square lattice used for HPP model.



(b) Hexagonal lattice used for FHP. Notice that in this case the encoding does not identify the real distance. In order to do so, we need to take into account the factors $\Delta x = 0.5$ and $\Delta y = \frac{\sqrt{3}}{2} \approx 0.866$

Figure 4: Square lattice neighbours (HPP Bravais lattice) compared to Hexagonal lattice neighbours (FHP Bravais lattice). Figure from [86].

The channel directions in Cartesian components are $c_1 = (1, 0)$, $c_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_3 = (-\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_4 = (-1, 0)$, $c_5 = (-\frac{1}{2}, -\frac{\sqrt{3}}{2})$, $c_6 = (\frac{1}{2}, -\frac{\sqrt{3}}{2})$. Some of the FHP models differ in the number of channels, as increasing this number increases the effective collisions⁷. We can see the collision rules of FHP-I, FHP-II, and FHP-III in Figure 5. The increment of collisions allows us to model fluids with lower viscosity and, therefore, a higher Reynolds number Re , where Reynolds number is defined as $Re = \frac{L\bar{u}}{\nu}$, with L the characteristic length, $\bar{u}$ the mean velocity and ν the viscosity. Additionally, FHP models include non-deterministic collision rules, but as indicated in [75], deterministic collision rules may give the same result:

"Note that a fully deterministic FHP model can also be considered to avoid generating random numbers to select among the possible outcomes of the two-body collision. The usual strategy for this purpose is to alternate between two outcomes."

Special collision rules can also be included to replicate the wall-particle interactions at the boundaries, but unless otherwise stated, we will use open boundary conditions. We distinguish three different collisions: Bounce-back, specular and diffusive reflections. The bounce-back reflection operator sends the particle where it came from. Specular reflection makes the particle bounce elastically on the surface, where the normal momentum concerning the wall is reversed. Finally, the diffusive reflection operator combines the former two with specific probabilities.

⁷We define the effective collision ϵ_c as the ratio between the number of states transforming the local information of the node after the collision operator n_c and all possible states N . Therefore $\epsilon_c = \frac{n_c}{N}$

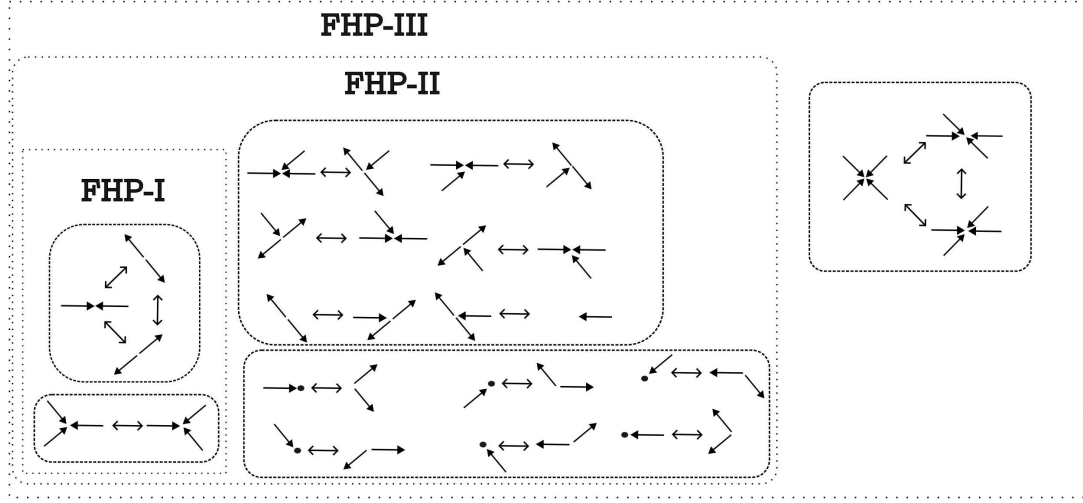


Figure 5: FHP-I, II, and III collisions rules. Each model includes new collision rules and those used in the previous model. When a collision has two possible outcomes, the probability of having each result is equal. Additionally, we used a dot as a notation for a rest particle, when the rest particle does not appear in a collision, it means the result does not change whether it is present or not. Notice that the FHP-III model can include more collision rules or with other combination of probabilities and the definitions differ from author to author [74, 77, 87]. As we will develop a FHP-III version with six particles for our quantum model in section 6, we included the reduced number for FHP-III.

FHP lattice-gas automata have been proven to replicate successfully Navier-Stokes equations [88]. However, after deriving the macroscopic evolution of the boolean model, several anomalies were discovered, making FHP unpractical for fully developed turbulence flows. Specifically, lack of Galilean invariance⁸ and anomalous fluid velocity dependence of the fluid pressure is observed, not being possible to avoid both simultaneously. These problems are caused due to the limited range of velocities among channels (every channel has the same velocity). To prevent these issues, new models have been proposed [90, 91, 92]. Additionally, high noise and low Reynolds numbers are encountered due to the discretisation of the phase-space and low effective collisions. Quantum computing can solve these last problems by providing many lattice sites due to the exponential encoded states. An additional issue related to FHP model is its limitation to be generalised to 3D, needing to expand the method to four dimensions to accomplish this [13]. Another paper by Klales develops FHP to curve surfaces [93]. Finally, to not extend excessively this introduction, we will refer the reader to more studies using FHP in computational physics [85, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103].

⁸In the context of LGA, the lack of Galilean invariance refers to the phenomena observed when the mean velocity of the fluid changes with respect to the lattice. When the same initial conditions are imposed modifying the flow velocity in one direction, the behaviour obtained changes. In specific, a quadratic term proportional to the square of the sound velocity appears. This behaviour is not characteristic of Navier-Stokes equations and therefore it is considered a deficiency of the model [88, 89]

3.2 QLGA bibliographic review

This section will review the scientific research on quantum lattice-gas automata. Due to the early transition to LBM, the research on quantum computing has also been based mainly on it. However, QLGA has gained interest in the community for its potential usage to simulate non-linear terms quantum natively despite the linearity intrinsic to quantum mechanics. We will divide the review in three sections. First we will discuss if it is possible to build a quantum LGA in a quantum computer. At the time, it was thought that there is no unitary operator that be used to simulate QCA. However, this restriction was proven to be avoidable weakening the translation invariance from quantum physics. Second the next step was to derive a quantum mechanical description of the mesoscopic transport equation. Finally, once the theoretical foundations were set, quantum algorithms began to be designed. The final part of this overview makes a summary of the different approaches and their advantages and limitations.

3.2.1 Is it possible to build a quantum LGA?

The first author to introduce a quantum model for LGA was David Meyer [104]. Meyer argues that quantum computing has great potential for solving classical computational problems and highlights the importance of finding a unitary operation compatible with quantum mechanics to solve quantum algorithms. Specifically, he believed that a quantum computer based on arrays of simple devices would be more accessible to design and build than a single, more complicated one. From here, he established parallelism between quantum computers and quantum cellular automata (QCA), which motivates his study, relying on what was described later as type-II quantum computers⁹. His most relevant contribution was to prove that QCA can be computed in a quantum device compatible with quantum mechanics, the opposite of what was believed at the time [105, 106]. This was known as the No-Go Lemma that says

Lemma 1. *In one dimension there exists no nontrivial, homogeneous, local, scalar QCA. More explicitly, every band c -diagonal unitary matrix (collision operator) which commutes with the 1-step translation matrix is also a translation matrix times a phase.*

This theorem means that at a quantum level, the total evolution operator $\mathbf{W}$ composed by a collision operator U and translation operator T would result in a

⁹A Type-II quantum computer is a complex network composed of multiple smaller quantum computers, each interconnected through classical communication channels. Within this network, each qubit is capable of entanglement, but only with those close to the same node of the quantum computer. Moreover, this entanglement is transient, lasting only briefly. We can interpret each quantum computer in the network as a node

translation matrix times a phase ϕ , which does not effectively provide an interaction between particles. This reads as

$$\mathbf{W} = TU = e^{i\phi}T \quad (20)$$

However, Meyer argues that a slight weakening of the translation invariance could overpass this problem. Given that a classical CA can be expressed generally as

$$\phi_{t+1}(x) = \sum_i f(\phi_t(x + e_i) : e_i \in \mathbf{E}(\mathbf{t}, \mathbf{x})) \quad (21)$$

with $\phi_t(x)$ the state of the cellular automata at time t and position x over a lattice $\mathcal{L}$ under a transformation $f(\phi)$. The notation $\mathbf{E}(\mathbf{t}, \mathbf{x})$ refers to the local neighbourhood of the cell with e_i one of its neighbours. In general, this transformation can be expressed in terms of the unitary evolution of a state vector defined in a given Hilbert space as

$$|\phi(t+1)\rangle_i = \sum_j^N w_{ij} |\phi(t)\rangle_{j=1} : i \in \mathcal{L} \quad (22)$$

with w_{ij} the elements of the operator $\mathbf{W}$. This operator must be sparse (most terms are zero) due to the low degree of the Bravais lattice associated to LGA. Therefore if $(i, j) \notin \mathbf{E}$, then $w_{ij}=0$. Nonetheless, as discussed, this is impossible due to the Lemma 1. However, this can be overpassed by weakening the translational invariance to the second order. In standard quantum mechanics, the dynamical evolution of a quantum state gives the same result whether we apply a translation operator T before or after a unitary operator U encoding the collision term of LGA. This means that the operators commute. This can be written mathematically as $UT = TU$. A way to relax this is to set this condition to $T^2U = UT^2$, allowing to execute shifts of particles over a lattice and unitary dynamical transformations where the order of the operations translates into different results. This assumption makes possible to construct a unitary where the total evolution operator $\mathbf{W} \neq e^{i\phi}T$. Therefore, for 1D, $\mathbf{W}$ can be written as a block diagonal matrix S composed of

$$S = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad (23)$$

with $a, b, c, d \in \mathbb{C}$ acting on pairs of connected lattice sites. Equation (23) shows that every lattice site is only affected by another. However, this can be expanded as a two-component quantum cellular automata by a linear combination of two pairs. This allows to model the lattice dependencies better as each vertex in the connectivity graph has two neighbours. The expression reads as

$$|\phi(t+1)\rangle_i = \begin{pmatrix} 0 & i\sin(\theta) \\ 0 & \cos(\theta) \end{pmatrix} |\phi(t)\rangle_{i-1} + \begin{pmatrix} \cos(\theta) & 0 \\ i\sin(\theta) & 0 \end{pmatrix} |\phi(t)\rangle_{i+1}. \quad (24)$$

Finally, Meyer sets the foundations of QLGA, expanding his research from CA to the former. This is done by generalising the two-component QCA to a model with different states representing the occupancy of α number of particles. Without loss of generality, he focuses on 1D. The unitary diagonal block in the computational basis reads as

$$S = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & ie^{i\alpha}\sin(\theta) & e^{i\alpha}\cos(\theta) & 0 \\ 0 & e^{i\alpha}\cos(\theta) & ie^{i\alpha}\sin(\theta) & 0 \\ 0 & 0 & 0 & e^{i\beta} \end{pmatrix} \quad (25)$$

In this case, the matrix is 4×4 as we are encoding the unitary evolution of two channels per lattice site with four possible combinations. Later, Meyer's procedure was used to simulate the Schrödinger equation for one and two-particle systems [107, 108]. This proof and unitary evolution encouraged further research of the quantum model, which led later to the derivation of the quantum mechanical description of the kinetic transport equation for the mesoscopic scale.

3.2.2 Mesoscopic QLGA and factorisation

The microscopic quantum evolution of the particles via entanglement and superposition can effectively represent the dynamics at larger scales. This is exactly what Jeffrey Yepez showed in 2001 [109].

QLGA can be described using collision and streaming operators over the quantum state of the whole lattice

$$|\Psi(\bar{\mathbf{x}})\rangle = \bigotimes_{i=1}^L |\psi(x_i)\rangle$$

with dimension 2^L , being L the number of lattice sites, which set is defined by the vector $\bar{\mathbf{x}} = (x_1, x_2, \dots, x_N) \in \mathcal{L}$ such that

$$|\Psi(\bar{\mathbf{x}}, t + \delta t)\rangle = \mathbf{TC} |\Psi(\bar{\mathbf{x}}, t)\rangle \quad (26)$$

As previously introduced by Meyer, as stated in section 3.2.1, the collision operator $\mathbf{C}$ can be modelled as a complex block diagonal unitary with the scattering matrix S from (30) as a diagonal block. This means that within a node, different channels are associated each with distinct local states. These states, denoted as $|\psi(x_i, t)\rangle_c$, exhibit entanglement and possess dimensionality of 2^{N_C} , with N_C the number of channels. This is not the case for the shift operator $\mathbf{T}$, which is done via classical communication using type-II quantum computers in this research's implementation. Given that the qubits corresponding to the channels are not entangled, we can decompose the collision operator in unitaries of dimension 2^{N_C} acting on each node.

$$\mathbf{C} = \bigotimes_{i=1}^L \mathbf{S}_{\mathbf{x}_i} \quad (27)$$

We can sum up the local and global transformations respectively with the following equations .

$$\widetilde{|\psi(x_i, t)\rangle} = \mathbf{S}_{\mathbf{x}_i} |\psi(x_i, t)\rangle \quad (28)$$

$$\widetilde{|\Psi(\bar{\mathbf{x}}, t)\rangle} = \mathbf{C} |\Psi(\bar{\mathbf{x}}, t)\rangle = \sum_{i=1}^{2^L} \alpha_i (\mathbf{S}_1 |\psi_1\rangle \otimes \dots \otimes \mathbf{S}_L |\psi_L\rangle)_i \quad (29)$$

using the symbol tilde to mark the post-collision or outgoing state, x to a given lattice site and $\bar{\mathbf{x}}$ to the set of lattice sites. It is essential to notice that in this case, the total number of qubits is $N = L \times N_C$.

Once the microscopic transformation is formulated via equations (23), (28) and (29), we are ready to derive the mesoscopic transport equation, which will be the same as for the classical case. Following Yepez's derivation, let us consider two global lattice states (neighbouring each other) $|\Psi_1(\bar{\mathbf{x}})\rangle$ and $|\Psi_2(\bar{\mathbf{x}}')\rangle$ with $\bar{\mathbf{x}}' = \bar{\mathbf{x}} + \bar{\mathbf{e}}$, however as a notation we will write from now on the spatial dependency implicitly. Let us suppose the model is 1D and that after the shift, the particle moves from $|\Psi_1\rangle$ to $|\Psi_2\rangle$. This means

$$\langle \Psi(t) | \mathbf{n}_1 | \Psi(t) \rangle = \langle \Psi(t) | \mathbf{S}^\dagger n_2 \mathbf{S} | \Psi(t) \rangle \quad (30)$$

with $\mathbf{n}_i$ the number operator which acts under a local state as

$$\mathbf{n}_i |\psi(x_j)\rangle = \begin{cases} |\psi(x_j)\rangle & \text{if site } x_j \text{ is occupied by the particle's channel } i \\ 0 & \text{if site } x_j \text{ is unoccupied by the particle's channel } i \end{cases} \quad (31)$$

As we are using only one particle for this example, the collision operator is trivial, but we will derive the equation without any loss of generality. The number operator can also be generalised to a $2^{N_C} \times 2^{N_C}$ dimension operator. Using the property that the operators are unitary and therefore $\mathbf{T}^\dagger = \mathbf{T}^{-1}$

$$\langle \Psi(t) | \mathbf{C}^\dagger \mathbf{n}_1 \mathbf{T}^\dagger | \Psi(t + \Delta t) \rangle = \langle \Psi(t) | \mathbf{C}^\dagger \mathbf{n}_1 \mathbf{C} | \Psi(t) \rangle \quad (32)$$

connecting the states at time t and $t + \Delta t$. Using again the unitarity of the shift operator and equation (30), we know that $\mathbf{n}_1 \mathbf{T}^\dagger = \mathbf{T}^\dagger \mathbf{n}_2$. Using this relation and equation (32), we have

$$\langle \Psi(t) | \mathbf{C}^\dagger \mathbf{T}^\dagger \mathbf{n}_2 | \Psi(t + \Delta t) \rangle = \langle \Psi(t) | \mathbf{C}^\dagger \mathbf{n}_1 \mathbf{C} | \Psi(t) \rangle \quad (33)$$

The operators from the left-hand side of the equation can be written implicitly using equation (23) as

$$\langle \Psi(t + \Delta t) | \mathbf{n}_2 | \Psi(t + \Delta t) \rangle = \langle \widetilde{\Psi(t)} | \mathbf{n}_1 | \widetilde{\Psi(t)} \rangle \quad (34)$$

If we define now the classical probability of occupancy associated with any number operator $\mathbf{n}_i$ as

$$f_i(t) = \text{Tr}(\rho(\mathbf{t}) \mathbf{n}_i) \quad (35)$$

with $\rho(t) = |\Psi(t)\rangle \langle \Psi(t)|$, then we can write (34) as

$$f_2(t + \Delta t) = f_1(t) + \langle \Omega_1(\Psi) \rangle, \quad (36)$$

where $\langle \Omega_1(\Psi) \rangle = \langle \Psi(t) | \mathbf{C}^\dagger \mathbf{n}_1 \mathbf{C} - \mathbf{n}_1 | \Psi(t) \rangle$. We have just derived the lattice-Boltzmann equation for the QLGA, which exactly describes the kinematic equation of the lattice. For a more precise derivation of the connection between LGA and LBM, we refer the author to read the annex A.

This novel formulation led to a line of research with new quantum-native simulations conducted by Yeppez [109, 110, 111, 112, 113, 114]. However, while this is a general procedure that uses previous research by Meyer to its advantage, it has some problems. Quantum computing is an excellent alternative for extensive simulations due to its power to encode an exponential number of states with a linear number of qubits. To make quantum algorithms efficient, we must ensure at least an exponential speed-up with respect to their classical counterpart in the algorithm themselves. Additionally, state preparation and measurement procedures must be efficient to maintain the computational advantage obtained as these are necessary steps. More details on this in section 2.4. An improvement in computational complexity is needed as quantum computers are several orders of magnitude slower (time required per operation) than classical ones [115]. This means that the algorithm needs to encode exponentially the lattice sites, needing N qubits for $L = 2^N$ lattice sites. At the same time, an efficient algorithm needs to compute this without intermediary measurements (only measuring once the simulation is completed after several time steps) or, at least reduce the number of measurements in each time step and the number of gates in the state preparation for the following step. In this line, we highlight two recent articles [116, 117] that achieve this to some extent and on which we will focus in subsection 3.2.3.

Before delving into the articles mentioned above, we would like to take a moment to discuss two other pieces that address two highly relevant topics related to these algorithms. First, we want to manage the close relationship between QLGA and quantum random walks (QRW). QRW are the quantum version of classical random walks [118]. In these algorithms, every lattice site has a coin qubit which encodes the direction of propagation and a register to encode lattice sites. In each time step, the coin qubit is modified by an operator. There is an explicit parallelism between QRW and QLGA, given the unitary operations and propagation in both of these models. QRW use unitary operators on the coin qubits to control the propagation probability in every direction and the stationary probability distribution over the lattice. Adding history-dependent dynamics, it is possible to replicate QLGA collision using QRW, which allows seeing the problem in a double perspective [119]. Secondly, we may ask ourselves if any alternative encoding exists to generalise the model. In this regard, Peter Love concludes that the restrictions imposed by conserving quantities (quantities unaffected after the collision operator) do not affect the final result [120]. This means that if we change from a computational basis ($|0\rangle$ and $|1\rangle$) to another, like $|+\rangle$ and $|-\rangle$, we will obtain the same result.

3.2.3 Recent algorithmic implementations on QLGA

Yepez’s work [110] was focused and specially designed for type-II quantum computers [121]. This limited the quantum algorithms needing to measure and apply a shift operation via classical transmission. In a recent pre-print article [116] it is argued that there is significant potential for further refinement and optimisation considering modern quantum computing capabilities. Furthermore, this enables time-step concatenation. This means that measuring and reinitialising the registers every time step will not be necessary. Chrit *et al.* propose a quantum algorithm adapted to modern hardware with a quantum streaming consisting of swap gates to do the shift and quantum phase estimation (QPE), avoiding initialisation via an iterative phase estimation algorithm (IPE) [122]. Following Yepez’s methodology, there is one qubit per channel and lattice site. This means that the total number of qubits is $N = C \times L$, with C the number of channels and L the number of lattice sites. The 1D model with two channels uses local registers consisting of two qubits. Each qubit encodes a linear combination of $|0\rangle$ (hole) and $|1\rangle$ (particle). Swap gates exchange information between $|\rightarrow(x)\rangle$ and $|\leftarrow(x+\Delta x)\rangle$, $\forall x \in \mathcal{L}$, denoting the channels’ directions with arrows. On the other hand, while the IPE algorithm allows skipping the intermediate initialisation between time steps, it requires an ancilla qubit per lattice site. Additionally, the number of gates scale as $O(2^m - 1)$, where m is the number of iterations. The more iterations we carry out, the better the results of the phase estimation will be. Other alternatives, such as QPE as introduced by Kitaev [123], gain precision with the number of ancilla qubits used. An important limitation of Chrit *et al.* work is the number of qubits needed, not obtaining a clear advantage with respect to classical algorithms. Therefore, effective parallel computations are needed to encode L lattice sites in $\log_2(L)$ qubits.

In a more recent work [117], Fonio *et al.* implement an alternative version of quantum FHP (QFHP) with exponential advantage in space complexity. This is achieved by two registers encoding x and y coordinates and another for the channels. Specifically, their algorithm initialises every particle and lattice site with Hadamard gates and then uses an additional qubit to encode which particles are occupied. The superposition of lattice sites and channels with an additional qubit to encode occupancy from quantum pointers. The collision rule is executed taking into account the conserved quantities (quantum invariants) in a systematic way, going from quantum pointers encoding (needed to execute the shift operator) to the computational basis (required to execute the collision operator). This approach is based on [124] to add quantum states in superposition and [125] to compare the addition of states within the same lattice sites. QPE is used to retrieve information on momentum and mass using their operator description [120], measuring using ancilla registers.

However, retrieving information from the quantum system with a quantum point-

ers' encoding of the space needs a high number of measurements, as states encoding particles have a low probability of being measured. Additionally, the initialisation uses multi-control Toffoli gates with exponential complexity. While this work is the first to apply QLGA-FHP it does not provide quantum advantage due to the exponential complexity in the initialisation and the need to reinitialise every time step, or alternatively, creating new ancilla qubits every time step. No simulation results are presented in [117] which can be caused by the high number of qubits needed for the collision, as one needs to go from one encoding to another, and the need for several ancilla qubits. These characteristics make it costly to simulate in classical. Therefore, a quantum algorithm for FHP with time step concatenation (perform the algorithm for different time steps without intermediate measurements of the system) and an alternative encoding to reduce the number of qubits is yet to be found. This would allow one to test and validate the model against analytical results in use cases that LGA successfully models.

4 Quantum Lattice Boltzmann Method

In the previous section, we have reviewed quantum lattice-gas automata. This method is one of the most promising for quantum algorithms on CFD. This is because it can simulate the Navier-Stokes equations with non-linear terms in an efficient way, linked to the fact that collision is local and propagation is node-independent. Nonetheless, at its current stage, the method is inefficient as measuring and preparing classical data in a quantum computer is needed every time step. This is a clear bottleneck that ought to be overcome. This section will review another lattice-based method for CFD, the lattice Boltzmann method (LBM), which has also been used for simulating fluid flows using quantum algorithms [126, 127, 128, 129, 130, 131].

We will divide this section into three parts. First, we will introduce the LBM. Second, we will use a specific kind of LBM that has been widely used for its ability to provide an easy and straightforward formulation. This is called the Bhatnagar Gross and Krook (BGK) method. It avoids the complicated collision rule from the derivation of LBM, from ensemble averaging the LBE obtained from the microscopic rules of LGA. Finally, we will briefly summarise the research work on quantum LBM, pointing out its progress and bottlenecks. For more details about the connection between LGA and LBM, we refer to the appendix A.

4.1 Lattice-Boltzmann method and the BGK Approximation

Following [132], a statistical description of a system can be done using a distribution function $f(x, c, t)$ indicating the mean number of particles, with position, velocity and time dependencies respectively. If we consider a scenario where no collision takes place and we only apply the transport operator, the probability distribution $f(x, c, t)$ before the stream, and $f(x + c\Delta t, c + F\Delta t, t + \Delta t)$ after, are equal in velocity and position. Here we have used a to denote acceleration. This means that

$$f(x + c\Delta t, c + a\Delta t, t + \Delta t)dxdc = f(x, c, t)dxdc \quad (37)$$

However, if we include collision between particles, this may not be true anymore. In this case, we include an additional term to the equations, called the collision operator $\Omega(f)$

$$f(x + c\Delta t, c + a\Delta t, t + \Delta t)dxdc - f(x, c, t)dxdc = \Omega(f)dxdt dc \quad (38)$$

This is the lattice Boltzmann equation, having the same form as (36) and known as the transport equation. LBM includes the same two main steps as LGA: collision and propagation. This novel interpretation, initially conceived as a mesoscopic description of LGA to formulate macroscopic equations, has subsequently been employed for CFD simulations. The relationship between LGA and LBM is elaborated in Appendix A.

The initial LBM models, called first-generation LBM, have the same problems as their boolean counterparts (LGA). In 1954, Bhatnagar, Gross and Krook (BGK) developed a novel simplification to the collision term (r.h.s of (38)) of the Boltzmann equation [133], which was introduced as a simplification of the LBE by Koelman [134]. This is called the Lattice-BGK (LBGK) model. Using this method, we can write the collision term as

$$\Omega = \omega(f^{eq} - f) \quad (39)$$

with ω the collision frequency and with f^{eq} the local distribution function of interest. Using this method, we can obtain a LBM formulation without using the distribution functions (derived by ensemble averaging the boolean transport equation) obtain in appendix A. Therefore, instead of using the Fermi-Dirac distribution function (with exclusion principle), we use the Maxwell-Boltzmann distribution (40) from real gases to simulate the Navier-Stokes equations.

$$f(c) = \frac{\rho}{(2\pi K_B T)^{\frac{3}{2}}} e^{-\frac{(c-u)^2}{2K_B T}} \quad (40)$$

with K_B the Boltzmann constant and u mean velocity. However, we can also use other local equilibrium distribution functions or additional linear terms for

external forces, simulating other partial differential equations. The discretized LBGK equation with the external force term K would read as

$$f(x + c_i \Delta t, t + \delta t) - f(x, t) = -\omega(f(x, t) - f^{eq}(x)) + K(x, t) \quad (41)$$

Using this formulation, the collision is simply the difference between the current distribution and the equilibrium we want to achieve. After the collision, a streaming operator is used to compute the final result for the next time step. We use therefore the same steps as in LGA, where $f \in \mathbb{R}$ instead of being boolean. Following Wolf's book on LBM [135], we can distinguish three main ingredients or constituents of the model

1. A lattice, originally defined by Koelman with nine velocities (D2Q9). The velocities are $c_0 = (0, 0)$, $c_{1,3} = (\pm c, 0)$, $c_{2,4} = (0, \pm c)$, $c_{5,6,7,8} = (\pm c, \pm c)$, with c a certain velocity, usually one.
2. The equilibrium distribution function (Maxwell-Boltzmann for Navier-Stokes)
3. A transport equation: Discretized LBE with BGK approximation.

Given a function $f(x, t)$ not at equilibrium, we can write

$$f_i(x, t) = W_i + F_i(x, t) \quad (42)$$

where W_i is a discretized global homogeneous distribution function and $F_i(x, t)$ a weak deviation. The terms W_i are also known as weights as they scale the equilibrium distribution function and encode the contribution of each velocity. The W_i terms for identical speeds are equal due to symmetry. As W_i must be of the form of the Maxwell-Boltzmann distribution function, it follows that

$$\begin{aligned} \sum_{i=1}^b W_i c_{i\alpha} &= 0 \\ \sum_{i=1}^b W_i c_{i\alpha} c_{i\beta} c_{i\gamma} &= 0 \\ \sum_{i=1}^b W_i &= \rho \\ \sum_{i=1}^b W_i c_{i\alpha} c_{i\beta} &= \rho \frac{K_B T}{m} \delta_{\alpha\beta} \\ \sum_{i=1}^b W_i c_{i\alpha} c_{i\beta} c_{i\gamma} c_{i\delta} &= \rho \left(\frac{K_B T}{m} \right)^2 (\delta_{\alpha\beta} \delta_{\gamma\delta} + \delta_{\alpha\gamma} \delta_{\beta\delta} + \delta_{\alpha\delta} \delta_{\beta\gamma}) \end{aligned} \quad (43)$$

In the last equation, we have used the isotropy property of the tensor to decompose it into delta functions. Solving these equations for the D2Q9 model, we obtain

the terms W_i , usually interpreted as the weights of each lattice site to maintain isotropy, and therefore, they are Bravais-lattice dependent. These are

$$\begin{aligned} W_0 &= \frac{4}{9}\rho \\ W_{1,2,3,4} &= \frac{1}{9}\rho \\ W_{5,6,7,8} &= \frac{1}{36}\rho \end{aligned}$$

Notice that the sum of the weights is 1. We also obtain a relation between each channel velocity and the temperature

$$\frac{K_B T}{m} = \frac{c^2}{3}$$

The local distribution function can be obtained by a Taylor expansion of the Maxwell distribution function according to equation (40) with u the mean velocity and c the instantaneous velocity scaled with their corresponding weights. Notice that the difference between different physical problems depends mainly on the equilibrium distribution function we choose. For this particular case we have

$$\begin{aligned} f_i^{eq} &= \frac{4}{9}\rho \left(1 - \frac{3}{2} \frac{u^2}{c^2} \right) \quad i = 0 \\ f_i^{eq} &= \frac{1}{9}\rho \left(1 - \frac{3c_i \cdot u}{c^2} \frac{u^2}{c^2} + \frac{9}{2} \frac{(c_i \cdot u)^2}{c^4} - \frac{3}{2} \frac{u^2}{c^2} \right) \quad i = 1, 2, 3, 4 \\ f_i^{eq} &= \frac{1}{36}\rho \left(1 - \frac{3c_i \cdot u}{c^2} \frac{u^2}{c^2} + \frac{9}{2} \frac{(c_i \cdot u)^2}{c^4} - \frac{3}{2} \frac{u^2}{c^2} \right) \quad i = 5, 6, 7, 8 \end{aligned} \quad (44)$$

The macroscopic variables such as pressure or viscosity can be derived with a multi-scale expansion [74].

However, there are other models that we can consider as introduced by Qian [136]. Usually, $Dnqm$ is used as a notation to refer to the number of dimensions n and velocities m . In these other models, the weights are derived in a similar way, changing the velocities corresponding to each model. For example for D1Q3, the weights are $W_0 = \frac{1}{3}$ and $W_1 = \frac{1}{6}$. As in LBM we have probability distributions instead of real particles, all channels in every lattice site is partially occupied. Examples of different lattices are depicted in figure 6

Finally, as the LBM is an iterative computational scheme, we need to recover the macroscopic variables from distribution functions and update back the equilibrium distribution functions for the next time step. The macroscopic quantities are given by the moments of the distribution function. For example, for the Maxwell

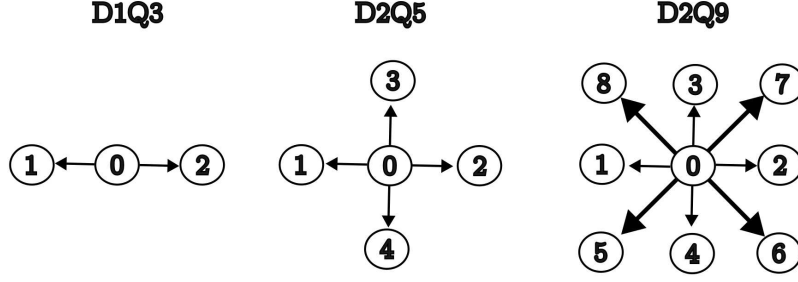


Figure 6: Schematic representation of main lattice arranges for LBM (D1Q3, D2Q5 and D2Q9) where D refers to the dimension and Q to the number of channels. Each channel is associated to a different direction during the propagation step.

distribution function, we can obtain the mass, momentum and energy densities as

$$\begin{aligned}
\sum_{i=1}^b f_i &= \rho \\
\sum_{i=1}^b f_i c_i &= \rho u \\
\sum_{i=1}^b f_i c_i^2 &= \rho u^2 + 3\rho K_B T.
\end{aligned} \tag{45}$$

A more concise explanation of the algorithm is the following. First, a density ρ and velocity c distributions are selected, and the initial values of the local distribution function are set as $f(x, 0) = f^{eq}(x)$. The local equilibrium distribution functions are obtained using equation (44). Then, the local distributions are transformed using collision and propagation according to the LBE (41). The next step is to recalculate the macroscopic variables using (45) and the equilibrium distributions for the next time step (44) back again. This method is iteratively repeated over time. In order to derive the equilibrium distribution function, an ansatz of the solution with certain restrictions is used. Then, Champan-Enskog expansion can be used to verify the result. Some examples of the procedure can be found in [132] for the advection-diffusion equation and in [137] for the stream-vorticity formulation for the Navier-Stokes equations. Briefly, the algorithm is as follows:

Initialization: ρ and u set f^{eq} (44) and $f(x, 0) = f^{eq}(x)$.
for $t < T$, $t++$ **do**
 Collision calculations given by r.h.s (41)
 Streaming calculations l.h.s (41)
 Next time step preparation: Obtain macroscopics (45) and f^{eq} (44) for next time step.
End

An example of how the algorithm works can be found in the appendix B using D1Q3 with diffusion equation.

4.2 Quantum Lattice-Boltzmann Method bibliographic review

In light of the challenges posed by the lattice-gas automata (LGA) method, particularly its lack of Galilean invariance and the presence of statistical noise, researchers are exploring alternative quantum algorithms for fluid dynamics. One such promising alternative is the Lattice Boltzmann Method (LBM). The LBM is a probabilistic model that employs amplitude encoding, a technique that is more intuitive and commonly used in quantum algorithms. This approach contrasts with the computational basis encoding utilized in LGA, offering potential advantages in terms of simplicity and efficiency as we are simulating directly at a mesoscopic scale.

The first attempt to develop a quantum algorithm was made by Todorova *et al.* in 2020 [138] using a collisionless LBM limited to applications with free-molecular flows without interaction. This marked the first instance of a quantum algorithm for fluid dynamics that did not require intermediate measurements at each time step. The approach implements the streaming using a quantum circuit based on Toffoli gates with logarithmic encoding of the lattice sites using quantum superposition. The model also considers solid walls with specular-reflection boundary conditions using a quantum circuit. In specific, the work focuses on rarefied¹⁰ gas simulations. As LBM is, in general, not accurate under these conditions, an alternative velocity model is preferred. The model consists on particles that can move by a fraction of a lattice unit depending on their discrete velocities and the chosen time step [139]. For that, a reservoir technique is used to account for fractional velocities without numerical diffusion [140]. This method involves variables that are updated to store the information on partial velocities between different lattice sites. This allows the application of rarefied flows with LBM accurately using a quantum computer without intermediate measurements. The algorithm was later used for simulating the evolution of self-gravitating systems, adding gravitational terms respecting Todorova's original work [141].

Shortly after Todorova's work, Ljubomir Budinski designed a 1D and 2D quantum algorithm for the full LBM including collisions, first for advection-diffusion equations [142] and later for the Navier Stokes equations with stream-vorticity formulation [143]. Stream-vorticity formulation is derived by taking the curl of the Navier-Stokes equations.¹¹ Using the local distribution function for advection-

¹⁰Flows with considerable mean-free path respect to smallest macroscopic scale

¹¹The relationship between this approach and the usual expression for the Navier-Stokes equa-

diffusion equation, Budinski uses a non-unitary block-diagonal operator $\mathbf{A}$ to relax the mentioned distribution function. To design a quantum circuit with unitary operators using this block-diagonal matrix, a linear combination of unitaries (LCU) approach is used [145]. This approach consists of writing a non-unitary operator using a linear combination of unitaries via ancilla qubits. Additionally, Budinski includes a novel approach based on the macroscopic Lattice-Boltzmann method (MacLBM) where the collision frequency is set to $\omega = 1$ to simplify the equations, the applicability of which has been verified by Zhou [146]. In [143], Budinski uses the stream-vorticity formulation to write the transport equation as an advection-diffusion equation. The equilibrium distribution functions for this formulation in LBM have been verified by Bo Yan *et al.* [137]. In both models, the presence of nonlinear terms in the local equilibrium distribution function requires measuring and re-initializing every time step. This is because when $\mathbf{A}$ is not anti-Hermitian, the best-case scenario of LCU approach has an exponentially small success probability. However, this can be partially solved according to Berry *et al.* [147], who developed an alternative LCU method with higher accuracy and success probability.

In 2022, a new work by Schalkers and Möller [148] further optimized the previous collisionless LBM. A novel encoding reduces the number of gates by adding a quantum register to encode each macroscopic magnitude and channel information. While this approach increases the number of qubits used, it decreases the number of gates. It also renders the specular reflection by wall collisions more efficient. Additionally, the collision in the corners is corrected; the original QLBM model produced reflection errors when a particle collided in a non-axis-aligned trajectory. Another novelty of this work was the introduction of quantum-Fourier-transform-based streaming for the streaming initially conceived by Thomas.G [149]. In 2023, Schalkers and Möller proposed a novel encoding to apply the collision and propagation steps with unitary operators, adding new qubits for each time step [150]. Traditional encodings based on amplitude or computational basis are proven not to allow both collision and propagation to be unitary. The reason is that a unitary operator cannot satisfy all the equivalence relations that conserve mass and momentum for the collision. In the case of the computational basis encoding, the collision is unitary, but the streaming is not, as this is non-local. Therefore, according to the authors, there are three alternatives to developing a fully unitary lattice Boltzmann method. The first option is to develop an amplitude base encoding with non-orthogonal velocities. The second option is to avoid the non-locality of the computational basis encoding (one example may be the back-and-forth encoding method used in QLGA). Finally, as the third option, one could use an alternative encoding that is different from the ones mentioned. In their article, the authors decide to develop this last alternative with a model using $O(N_t d)$ channels for each lattice site, with N_t number of time steps and d dimensions. For most applications, the number of qubits used is excessive, not achieving any quantum advantage. The

tions can be seen in [144].

idea to avoid the non-locality of the streaming function is to save every possible channel for the neighbourhood in each lattice site. Neighbourhood, in this case, is defined in a broad sense, as all the channels from all lattice sites that could reach a given lattice site x_i in N_t time steps.

Recently, research has focused on applying Carleman linearization [151] to the standard LBM. This method allows to express nonlinearities using an infinite amount of linear terms. However, good approximations can be obtained with a discrete (but large) number of terms. Using quantum computers scaling, research has focused on simulating these terms using only linear equations. In 2022, Itani and Succi used for the first time Carleman linearization on the collision term of LBM [152]. The method's accuracy was analyzed, observing an improvement with the linearization order (as the method is only exact using an infinite number of terms). However, the exactness of the streaming is compromised, and the number of variables to keep track of blows up. In 2023, a new implementation of Carleman linearization in LBM achieved a higher accuracy for low-order Carleman linearization than previously thought to be possible, paving the way for a quantum Carleman LBM [153]. In the same year, Bakker and Watts [154] introduced the implementation of the boundary conditions into the Carleman framework with the bounce-back boundary conditions scheme. Finally, a recent article by Claudio Sanavio and Claudio Succi [155] implements a new quantum circuit approach that treats the Carleman variables with a qubit register and obtains similar accuracies for low Reynolds numbers to those of LBM. However, the depth of the circuit makes its applicability unviable in the near future, and more work needs to be done in this direction.

5 Other quantum algorithms for fluid dynamics

In this chapter, we will review alternative algorithms for CFD. The goal of this chapter is to introduce briefly other methods and review their progress in scientific research.

The Navier-Stokes equations are nonlinear partial differential equations. However, quantum physics is based on unitary operators and linear algebra, and representing nonlinear dynamics is a major barrier. Consider, for example, computing the square of a value u , which would imply that a copy of the variable is needed. However, this is not possible due to the no-cloning theorem 1. Therefore, quantum research has initially focused on solving linear systems of equations. However, nonlinear differential equations can be discretized and linearized using different methods such as Picard method [156] or Newton method [157].

The following review is not by any means exhaustive, but it introduces some of

the most important non-lattice-based quantum algorithms for CFD. Some other possible alternatives to lattice-based models are, e.g., based on the hydrodynamic Schrödinger equation [158], deep learning quantum algorithms applied to nonlinear differential equations [159] and the solution of nonlinear stochastic differential equations [160, 161].

5.1 Quantum algorithm for linear system of equations (HHL)

The first attempt to solve a linear system of equations in a quantum computer was done by Harrow, Hassidim, and Lloyd [8], known as the HHL algorithm. In their research, Harrow *et al.* developed a method to obtain an approximation of the expectation value of an unknown vector in a linear system. Instead of solving the system $\mathbf{A}x = b$ with $\mathbf{A}$ a $N \times N$ matrix and $x, b \in \mathbb{R}^n$, the expectation value of an operator $\mathbf{M}$ respect x is obtained. In this context, the algorithm calculates $f = \langle x | \mathbf{M} | x \rangle$, which retrieve information respect the unknown quantum state, which encodes the original x in its amplitude. Without considering initialization and measuring and when $\mathbf{A}$ is sparse, the algorithm provides an exponential advantage. The quantum algorithm scales as $O(\text{polylog}(N)\kappa^2)$, while the best iterative classical algorithms for sparse matrices do as $O(N\kappa)$ in its best case scenario. Here κ is the condition number, the ratio between the largest and smallest eigenvalues of $\mathbf{A}$.

First in the algorithm, the vector b is represented as a quantum state $|b\rangle = \sum_{i=1}^N b_i |i\rangle$ in the computational basis. This representation will depend on the encoding used. The next step is to write the operator $\mathbf{A}$ as an exponential to later on estimate its eigenstate using the quantum phase estimation algorithm (QPE) [162]. The problem of writing a hermitian matrix $\mathbf{A}$ as an exponential e^{-iHt} describing its exact time evolution is known as Hamiltonian simulation [163]. Then, once the eigenstate $|\lambda_j\rangle$ is obtained, we obtain the following state

$$|\psi\rangle = |b\rangle^{\otimes n} |l\rangle^{\otimes n_l} |0\rangle \quad (46)$$

with n_l the number of qubits used to encode l . Then, we calculate the inverse of the eigenstate (as detailed in the bibliography), to finally obtain

$$|\psi\rangle = C \sum_{j=0}^{N-1} b_j \lambda_j^{-1} |b_j\rangle |\lambda_j\rangle |1\rangle \quad (47)$$

Taking into account that

$$|x\rangle = C \sum_{j=0}^{N-1} b_j \lambda_j^{-1} |b_j\rangle$$

and doing the inverse quantum phase estimation (IQPE), which uncompute the state $|\lambda_j\rangle$, we obtain finally

$$|\psi\rangle = C |x\rangle |0\rangle^{\otimes n_l} |1\rangle \quad (48)$$

The authors suggest that this state can then be measured in an appropriate computational basis to obtain the desired result. This algorithm has been proven to be efficiently implemented in a hybrid quantum-classical CFD solver [164, 165] using a lid-driven cavity test¹².

Although the algorithm allows to obtain the expectation value of unknowns in linear systems, its applicability is limited. The exponential advantage of the algorithm can only be achieved if quantum state preparation is done efficiently, which as explained in section 2.4.1 is an unsolved problem. In the same way, the measurement procedure via expectation value of the unknown state $|x\rangle$ into $\langle x|\mathbf{M}|x\rangle$ is not provided. As the algorithm scales with κ as $O(\kappa^2)$, the exponential advantage is only achieved if $\kappa = O(\text{polylog}(N))$ at most. This is because in the algorithm we are estimating λ_j^{-1} . As the $\lambda \in [\frac{1}{\kappa}, 1]$, the QPE algorithm needs to run for a longer time to represent them correctly and distinguish from zero. In 2013, Clader *et al.* [166] generalized the HHL algorithm to solve these issues. The innovative methodology presented encompasses both initialization and measurement processes. It addresses the issue of the condition number through a technique known as preconditioning, as detailed in [167]. During this procedure, the linear system $\mathbf{A}x = b$ is substituted by $\mathbf{M}\mathbf{A}x = \mathbf{M}b$ using an operator $\mathbf{M}$ to simplify the problem. This decreases the computational complexity of the algorithm in terms of κ from quadratic to linear. In 2017, Leonar Wossning *et al.* [168] expanded HHL for dense matrices with a computational complexity of $O(\kappa^2 \text{polylog}(N) \|\mathbf{A}\|_F)$ where $\|\mathbf{A}\|_F$ is the Frobenius norm $\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^N |a_{ij}|^2}$. Later on in 2021, Yoshiyuki Saito *et al.* [169] proposed an iterative quantum algorithm improving the accuracy of HHL without increasing the number of qubits and reducing the number of measurements. Another alternative to optimize this algorithm was developed by Yonghae Lee *et al.* [170] making use of a hybrid approach to reduce an intermediate step between the eigenstates estimation via QPE algorithm and the inverse mapping to obtain $|x\rangle$. Using this approach the authors decreased the number of two-control gates needed using a priori classical information to shorten the quantum circuit. In 2022, Fang Gao *et al.* [171] developed a hybrid quantum algorithm for HHL decreasing the number of qubits needed using more measurements.

Linear differential equations can also be solved using HHL. In order to do so, a discretized representation of a linear differential equation is obtained, transforming the problem into a linear system of equations. An example of this, is the application of the HHL algorithm on the Poisson equation by Daribayev *et al.* [172]. However, a novel research by Tao Xin *et al.* [173] in 2018 approached the problem differently. A Taylor expansion of the general analytical solution of the linear differential equation was used. This allowed to scale the computational complexity as

¹²The lid-driven cavity test is a standard test in CFD. It consists of three solid walls with non-slip conditions and a lid moving tangentially. The lower left corner has a reference pressure $P = 0$

$O(\log(1/\epsilon))$, where ϵ is the precision to achieve. This is a significant improvement compared to $O(1/\epsilon)$ of the original HHL algorithm's dependency on this variable. However, the precision also depends on the order of the Taylor expansion k , and the number of required ancilla qubits grows linearly with it.

5.2 Variational quantum algorithms

A more recent alternative to solve quantum linear systems of equations was presented in 2019 by Bravo-Prieto *et al.* [174] making use of a variational quantum circuit (VQC)¹³. Several other similar approaches were presented at the same time [177, 178, 179]. However, the algorithm developed by Bravo-Prieto *et al.* is the most broadly used and it is known as the variational quantum linear solver (VQLS) algorithm. It makes use of variational hybrid quantum-classical algorithms (VHQ-CAs), which reduce the depth of the circuit while using optimization in one of its steps classically. This algorithm is presented as an alternative to HHL, where fewer qubits are needed and therefore, it is ideal for NISQ devices. The algorithm consists of a quantum processing unit (QPU), which estimates, with a certain cost function, the closeness between $\mathbf{A}x$ and b , and a classical optimization loop, which modify the parameters used in the quantum circuit. These parameters are $\Phi \in [0, 2\pi]$ and $\Theta \in [0, \pi]$, used in rotation operator gates $\mathbf{R}_x$, $\mathbf{R}_y$ and $\mathbf{R}_z$. The algorithm is proven to be advantageous for certain cost functions where no efficient classical estimation can be obtained. The authors find that the VQLS algorithm scales as $O(\text{polylog}(N)\kappa)$ for a constant precision ϵ in the worst-case scenario.

¹³A variational quantum circuit is a quantum circuit that depends on free parameters θ . VQC is widely used in quantum machine learning to optimize their internal parameters according to a cost function in order to solve a problem [175] or compress information in the Hilbert space [176].

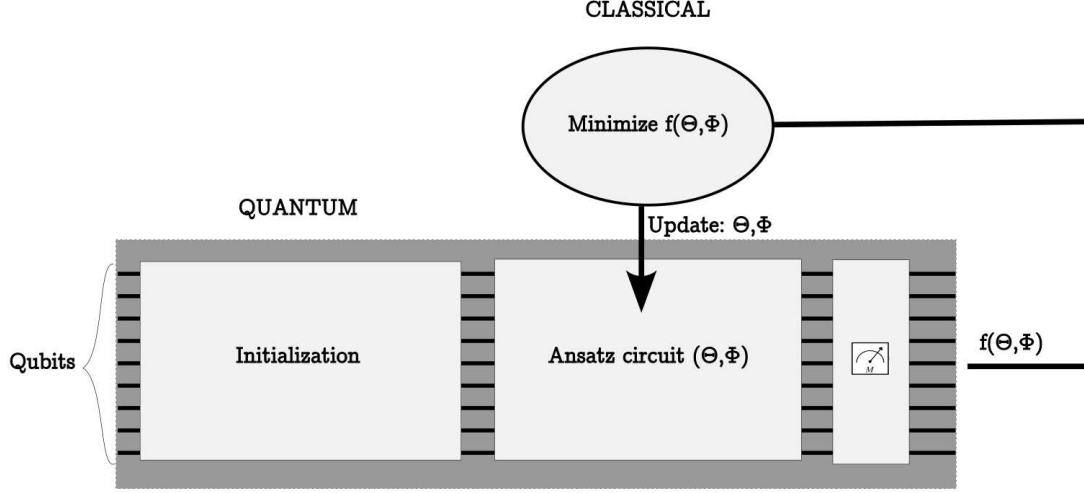


Figure 7: Simplified algorithmic scheme of variational quantum algorithms such as VQLS. In general, we have a quantum algorithm consisting on rotational gates $\mathbf{R}_x$, $\mathbf{R}_y$, $\mathbf{R}_z$ depending of a set of parameters we call Θ and Φ

This method has been verified on its applicability on fluid dynamics [180] by solving a potential flow¹⁴ in a circular cylinder and Stokes flows¹⁵ in the lid-driven cavity test. The authors also make use of the generalized differential quadrature (GDQ) [181] to reduce the size of the linear system needed to conserve the same accuracy. The discretization technique consists of writing the partial derivative of a function at a given spatial point as a weighted linear sum of the function at that point. The authors also verified the algorithm for the heat conduction equation [182]. However, other studies point out that training and updating the parameters of VQLS takes more iterations than the best classical algorithms, and its implementation is inefficient for the application of the Poisson equation [183]. In 2022, Yew Leong *et al.* [184] developed a VQE for linear systems and nonlinear systems (computing the nonlinear terms in a classical computer) with time dependence, extending the method for temporal discretization schemes.

Variational quantum algorithms have also been proven to solve nonlinear differential equations. In 2020, Lubash *et al.* [185] developed a quantum algorithm for these systems using a quantum non-linear processing unit (QNLPU) to calculate nonlinear terms, which consist of a unitary operator conditioned to ancilla qubits. This work was further improved in 2023 [186] for multidimensional partial differential equations, solving the Buckmaster equation which rules the time evolution of the surface of a viscous liquid. Another work in 2021 [187] utilized a non-linear variational solver for a specific Navier-Stokes simulation.

¹⁴Flow with zero vorticity

¹⁵Flows with near zero Reynolds number

5.3 Quantum finite volume method (QFVM)

The finite volume method (FVM) is a popular algorithm for solving CFD problems in classical computers. The method is based on the divergence theorem, which states that the integral of the divergence of a given vector over a closed volume is equal to the flux of this vector across a closed surface bounding the volume. Note that this is a special case for a volume in $\mathbb{R}^3$ of the Stokes' theorem. For a compressible fluid, the Navier-Stokes equations in integral form can be written as

$$\frac{\partial}{\partial t} \int_{\Omega} W dV + \oint_{\partial\Omega} F \cdot dS = 0 \quad (49)$$

with Ω the volume over which we integrate, $\partial\Omega$ its boundary, W being a 4×1 vector containing mass, momentum and energy and F a matrix representing their flux on x and y coordinates. Specifically, we can write W and F as

$$W = \begin{bmatrix} \rho \\ \rho u_x \\ \rho u_y \\ \rho E \end{bmatrix}, \quad F = \begin{bmatrix} \rho \rho u_x & \rho u_y \\ \rho \rho u_x^2 + p & \rho u_y^2 + p \\ \rho \rho u_x u_y & \rho u_y^2 + p \\ \rho \rho u_x E & \rho u_y E \end{bmatrix} \quad (50)$$

The previous equation can be discretized using a mesh. This means that the space is divided into cells of different shapes and sizes covering the simulation space. The flux and velocity will be obtained for each cell, integrating the governing equations. After the discretization,

$$\frac{\Omega_i}{\Delta t} (W_i^{n+1} - W_i^n) = - \sum_{\partial\Omega} F_{i,\partial\Omega}^{N+1} \Delta S_{i,\partial\Omega}^{N+1}. \quad (51)$$

the equation can be linearized by rewriting the right-hand side of (51) as R_i^{n+1} called residue. Now we can relate the next time step $n + 1$ with n as $R_i^{n+1} = R_i^n + \frac{\partial R_i^n}{\partial W_i^n} \Delta W_i^n$. This yields the equation

$$\mathbf{A} \Delta W_i^n = -R_i^n \quad (52)$$

using $\mathbf{A} = \left(\frac{\Omega_i}{\Delta t} + \frac{\partial R_i^n}{\partial W_i^n} \right)$. This is the linearized Navier-Stokes equation using the FVM.

In 2022, a new study Zhao-Yun *et al.* developed a quantum algorithm for FVM, using the same methodology as the classical algorithm [188]. The main contribution of the article is not the quantum algorithm itself, as the method makes use of quantum linear solvers with a quantum preconditioner [166] to calculate the linear system obtained by the discretization. Instead, the research focuses on how to do this efficiently and make use of a quantum random access memory (QRAM) [189]. Therefore, they compose a data layout to prepare the state, execute the subqueries and measure efficiently. However, such memory has not been produced

yet and the assumptions are only based on early physical implementations in the laboratory that are not widely available. The algorithm is one of the few papers that has been prepared and studied from the point of view of an existing QRAM and its computational complexity is polylogarithmic in the number of cells N . This presents a substantial quantum superiority and potential, contingent upon the availability of QRAM.

5.4 Quantum algorithm for nonlinear differential equations

In 2008, Leyton *et al.* [190] published an article on a quantum algorithm for nonlinear differential equations motivated by the HHL algorithm. The method is capable of solving a system of n first-order nonlinear ODE equations. It consists of the encoding of one variable per quantum register and its transformation using Hamiltonian simulations. The final state has a high probability of being measured as long as the Hamiltonian simulation has a small error. However, the successful transformation of the quantum registers with high probability needs several copies of every variable, growing exponentially with accuracy and time.

In 2021 Jin-Peng Liu *et al.* [191] developed a quantum algorithm based on Carleman linearization to solve quadratic ODE differential equations where convergence is only guaranteed when nonlinear terms are small. It was later successfully applied for reaction-diffusion equations [192]. This algorithm improves Leyton's works, reducing the computational complexity on time t from exponential to quadratic. Subsequent work was published later in 2023 by Krovi (one of the authors from the previous work) in 2023 [193]. Krovi presented an improved version of the algorithm, reducing errors and enabling its use for a wider range of nonlinear terms. Finally, in December 2023, a new study presented a quantum algorithm [194] that demonstrated a linear time complexity through the use of Carleman linearization for nonlinear differential equations. This algorithm achieves, to our current understanding, the lowest error bounds reported in this area. Another approach with similar time complexity was published in late 2020 [195] where the non-linear Schrödinger equation [196] is combined with a QLS, however, no quantum circuit implementation is provided therein.

6 Novel quantum models for fluid dynamics

In this section, we will review two quantum algorithms developed for this work. A 1D model called D1Q3, with D standing for dimension and Q for the number of channels and the quantum version of the FHP-III model with six particles. To verify the quantum models, we will compare the results with their classical

counterparts in different benchmarks. In the case of the FHP-II, which can model Navier-Stokes equations as discussed in section 3.1, we will also compare the results with the theoretical equation for an incompressible fluid flow.

6.1 Quantum Lattice-gas automata algorithm overview

In this section, we introduce the structure of both quantum algorithms developed for this thesis. Both algorithms (for 1D and 2D) are composed of the same steps: Initialization, mapping, collision, propagation, and measurement, as seen in Figure 8. Without loss of generality, we introduce the steps and equations in 1D, but we can formulate them the same way for 2D.

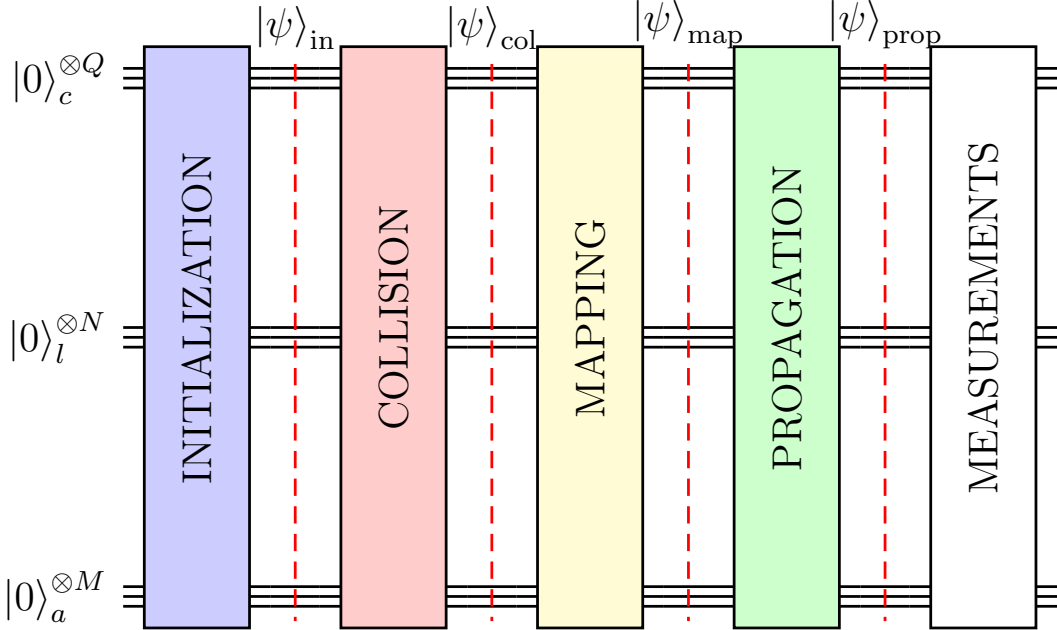


Figure 8: Overview of quantum lattice-gas automata algorithm.

- Initialization: At the end of each time step in the algorithm, the final quantum state will be measured and then initialized for the next time step. In the initialization, the classical data is encoded in the form of a state in a quantum device. As introduced in section 2.4.1, there are different ways to optimize the initialization to reduce its computational complexity. However, in this case, the algorithm available in Qiskit based on the work by Shende *et al.*[59] is used. The encoding consists of a register using a qubit per each channel noted as c , a second register noted as l for the lattice sites, using $N = \log_2(L)$ qubits with L the number of lattice sites and an ancilla register

a with M number of qubits. We can write it mathematically as

$$\begin{aligned} |\psi\rangle_{in} &= \mathbf{I}|\psi\rangle_0 = \mathbf{I} \left(|0\rangle_c^{\otimes Q} \otimes |0\rangle_l^{\otimes N} \right) \otimes |0\rangle_a^{\otimes M} \\ &= \frac{1}{\sqrt{2^N}} \sum_{i=0}^{L-1} |c_1 \dots c_Q l_1 \dots l_N\rangle_i \otimes |0\rangle_a^{\otimes M} \end{aligned} \quad (53)$$

with $|\psi\rangle_0 = |0\rangle^{\otimes(Q+N+M)}$ the initial state. For example, the quantum sub-state $|110|0100\rangle$ represents the occupation of the first and second particles (in a case with three channels) and the lattice site corresponding to the binary string 0100. If we use the least significant bit encoding to the right, 0100 corresponds to the lattice site number five.

- Collision: The collision will have the same role as in the classical algorithm. Making use of a look-up table, a unitary operator can be used to implement an effective interaction between particles in every lattice site in parallel, as introduced in Section 3.2.2. Since the occupancy of the channels is stored only in the register c , the collision step only involves Q qubits. However, we can also use ancilla qubits to reduce the number of gates or implement complex rules. We will denote as $\mathbf{C}$, to the collision operator and $|\widetilde{c}\rangle = |\widetilde{c_1 \dots c_Q}\rangle$ to the post-collision state. Therefore, we can write the step as

$$\begin{aligned} |\psi\rangle_{col} &= \mathbf{C}|\psi\rangle_{in} = \mathbf{C} \left(\frac{1}{\sqrt{2^N}} \sum_{i=0}^{L-1} |c_1 \dots c_Q l_1 \dots l_N\rangle_i \right) \otimes |0\rangle_a^{\otimes M} \\ &= \frac{1}{\sqrt{2^N}} \sum_{i=0}^{L-1} (|\widetilde{c_1 \dots c_Q}\rangle |l_1 \dots l_N\rangle)_i \otimes |0\rangle_a^{\otimes M} \end{aligned} \quad (54)$$

- Superposition mapping: The next step in QLGA algorithm is to propagate the different particles which reside in different lattice sites. However, as we have seen in Section 4.2 as introduced by Möller [148], using a basis encoding (with all the information carried by the binary encoding of the states and not by their amplitude), it is not possible to write the total evolution of the system as a unitary operator. This is because the encoding currently used, is written in a superposition of lattice sites but the propagation step is different for every channel. To solve this problem, we need an intermediary step. The superposition mapping changes the information representation from a binary encoding of channels $|c_1 \dots c_Q\rangle_i$ with every qubit representing the occupancy of the state i to a superposition encoding with $\frac{1}{Q} \sum_{j=0}^Q |a_1 \dots a_{\log_2(Q)}\rangle$, using the ancilla register noted by a . It is important to notice that this operation is non-unitary and the previous channel register will carry information that is not anymore of use.

There are two main approaches to perform this mapping step. The binary-superposition encoding with swaps and without swaps. The first consists

of creating an equal superposition of all possible channel occupation in the ancilla register and using a control swap gate to mark the states of interest, creating junk states in the process. We can see an example of this method in Figure 9. It can be formally written as

$$\begin{aligned}
|\psi\rangle_{map} &= \mathbf{M} |\psi\rangle_{col} = \\
&= \mathbf{M}_1 \frac{1}{\sqrt{2^N}} \sum_{i=0}^{L-1} (|c_1 \dots c_Q\rangle |l_1 \dots l_N\rangle)_i \otimes |0\rangle_{a_1} \otimes \mathbf{H} |a_2\rangle \otimes \dots \otimes \mathbf{H} |a_{\log_2(Q)}\rangle = \\
&= \frac{1}{\sqrt{2^N 2^{\lceil \log_2(Q) \rceil}}} \sum_{i=0}^{L-1} \sum_{j=0}^{Q-1} (|\widetilde{c}_1 \dots \widetilde{c}_{j-1}, a_1, \widetilde{c}_{j+1} \dots \widetilde{c}_Q\rangle |l_1 \dots l_N\rangle)_i \otimes |\widetilde{c}_j\rangle \otimes |d\rangle_j^{\otimes \lceil \log_2(Q) \rceil}
\end{aligned} \tag{55}$$

where $|d\rangle_j$ is the j -th binary string of the ancilla superposition. For example, if the number of channels is three, then two ancilla qubits are necessary and the possible states would be 00, 01, 10 and 11. For instance, the third state would be 10.

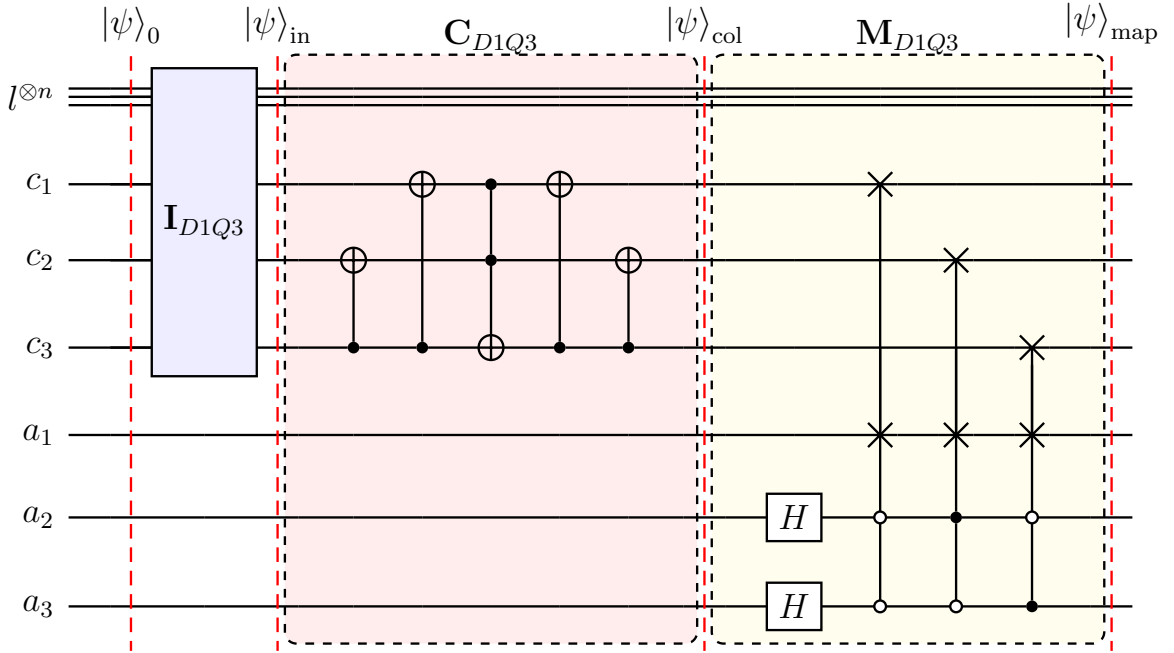


Figure 9: Quantum circuit for the initialization, collision and mapping step in D1Q3 model.

The second approach consists of creating a similar superposition using a succession of Hadamard and Toffoli gates. The advantage of this method is that the amount of state created decreases from Q (all possible states) to Q_O , with Q_O the number of occupied states. The quantum circuit for D1Q3 using this method can be seen in Figure 10. This makes possible to have

a junk-free state vector and reduce the number of measurements. However, ensuring equal amplitudes for every entangled state using this method is challenging. The applicability of this alternative is still possible with states with different amplitudes. However, the model becomes more vulnerable to noise under these conditions and the number of measurements to read every state is higher. Additionally, error mitigation techniques would be harder to accomplish and the fidelity of the results can be compromised. Therefore, we will not use this approach in the simulations presented later.

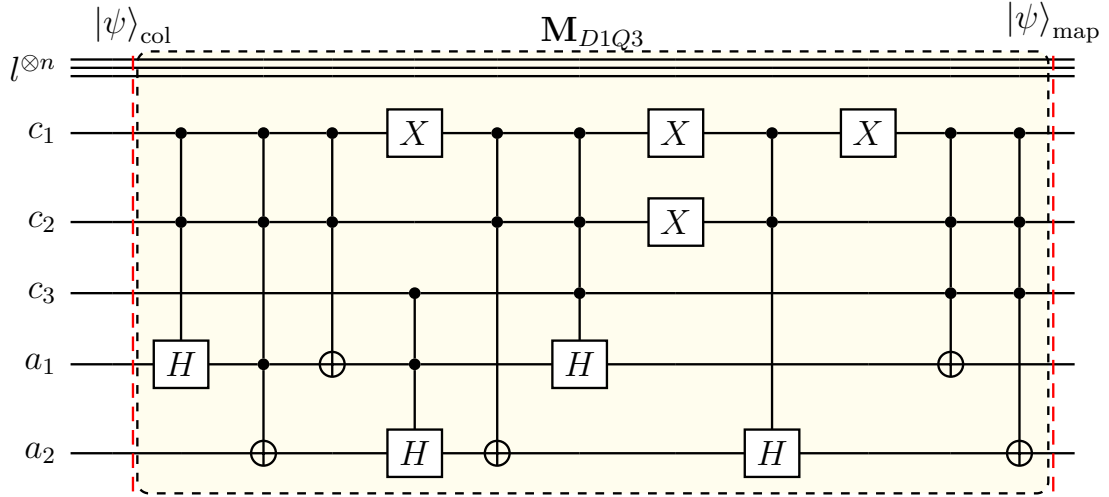


Figure 10: Quantum circuit for the alternative mapping step with uneven probabilities and no junk states in D1Q3 model.

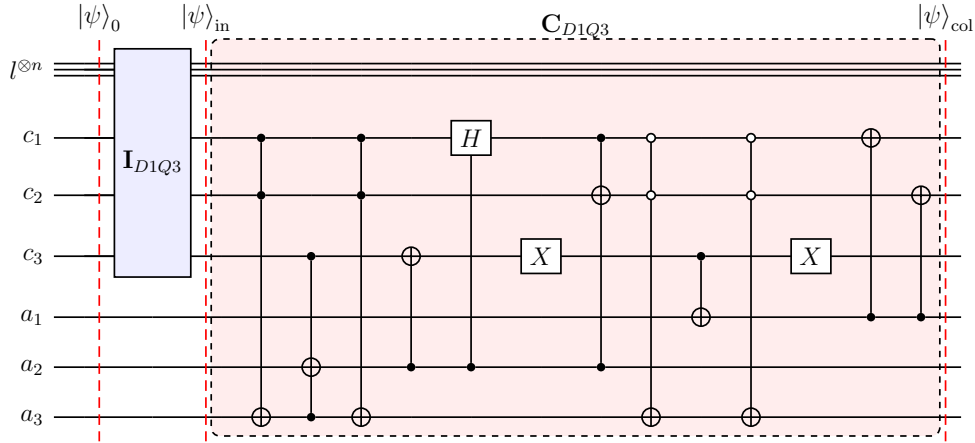


Figure 11: Quantum circuit for the initialization and collision in D1Q3 model using superposition-based QLGA. The unique tags to encode the different collisions have been done using the following mappings $|001\rangle \rightarrow |111\rangle$ and $(|100\rangle + |010\rangle) \rightarrow |000\rangle$ in the post-processing. Notice that in this case, the channels are initialised as a superposition and the state with zero particles is not included so we can use $|000\rangle$ for our encoding. This means that we only use $|100\rangle$, $|010\rangle$ and $|001\rangle$ for the channel register, unless there is one collision to be applied within a lattice.

- **Propagation:** The propagation of the particles along their corresponding directions can be implemented using different algorithms. The most simple of them, consists of using increment and decrement operators conditioned to each channel [197], an example of this circuit is represented in Figure 13. A more efficient implementation can be obtained by making use of the Quantum Fourier Transformation (QFT) for a transport algorithm as presented by Shakeel [198]. Another similar algorithm, more efficient in terms of computational complexity is presented by Budinski *et al.* [199] which incorporates the shift of different directions in parallel. We recommend the reader to review the former cite, as it also presents an introduction to particles propagation. Mathematically, we can express the propagation as

$$\begin{aligned}
|\psi\rangle_{prop} &= \mathbf{P} |\psi\rangle_{map} = \\
&\mathbf{P} \frac{1}{\sqrt{2^N 2^{\lceil \log_2(Q) \rceil}}} \sum_{i=0}^{L-1} \sum_{j=0}^{Q-1} (|\widetilde{c}_1 \dots \widetilde{c}_{j-1}, a_1, \widetilde{c}_{j+1} \dots \widetilde{c}_Q\rangle |l_1 \dots l_N\rangle)_i \otimes |\widetilde{c}_j\rangle \otimes |d\rangle_j^{\otimes \lceil \log_2(Q) \rceil} = \\
&\frac{1}{\sqrt{2^N 2^{\lceil \log_2(Q) \rceil}}} \sum_{i=0}^{L-1} \sum_{j=0}^{Q-1} (|\widetilde{c}_1 \dots \widetilde{c}_{j-1}, a_1, \widetilde{c}_{j+1} \dots \widetilde{c}_Q\rangle |l_1 \dots l_N\rangle)_{i+u(\widetilde{c}_j)} \otimes |\widetilde{c}_j\rangle \otimes |d\rangle_j^{\otimes \lceil \log_2(Q) \rceil} =
\end{aligned} \tag{56}$$

where $u(\widetilde{c}_j)$ is the velocity associated to the channel $\widetilde{c}_j$. In the case of a particle going one lattice to the right and one lattice down, $u_x(\widetilde{c}_j) = 1$ and $u_y(\widetilde{c}_j) = -1$. Notice that in the above equation, the lattice site has been shifted, changing therefore the position of the channel stored in the ancilla register.

Figure 12: Scaling on terms of **CNOT** gates respect the total number of lattice sites N for 1D algorithm using different real quantum devices. As seen in the figure, the complexity scaling is equivalent, as it only depends on the propagation algorithm used. QFT-based propagation and superposition-based QLGA have been used in this case. In parenthesis, the mean connectivity of the different architectures.

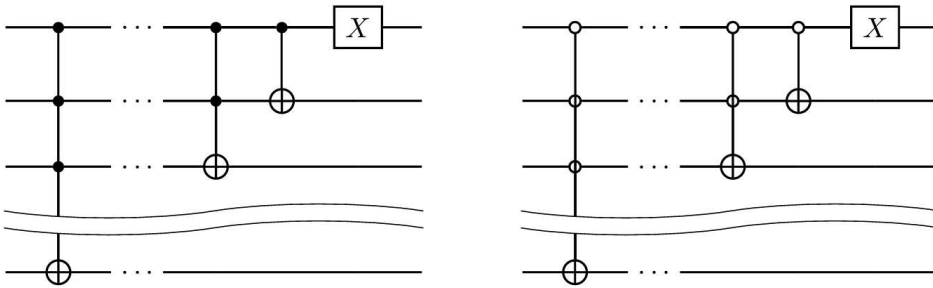


Figure 13: Simple quantum circuit for propagation using Toffoli gates. Additionally, these gates are conditioned on the channels depending on the direction of propagation.

- **Measurement:** The registers l and a are measured at the end of each time step. As the encoding in this model is binary, the states do not carry any information in the amplitudes or the phases. Information about the occupancy of the channel is stored in the qubit a_1 , and the particular channel is identified with the register d . This means that the qubit a_1 will identify which of the measured states are junk states and therefore do not carry useful information. The number of measurements will depend on the strategy used to do the algorithm and the fidelity we want to obtain but a high number of measurements is required to retrieve every particle with high probability. When the algorithm is applied in a real quantum computer, a larger number of measurements are necessary. However, if error rates are too high, the algorithm will fail even with a high number of measurements. More information about this will be shown in section 6.3.

Alternatively, it is possible to perform the algorithm without using the intermediary step. However, in order to do so, the algorithm has to become less quantum native by using a classical computer. The idea is to identify the states which undergo a collision with a new binary string and initialize the system in each time step with a a superposition of channels and lattice sites. Therefore, every different collision, will use an unique tag (a binary string) to perform the collision. This alternative requires a lower number of qubits as ancilla are not needed. However, this is not a great relative advantage as the number of channels is much lower than the number of lattice sites. An example of the algorithm applied to D1Q3 can be seen in Figure 11. We will call this secondary approach superposition-based QLGA. A summary of this implementation can be found in the next pseudo-code.

Algorithm 1 Superposition-based quantum LGA algorithm

```

1: procedure QLGA
2:    $|\psi\rangle_{\text{in}} \leftarrow \text{Initialize } |\psi\rangle_0$ 
3:   for  $t = 1$  to  $T$  do  $\triangleright T$  is the total number of time steps
4:      $|\psi\rangle_{\text{col}} \leftarrow \text{Collide } |\psi\rangle_{\text{in}}$   $\triangleright$  Collision affects only the states marked during
       post-processing
5:      $|\psi\rangle_{\text{prop}} \leftarrow \text{Propagate } |\psi\rangle_{\text{col}}$ 
6:      $\text{measurement} \leftarrow \text{Measure } |\psi\rangle_{\text{prop}}$ 
7:      $\text{state} \leftarrow \text{Post-process } \text{measurement}$   $\triangleright$  Mark each collision with an
       unique tag for next time step
8:   end for
9: end procedure=0

```

In specific, the initialization can be expressed mathematically as

$$|\psi\rangle_{\text{in}} = \mathbf{I}_{\text{D1Q3}} |\psi\rangle_0 = \mathbf{I}_{\text{D1Q3}} (|0\rangle_l^{\otimes n} \otimes |0\rangle_c^{\otimes 3}) \otimes |0\rangle_a^{\otimes 3} = \frac{1}{\sqrt{2^n}} \sum_{i=0}^N |l\rangle_i \otimes \sum_{k=0}^{n_i} \frac{1}{\sqrt{2^{n_i}}} |c\rangle_k \quad (57)$$

with n_i the number of particles in the lattice site i . Due to the simplicity of this algorithm and the lower amount of qubits and resources needed, we will use it in the following simulations for the 1D model as emulating a quantum computer with noise in a classical computer is computationally expensive. However, for the 2D model, we will make use of the first implementation with swap gates as we will use noiseless simulations which require less computation time. In the next sections, we will simulate the 1D model using open boundary conditions with high particle density in the centre. For the 2D case, we will simulate Pouiselle [200, pp. 298–302] flow making use of the FHP-III model of six particles which models Navier-Stokes equations.

6.2 Computational complexity

As introduced in the previous subsection, QLGA is composed of five major steps. Initialization, collision, mapping, propagation and measurement. As initialization and measurements are typical problems in introducing and extracting data from a quantum computer, we will not consider them in the following analysis.

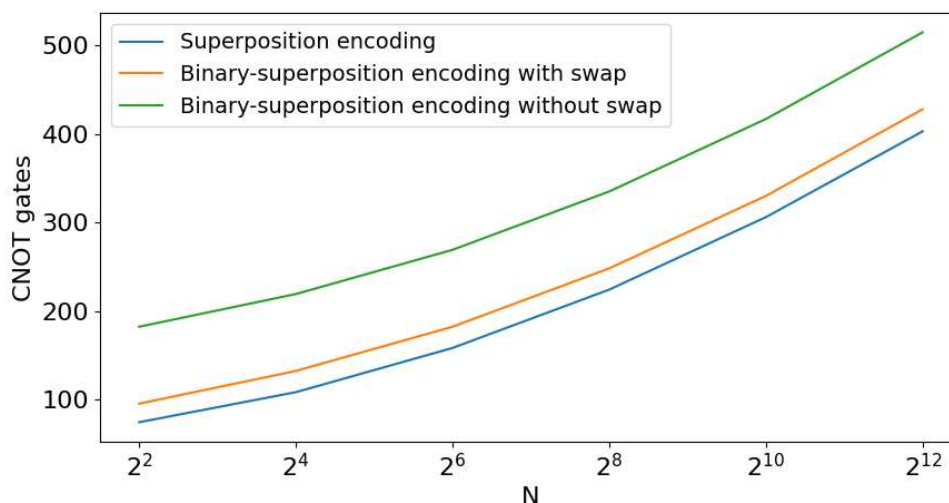


Figure 14: Scaling on terms of **CNOT** gates respect the total number of lattice sites N of the different algorithmic variants for 1D algorithm. As seen in the figure, the complexity scaling is equivalent, as it only depends on the propagation algorithm used. QFT-based propagation has been used in this case.

In the algorithm, the collision and mapping are steps dependent on the number of channels. As the number of channels is fixed and its values among lattice-gas models are small compared to the number of lattice sites, we will consider them to be $O(1)$. Therefore, in Figure 14, we do not observe any difference in the scaling of

the different implementations, as the only difference between them are the collision and mapping steps. However, the propagation step is dependent on the number of lattice sites and relies heavily on multi-controlled gates. In this regard, the two most efficient algorithms are the QFT propagation algorithm and the parallel shift algorithm as commented in the previous section. The QFT scales as $2n^2 - 4n + 2$ in terms of **CNOT** gates with n the number of qubits. This approach is more efficient for a low number of qubits (13 qubits or less). In contrast, the parallel shift algorithm scales as $n + 59$ and is more efficient for a higher number of qubits. These computational complexity dependencies have been obtained considering an all-to-all connectivity architecture. After transpilation in a real quantum device with different connectivity, the number of multi-controlled gates increases to map the original gates to the qubit connectivities. In Figure 15 we can see how the algorithm scales depending on the connectivity degree of the qubits of different real devices. We observe that when the connectivity decreases, the number of controlled gates increases and the scaling becomes superpolynomial.

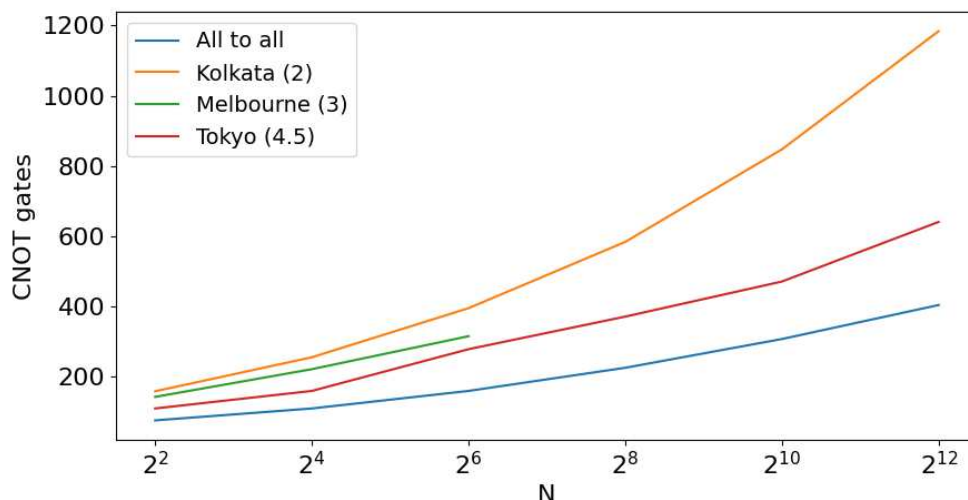


Figure 15: Scaling on terms of **CNOT** gates respect the total number of lattice sites N for 1D algorithm using different real quantum devices. As seen in the figure, the complexity scaling is equivalent, as it only depends on the propagation algorithm used. QFT-based propagation and superposition-based QLGA have been used in this case. In parenthesis, the mean connectivity of the different architectures.

We can also include a special collision step to model boundary conditions. However, it requires us to mark every lattice site where we want to activate the special collision rule. This means that this step encounters the same problems as the state preparation, where we need multi-controlled gates. In Figure 16, we can see how the number of **CNOT** gates change when increasing X and Y in the 2D model presented in this section. As the wall collision depends only on the number of lattice sites in y -axis, its complexity is $O(Y)$, compared to $O(\log^2(X))$ for

the x -axis due to the propagation step dependency. To reduce the computational complexity, we could use the decomposition of multi-controlled gates using extra ancilla qubits [16, pp. 183–184], possibly reducing the complexity to a logarithmic scaling. However, if the walls are complex and the marking procedure involves a high number of qubits, this will not be possible.

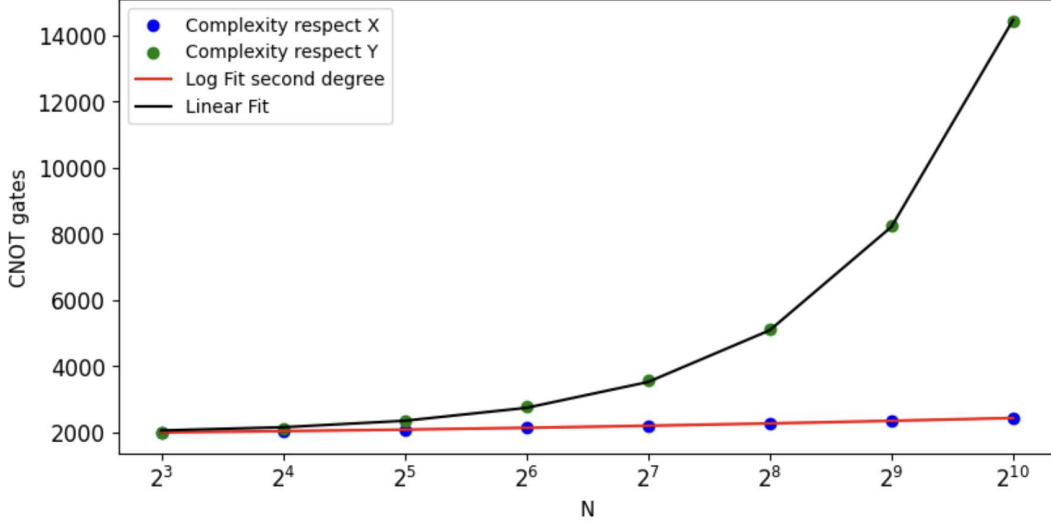


Figure 16: Computational complexity of QFHP comparing its scaling with X and Y with top and bottom wall boundary conditions. When X increases, Y is set to 8 and vice versa. As the gates used to encode the boundary condition scale with the register used to store the y -axis lattice sites, its computational complexity is linear (in the number of lattice sites) when scaling Y . On the other hand, the complexity respect X is $O(\log^2(x))$ due to the complexity of the QFHP-based propagation step. In the figure N is the number of lattice sites in the scaled direction.

6.3 1D model D1Q3 -Numerical results

In order to verify the model is working correctly and test how it behaves under noisy devices, we present here two benchmark tests: A comparison with the classical LGA and a comparison of the effect of simulated noise dependant on the number of shots. The quantum circuits for collision and mapping can be observed in Figures 9, 10, 11 and 13.

To begin with, we will compare the QLGA with its classical version. The simulation is done using 512 lattice sites with a sublattice of size 32. This means that 32 lattice sites are used for each lattice site plotted. This is done in order to reduce noise. Additionally, as the number of qubits we can simulate is limited, several realizations are simulated and averaged to smooth the final results. For the initial condition, a high probability distribution on the centre of the lattice with 95% occupancy is used while the rest of the lattice has 5% occupancy in terms

of channels occupied with respect to the total. The direction of these channels is generated randomly in each realization. After conducting different simulations, we can observe that the results are equivalent to the classical case, with small variations due to the random initialization. The simulations were performed using Aer simulator from Qiskit SDK [32]. As we can see in Figure 17, we obtain similar results with respect to the classical version and the results clearly show the propagation and diffusion modeled using D1Q3. This validates for 1D (toy-model) that we can simulate LGA using a quantum algorithm. In the next section, we will use a more advanced model which can replicate Navier-Stokes equations and obtain the macroscopic variables compared with the theoretical results.

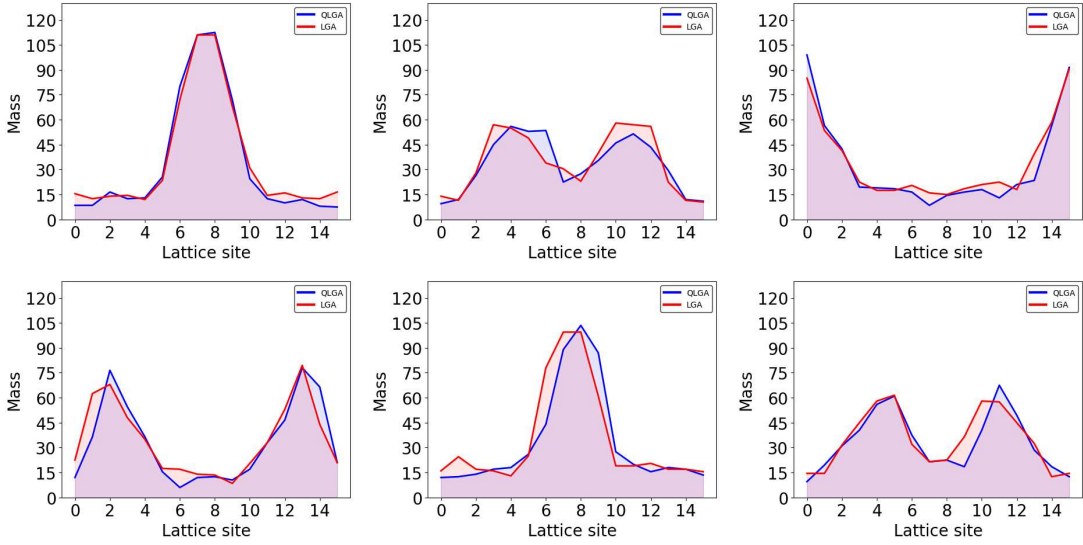


Figure 17: Comparison between LGA and QLGA for different time steps ($t = 0$, $t = 160$, $t = 320$, $t = 560$, $t = 752$, $t = 896$).

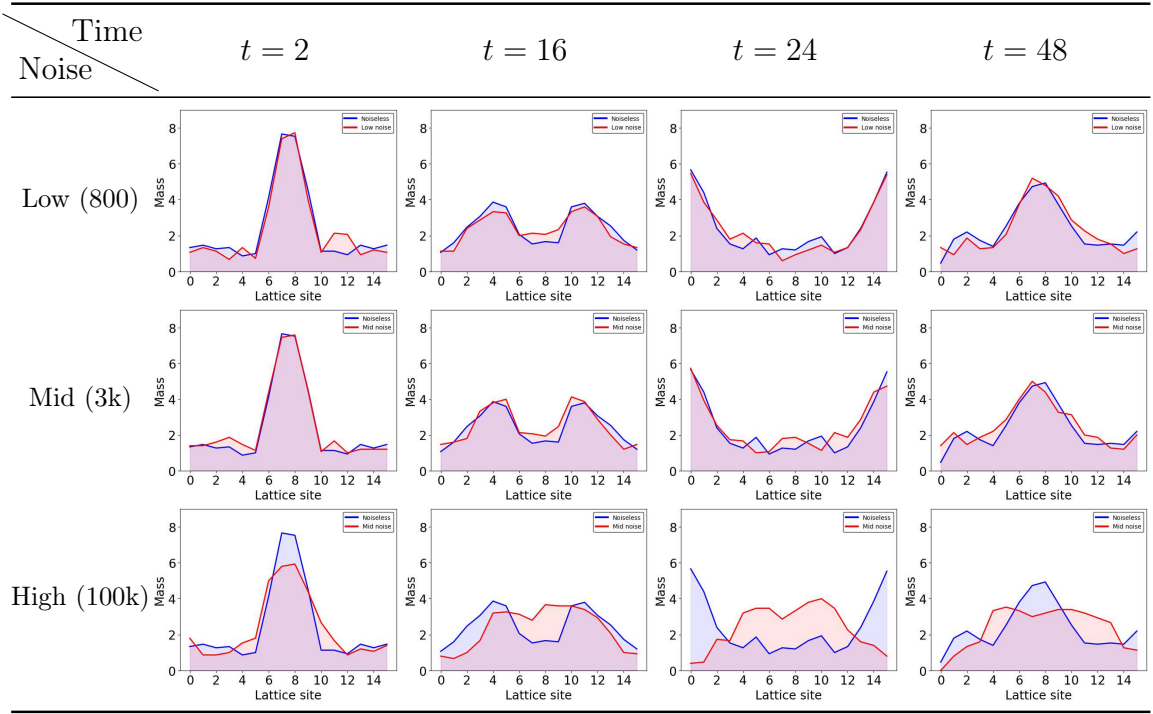
Nevertheless, quantum algorithms are not executed in ideal conditions. Current quantum devices have noise which affects the accuracy of each gate to perform the correct manipulation in the quantum states. As the circuit gets larger in depth, noise propagates and increases, creating large errors. This is the case for an increasing number of lattice sites, increasing the depth and therefore the error rates. To test how the algorithm behaves under noisy conditions, we use different error rates and number of shots. Due to the high cost of quantum computers and limited access to machines with varying error rates, we tested the algorithm with the Qiskit Aer simulator and constructed simple models to replicate the noise of real quantum devices. The error rates of the machine were manually included to go from early quantum devices (high noise) to low error rates as presented in Table 2. The mid-level noise corresponds to Quantinuum H2 device [38], available today. As presented in [201], depolarising errors are the major contributors to noise in real devices. Consequently, our simulations primarily incorporate these depolarising errors, along with measurement errors, to realistically emulate the noise. In order to simplify the analysis, we have used the native gates from IBM

devices: $\mathbf{R}_z$, $\mathbf{S}$, $\mathbf{SX}$ and $\mathbf{CX}$.

Table 2: Benchmarking Model Resilience to Noise

Noise Level	1-Qubit Error	2-Qubit Error	Readout Error
Low	10^{-5}	10^{-4}	10^{-4}
Mid	3×10^{-5}	2×10^{-3}	2×10^{-3}
High	6×10^{-3}	2×10^{-2}	2×10^{-2}

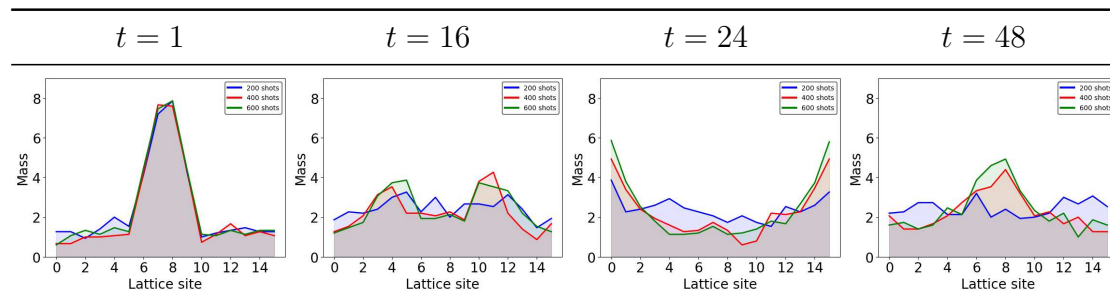
Table 3: Comparison between QLGA with different noise levels. The number of shots used appears in parentheses. The number of shots is adjusted to the minimum necessary for accurate simulations. Wave propagation is not observed in the model of high noise rate using up to 100k shots. An ensemble average of fifteen is used. Additionally, the noiseless simulation has been performed with 350 shots, which is found to measure all the states with high probability for this simulation. In the simulation, the number of lattice sites $L = 32$ and a sublattice of size two was used



In Table 3, we can see how the QLGA simulated with all-to-all connectivity varies its results from low to high noise compared to the noiseless version. As the accuracy of the results increases with the number of shots, we have used the minimum number of measurements in each case that allowed us to observe diffusion and propagation similar to the noiseless version. In the figures, we can see how accurate results can be obtained for the low and mid error rate levels, which corresponds to quantum devices such as Quantinuum H2. However, when the noise is high enough, we begin to measure different particles at early time steps which propagate for the

rest of the simulation. For the case of the high error rate, the shots were increased up to 100k with no correspondence with the ideal simulation. This emulated error simulation indicates that simulations could be done with current quantum devices. However, the 1D model is a simple model, which cannot simulate Navier-Stokes equations and therefore useful simulations will require a higher number of **CNOT** gates and therefore higher depth in quantum circuits. These considerations will make the simulations need a higher number of shots to obtain accurate results. Additionally, the number of shots is dependent on the total number of particles of the model and the number of times steps used. In this case, 40 particles were added for each realization on average. Another consideration that is important to notice is that the emulated error models usually underestimate noise from real quantum devices, which are challenging to emulate, further decreasing the fidelities.

Table 4: Table comparison with low noise QLGA with a different number of shots. A spatial average of two sublattice sites and an ensemble average of 15 are used, therefore not integer mass values.



To compare how sensitive the model is to a different number of shots, we tested the D1Q3 model with superposition encoding with a different number of shots using the low level for error rates. In Table 4, we can see how the lattice sites with a higher density of particles smooth out with a lower number of shots. It is not until 600 shots that we begin to differentiate between junk states and the desired states consistently and obtain accurate results. However, some errors persist until the number of shots is increased to 800. As seen in the figure, after 8 times steps, the information is mostly lost for the case of 200 shots and the two characteristic moving waves are no longer observed. For the case of 400 shots, discrepancies appear when the waves collide, observing a lower density of particles propagated to the limits of the lattice. As mentioned earlier, open boundary conditions were used in this case. However, as the initial condition is symmetric, the resultant effect is similar to a wall.

6.4 2D model QFHP and Numerical results

The QFHP is based on the classical FHP model introduced in section 3.1. We can name this model as an FHP-III with six channels instead of seven, as we include every collision rule except the rest particle collisions. Even though the general algorithm is equivalent to the D1Q3 model, the QFHP has some particularities. These are the grid encoding (hexagonal lattice), the collision rules, the propagation and the boundary conditions. First, we will address these novelties and then expose the numerical results obtained simulating a Poiseuille flow between parallel plates.

6.4.1 Introducing the 2D QFHP

The first problem that we encounter when we want to create a quantum FHP model, is how to encode the lattice in an efficient manner for a quantum computer. Hexagonal lattices have a different distribution of lattice points between the odd and even rows which create a non-rectangular shape in the borders. To avoid this, we make use of a regular square lattice with different spacing distances. The x -direction have a $\Delta x = \frac{1}{2}$ and y -direction $\Delta y = \frac{\sqrt{3}}{2}$. This means that to do a simulation with the same distance in x and y -directions, we need $\sqrt{3}$ more lattice sites in the x -direction. This implies that the number of lattice sites shifted in the propagation step for each channel will be $c_1 = (2, 0)$, $c_2 = (-2, 0)$, $c_3 = (1, -1)$, $c_4 = (-1, 1)$, $c_5 = (-1, -1)$, $c_6 = (1, 1)$. To propagate in the different directions, we will use a QFT-based propagation algorithm using ancilla and Toffoli-gates to control which channels are propagated and how many times, making use of three sub-propagation steps to shift every channel (two in the x -axis and one in the y -axis).

Lattice-gas automata models differ mainly in two properties: The Bravais lattice on which the model is built and the collision rules. In this case, the collision rules change completely as we have three times more particles and they are defined on a hexagonal lattice. Here we introduce a general procedure to perform the collision rules of any model. We can see the collision step as a mapping between states within a complete collisional invariant group. This means that as shown in Figure 5, the mass and x and y momentum components have to remain the same. This was introduced already by Fonio *et al.* [117]. For example, for the collision rules described in FHP-I, the collision of three particles with zero momentum can be done by rotating the channel's propagation directions 180 degrees and all the states within the group are switched. In the case of the two particles, the same applies, where a rotation of 120 degrees clock or anti-clockwise can rotate the states within a complete invariant group. Therefore, we will need three steps for any arbitrary collision rule we want to apply. First, identify the states within a complete invariant group, then mark them and finally rotate them. An example of this is

shown in Figure 18. As introduced by Fonio, the marking will be based on logical expressions. For example, for the two-particle collision with zero momentum, we can write the following logical expression to identify the required states

$$b = (c_6 \oplus c_3) \wedge (c_5 \oplus c_2) \wedge (c_4 \oplus c_1) \quad (58)$$

with $\oplus$ as **XOR** using **CNOT** operator and $\wedge$ as **AND** operators using a generalize Toffoli gate. Furthermore, as any of the states in a complete invariant group is marked in the same way, we can recover the ancilla qubits used in the process by applying the marking step back again.

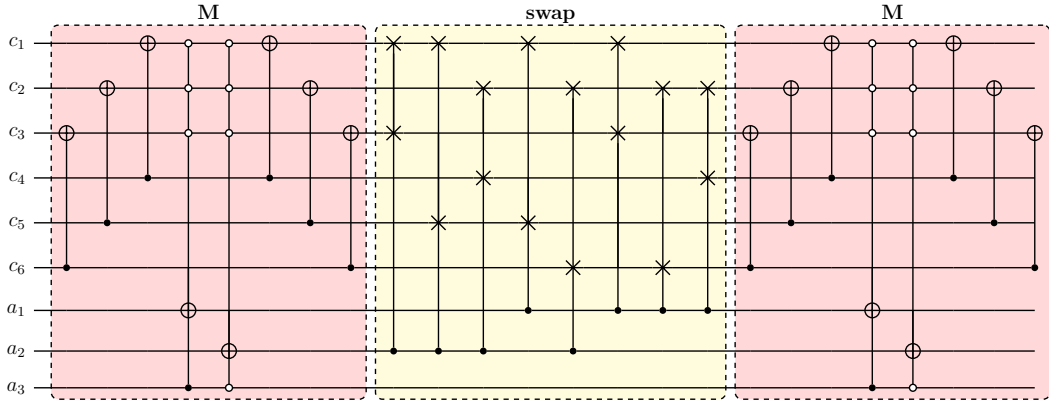


Figure 18: Two particles collision rule in the QFHP model. The step **M** marks the states with two particles and zero momentum and the **swap** rotate their direction in 120 degrees, depending if the collision is a clock or anti-clockwise according to the ancilla qubit a_3 . We have used the notation $c_1 = (1, 0)$, $c_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_3 = (-\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_4 = (-1, 0)$, $c_5 = (-\frac{1}{2}, -\frac{\sqrt{3}}{2})$, $c_6 = (\frac{1}{2}, -\frac{\sqrt{3}}{2})$ as directions of the channels

However, this procedure may not be the most optimal in cases where the invariant groups are numerous and small in quantity and other invariances can be found. Specifically, introducing the three particle collisions with non-zero momentum requires a large number of gates in order to identify them and restore the ancilla qubits. As we want to reduce the number of qubits used as much as possible, we will use another methodology to find an efficient circuit to reduce the number of gates and incorporate every transformation. Therefore, we have used the previous procedure only for three-particle-collision rules with zero net momentum (Figure 19), making use of an alternative for the rest of channel transformations. In this case, the logical expression is

$$b = (c_1 \oplus c_2) \wedge (c_2 \oplus c_3) \wedge (c_3 \oplus c_4)(c_4 \oplus c_5) \wedge (c_5 \oplus c_6) \quad (59)$$

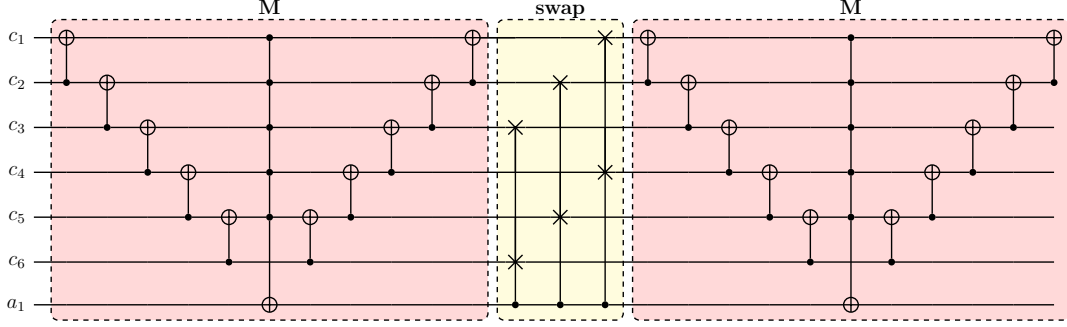


Figure 19: Three particles collision rule in the QFHP model. The step **M** marks the states with three particles and zero momentum and the **swap** rotate their direction in 120 degrees, depending if the collision is clock or anti-clock wise according to the ancilla qubit a_3 . We have used the notation $c_1 = (1, 0)$, $c_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_3 = (-\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_4 = (-1, 0)$, $c_5 = (-\frac{1}{2}, -\frac{\sqrt{3}}{2})$, $c_6 = (\frac{1}{2}, -\frac{\sqrt{3}}{2})$ as directions of the channels

The collision rules for the two and four particles will not be deterministic but will depend on a random bit for each lattice site, which will change every time step. This means that the collision outcome from a particular state will depend on every lattice site. This randomness will be encoded in the initialization classically using an extra ancilla qubit and added as a conditional gate to the collision. The collisions to implement in our QFHP six-channel model, can be seen in Figure 5. We can observe that all collisions, except for the specific case of a three-particle collision resulting in zero momentum, involve head-on collisions between particles. These collisions occur under three conditions: when no other channel is occupied, when one other channel is occupied by a particle, or when two or more channels are occupied by additional particles. Therefore the general rule for applying this transformation, will be to identify whether we have two head-on particle collisions and if the particular outcome direction after the collision is unoccupied. After the marking step, in order to reduce the number of qubits used, we will unmark the ancilla qubits. Unfortunately, after swapping the states to do the collision, this is not always possible. In contrast to the previous methodology, here we do not have complete groups with states that are permuted during the collision. Therefore, we used an ancilla qubit to accumulate junk information and keep the rest of ancilla in the ground state. Therefore, we use effectively only one more qubit with respect to the previous approach but with a much lower number of multi-controlled gates. A schematic overview of the circuit for one of the three possible head-on collisions can be seen in Figure 20. The rest of the collisions work in the same way, having two **swap** steps for the two possible random collision directions and therefore two marking steps. In successive quantum circuits for head-on collisions of two particles with opposite momentum, we must only add extra gates to ensure that the states have not collided yet, adding control in the a_f qubit. The final number of multi-controlled gates of the collision rule is 1100 in contrast with 100 that we had for the D1Q3 with the binary-superposition swap method as shown in Figure 14. However, the number of gates used for the collision can be reduced

by decomposing some multi-controlled gates used by adding ancilla qubits.

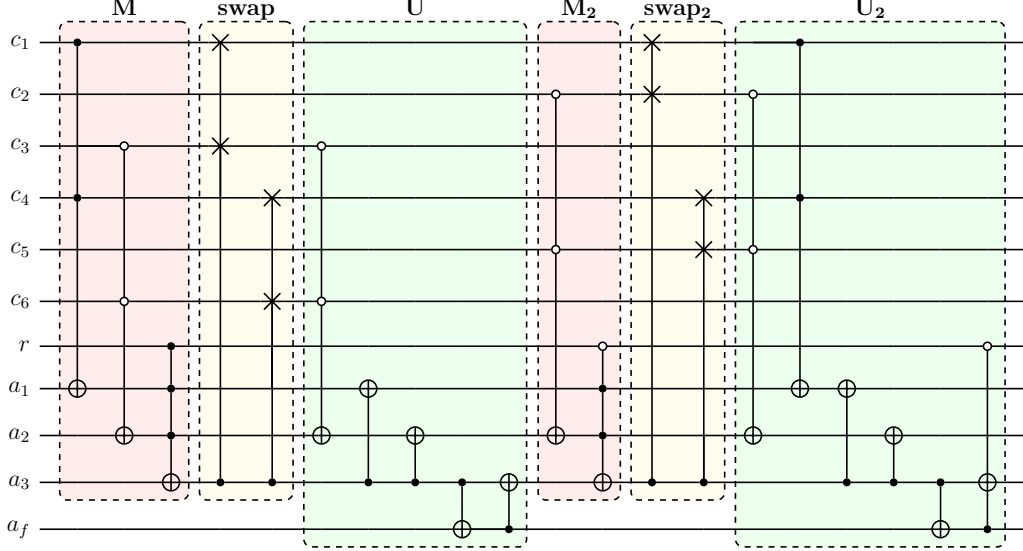


Figure 20: Quantum circuit for all possible Head-on collision rules of channels c_1 and c_4 . The sections **M** and **M₂** are marking states. **U₁** and **U₂** are recovering ancilla qubits and storing whether we encountered a collision in a_f . The **swap** sections are swapping the particles between channels to realize the transformation through the rotation. We have used the notation $c_1 = (1, 0)$, $c_2 = (\frac{1}{2}, \frac{\sqrt{3}}{2})$, $c_3 = (\frac{-1}{2}, \frac{\sqrt{3}}{2})$, $c_4 = (-1, 0)$, $c_5 = (\frac{-1}{2}, \frac{-\sqrt{3}}{2})$, $c_6 = (\frac{1}{2}, \frac{-\sqrt{3}}{2})$ as directions of the channels

The last step to model real-life cases is to establish the boundary conditions in the model. Specifically, for a Poiseuille flow benchmark test, we need top and bottom walls with no-slip conditions and an inlet boundary with set velocity. There are two alternatives to introduce this velocity. The first is making use of special collision rules that add certain momentum in every time step. An example of this alternative was elaborated by Kanadoff *et al.* [202]. The second option is to introduce particles at every time step which adds momentum to the flow simulated. To simplify the model and take advantage of the classical initialization in each time step, we made use of this second alternative to add the particles classically. This reduces the computational time needed to execute the quantum circuit in a classical device.

In contrast to the inlet velocity, the walls need to be introduced using a quantum circuit. As introduced in Section 3.1.3, there are three special collisions on walls: Bounce-back, specular and diffusive reflections. In this case, we have used the most simple of them, which is the bounce-back collision rule. The quantum circuit to implement this, is based on two steps. First marking the lattice sites with particles colliding with the wall during the next propagation step and second, changing their internal channels to the opposite direction. Notice that this direction must be unoccupied as the walls are blocking particles coming from such direction.

Therefore we can constitute the collision step with the standard collision rules and special transformation with the walls. More details of the computational complexity dependencies of this model can be seen in Figure 16.

6.4.2 QFHP Poiseuille flow: Numerical results

Novel numerical methods for CFD are often tested using benchmarks in which theoretical expressions or experimental results are already known. This allows to compare the accuracy and (in)stability of the method with others. To show the applicability of LGA and its quantum version, we will carry out a test simulating a Poiseuille flow with top and bottom walls. A scheme of the benchmark can be seen in Figure 21. In this particular case, we have used periodic boundary conditions adding particles at the extremes of the domain to keep a constant gradient of pressure. Later, we will discuss the results without periodic boundary conditions, using an outlet.

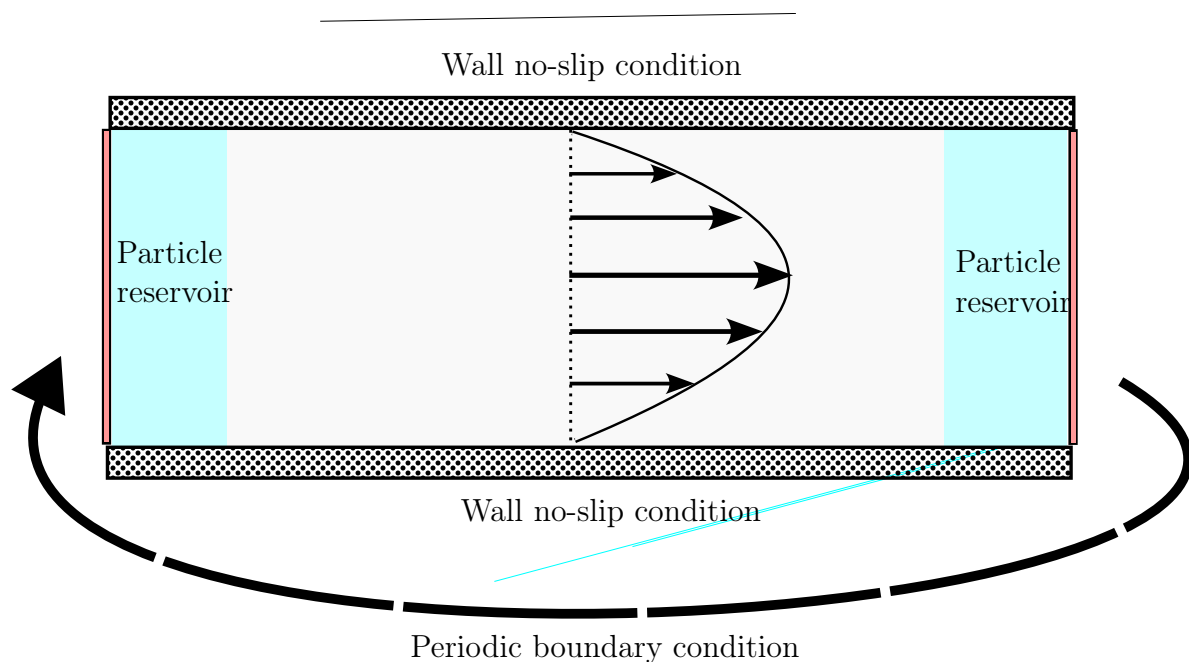


Figure 21: Laminar flow between parallel plates (Poiseuille flow). A benchmark used to validate FHP and QFHP. The boundary conditions consist of top and bottom walls with no-slip conditions and inlet and outlet with periodic boundary conditions where particles are injected with a certain probability.

As it is known in fluid dynamics [200, pp. 297], the equation of motion for an

uniform density and incompressible fluid without gravitational forces is

$$\rho \frac{Du}{Dt} = -\nabla p + \mu \nabla^2 u \quad (60)$$

where D is used to note the material derivative such that

$$\frac{Du}{Dt} = \frac{\partial u}{\partial t} + u \nabla u \quad (61)$$

As the equation is nonlinear, it is not possible to obtain an analytical solution in the majority of the cases. However, this is not the case for the steady flow between parallel plates. If we maintain a pressure gradient of particles from the beginning of a pipe to the end, the velocity profile of the pipe will evolve until it is fully developed. This means that the solution will be steady and theoretically we can prove it will be parabolic. A simpler case of this is the flow between two parallel plates. Using the incompressibility condition we know that

$$\frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} = 0 \quad (62)$$

As $\frac{\partial u_x}{\partial x} = 0$ due to the definition of developed flow, $\frac{\partial u_y}{\partial y} = 0$. Now using no-slip boundary conditions at the walls $u_{y|_{wall}} = 0$ and given that u_y velocity does not change along y , we have that $u_y = 0$ everywhere. In summary, the x and y momentum equations are

$$\begin{aligned} 0 &= -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \frac{d^2 u_x}{dy^2} \\ 0 &= -\frac{1}{\rho} \frac{\partial p}{\partial y} \end{aligned} \quad (63)$$

where we have used the kinematic viscosity $\nu = \frac{\mu}{\rho}$. Solving these equations, we obtain

$$u = -\frac{y}{2\mu} \frac{dp}{dx} (Y - y) \quad (64)$$

with Y the height of the channel. This corresponds to a parabolic velocity profile. It is important to notice that this is only applicable to a laminar and incompressible flow. We will consider a flow to be laminar if its Reynolds number $Re < 1000$, such that

$$Re = \frac{Y \bar{u}}{\nu} \quad (65)$$

with $\bar{u}$ the mean velocity. We will consider a flow to be incompressible for a Mach number $M = \frac{u}{c_s}$ under 0.5 where c_s is the speed of sound. We can now average (64) along the channel

$$\bar{u} = -\frac{1}{Y} \int_{y=0}^{y=Y} \frac{y}{2\mu} \frac{dp}{dx} (Y - y) dy = -\frac{1}{12\mu} \frac{dp}{dx} Y^2 \quad (66)$$

Using (64) and (66), we can express the velocity profile without using the viscosity nor the pressure gradient.

$$u = 6\bar{u}\frac{y}{Y^2}(Y - y) \quad (67)$$

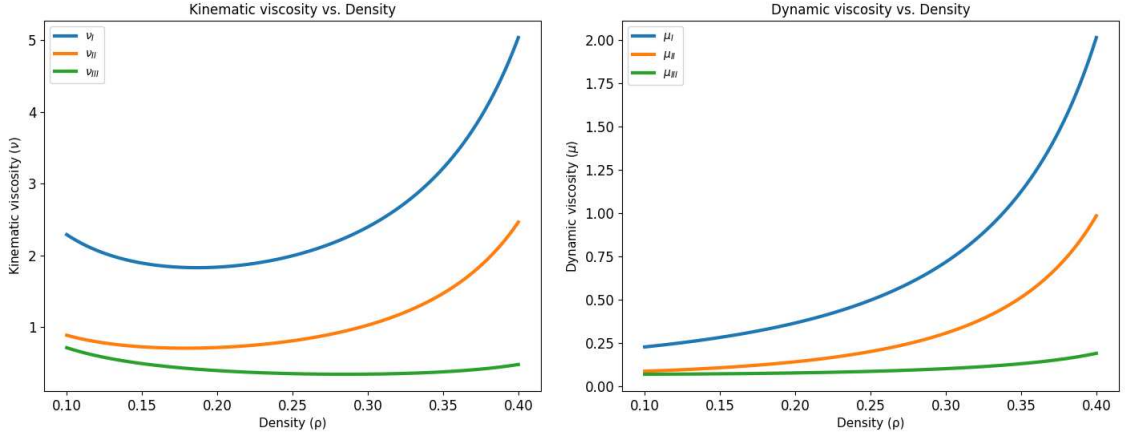
Finally, we must address the question of how many time steps to simulate to have a fully developed flow. In this regard, we can make use of the viscous entrance length for laminar flow

$$L_d = 0.0575 D Re \quad (68)$$

Once we know the theory, it is important to obtain the corresponding theoretical macroscopic quantities from LGA such as Reynolds number, sound velocity and viscosity of the model. These parameters will depend on the FHP model used and can be found in Wolf Gladrow's book on LGA and LBM [135, pp. 74–76]. The speed of sound for FHP depends on the number of channels and the lattice used. For our model of six particles $c_s = \frac{1}{\sqrt{2}}$. However, the viscosity will depend on the collision rules used and the number of particles. As we are not using a standard model (see Figure 5, in our case we are not using the collision with rest particle), we need to derive theoretically the viscosity. However, this is complex and not the focus of this research. Therefore, we will approximate the viscosity of our model to be somewhere between FHP-II and FHP-III. In LGA, the viscosity can be obtained as $\nu = \frac{\nu_g}{g(\rho)}$, with ν_g the galilean viscosity and $g(\rho)$ the non-galilean term. As this term depends on the channels of the model, we will use $g_{FHP_I}(\rho) = \frac{1}{2} \frac{1-2\rho}{1-\rho}$. Therefore, the kinematic viscosity of our model will be between FHP-II and FHP-III using the non-galilean term for six particles. Next the kinematic viscosities for FHP models.

$$\begin{aligned} \nu_I &= 2 \frac{1-\rho}{1-2\rho} \left(\frac{1}{12} \frac{1}{\rho(1-\rho)^3} - \frac{1}{8} \right) \\ \nu_{II} &= 2 \frac{1-\rho}{1-2\rho} \left(\frac{1}{12} \frac{1}{\rho(1-\rho)^3(1-\frac{4\rho}{7})} - \frac{1}{8} \right) \\ \nu_{III} &= 2 \frac{1-\rho}{1-2\rho} \left(\frac{1}{28} \frac{1}{\rho(1-\rho)(1-\frac{8\rho(1-\rho)}{7})} - \frac{1}{8} \right) \end{aligned} \quad (69)$$

with ρ the density per lattice site. We can see these equations plotted in Figure 22.



(a) Kinematic viscosity with respect to density for FHP-I to FHP-III models. (b) Viscosity with respect to density for FHP-I to FHP-III models

Figure 22: Comparison between FHP-I to FHP-III viscosities respect density using (69)

We can confirm this results plotting the Reynolds coefficient $Re^* = \frac{Re}{YM} = \frac{c_s}{\nu}$. As observed in Figure 23. The pressure can be also obtained for the limit of low Mach number M as $p = c_s^2 \rho$.

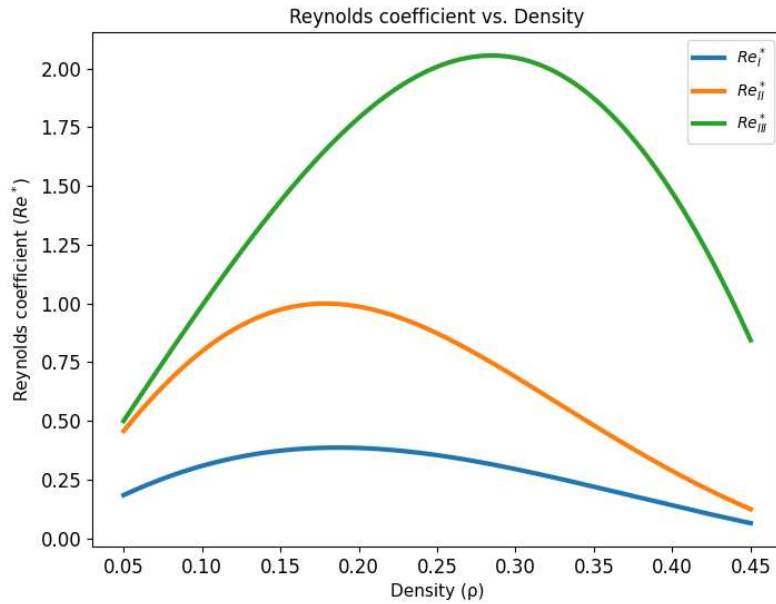


Figure 23: Reynolds coefficient $Re^* = \frac{c_s}{\nu}$ as a function of the density using (69).

To verify the model, we will first compare LGA with theoretical results from Navier-Stokes equations. The velocity profile can be obtained directly using (67) or

estimated with (64) using the pressure derivative obtained from a central difference scheme or a pressure fit. However, if we use the pressure gradient with a linear fit, the value of the kinematic viscosity is needed. In order to obtain its value, we can use (69) (FHP-II and FHP-III). As the flow initial ρ_0 density is low and then increases along the simulation by injecting particles, we will use the mean density during the flow development in the theoretical expressions. Another method to estimate the viscosity is to use the velocity and pressure fits to obtain the value μ from the Navier-stokes equations using our simulation data. As we know from (64), we can estimate the effective mean viscosity using a quadratic fit for the velocity

$$u(y) = ay^2 + by + e \quad (70)$$

and a linear fit for the pressure

$$p(x) = cx + d \quad (71)$$

Therefore, we get

$$\begin{aligned} \mu_{theo} &= \frac{Yc}{2b} \\ \mu_{theo} &= \frac{c}{2a} \end{aligned} \quad (72)$$

where $c = \frac{dp}{dx}$.

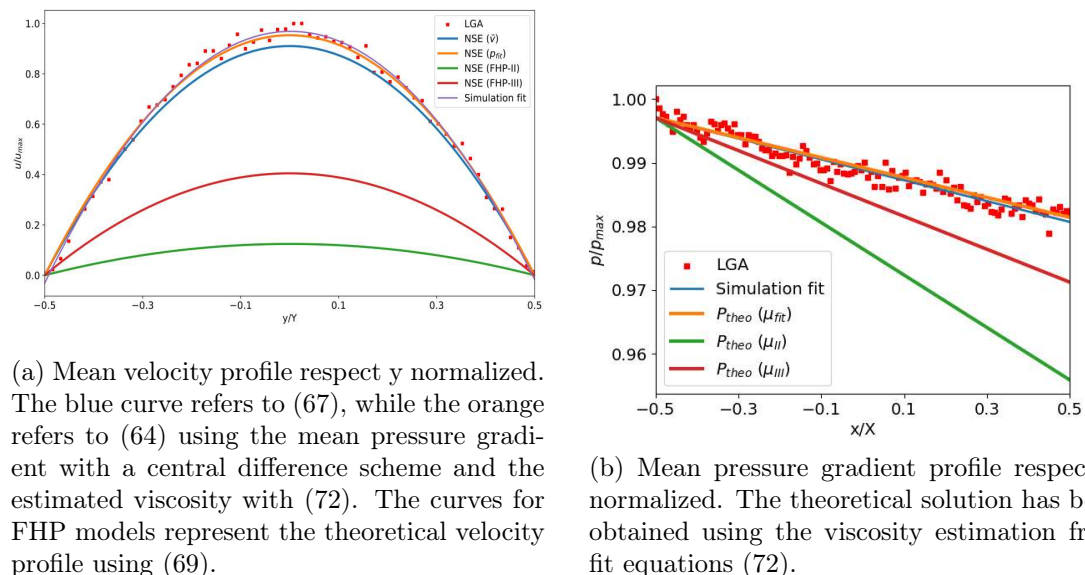


Figure 24: Flow between parallel plates simulation according to scheme given by Figure 21, using a lattice with 64×128 sites. The simulation time was 3840 time steps and the probability of adding particles $p = 0.01$ in the first and last 5% of the lattice. The initial conditions are set with initial density $\rho_0 = 0.2$, meaning that 20% of channels are occupied with random velocities. The mach number $M = 0.35$, the predicted Reynolds number $Re = 170$ and the viscosities $\mu_{theo} = 0.045$, $\mu_{II} = 0.36$ and $\mu_{III} = 0.11$. Therefore the simulated flow is laminar and incompressible.

The results of the simulation can be seen in the Figure 24. We obtain accurate results for both the velocity profile and the pressure gradient. In specific, the velocity profiles obtained are very similar independently of the method used to calculate it and both correspond to the fit from the simulation data noted as (LGA). Both the pressure and velocity profiles are the mean values obtained by averaging along x and y direction respectively and during the last 380 times steps. The velocity at the walls for $y = 0$ and $y = Y$ is zero, showing that we have indeed no-slip conditions. Even though the results are accurate compared to the theory, the estimation of the viscosity respect to the FHP models is very different to the theoretical values obtained using (69). Considering that we are using a model between FHP-II and FHP-III, we could expect an intermediary viscosity estimation. However, the current discrepancy cannot be explained by this fact. After different simulations, we observed that the longer the simulation is, the smaller the theoretical viscosity μ_{theo} becomes, which is the opposite tendency to what we expected. The reason is that as we increase the number of particles with time, the density increases and following Figure 22, the viscosity should increase. Therefore, to obtain a match between the theoretical viscosity and the viscosity from the FHP models, we will change the boundary conditions using an inlet (which adds particles with no y net momentum) without periodic boundary conditions as shown in Figure 25. In specific, the simulation time and the domain length was adjusted to develop the flow according to (68), not letting the flow reach the end of the domain.

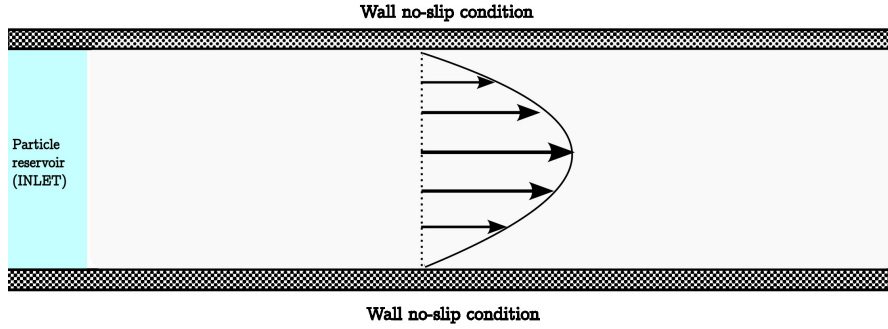
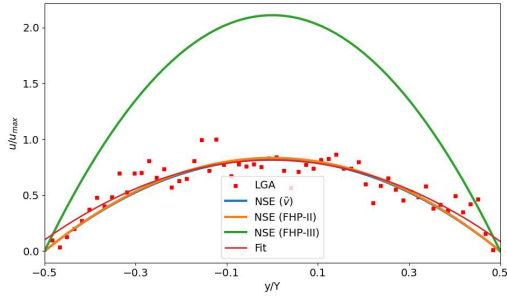


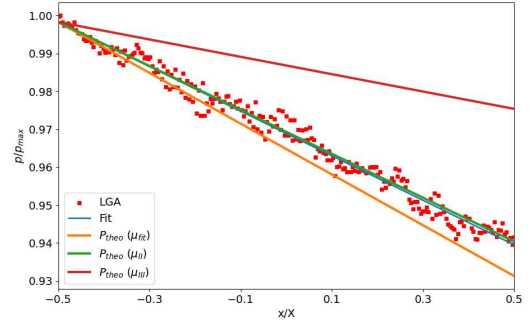
Figure 25: Laminar low between parallel plates (Poiseuille flow) benchmark used to verify FHP and QFHP. The boundary conditions consist of top and bottom walls with no-slip conditions and a long channel set according to the hydrodynamic entrance length given by (68).

In this new setting, the density, pressure and velocity will be averaged during the interval at which the flow reaches $x = L_d$ using (68) when it is developed to $x = X$. This means that the measure time starts at $t = T_d$ with $T_d = \frac{L_d}{U_{max}} = \frac{L_d}{2\bar{v}}$. According

to Figure 26, the results are again accurate respect the theory. However, this time, the estimation of the viscosity is in the expected interval (between FHP-II and FHP-III). Notice that in this case, the averaging of the velocity is done along the flow. This means that we average 50 lattice sites centred on the flow and we slide the section where we calculate the velocity and density according to the maximum flow velocity u_{max} . As we can see, using this method, we obtain the same velocity and pressure profile as in the FHP-II model. This is plausible according to our hypothesis. Additionally, we have found this fit to be accurate when increasing the width of the channel or modifying the initial conditions. We can further test the estimation of the viscosity using the model FHP-I as it is a model of six particles and we know its exact viscosity from (69). The Figure 27 indicates that we obtain accurate results, which validates the model and the methodology to estimate the viscosities used previously.



(a) Mean velocity profile respect y normalized. The blue curve refers to (67). The curves for FHP models represent the curve using instead using the viscosities described previously for FHP models (22)



(b) Mean pressure respect x normalized. The theoretical solution has been obtained using the viscosity estimation from fit equations (72)

Figure 26: Flow between parallel plates simulation according to Figure 21 using a lattice of 64×1000 sites which correspond to 55.4×500 scaling the lattice with $\Delta x = 0.5$ and $\Delta y = \sqrt{3}/2$. The simulation time was 1000 time steps and the probability of adding particles $p = 0.01$ in the first 5% of the lattice. The initial conditions are set with initial density $\rho_0 = 0.2$, meaning that 20% of channels are occupied with random velocities. The mach number $M = 0.21$, the Reynolds number $Re = 18$ and the viscosities are $\mu_{theo} = 0.27$, $\mu_{II} = 0.24$ and $\mu_{III} = 0.09$. Using (68), we have $L_d = 113$ lattice sites.

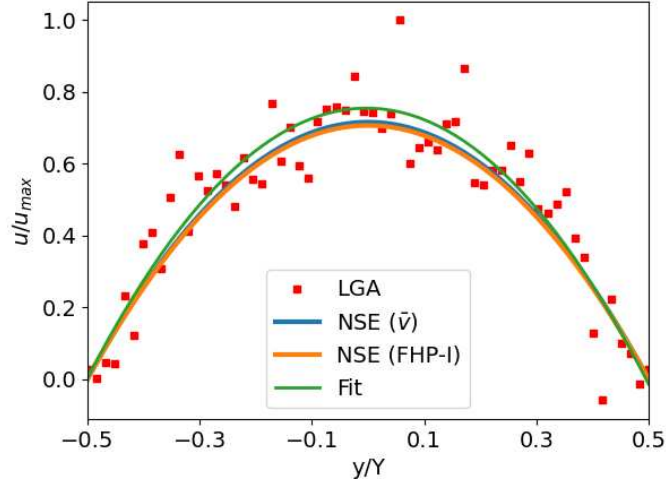
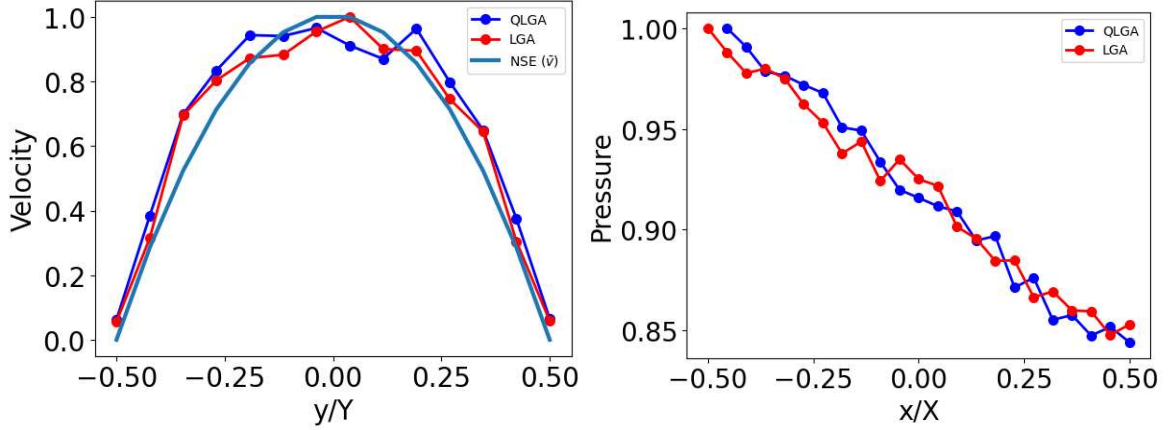


Figure 27: Flow between parallel plates simulation according to Figure 25 using a lattice of 64×600 sites. All other settings are kept similar as in Figure 26. The mach number $M = 0.15$, the Reynolds number $Re = 7$ and the viscosities $\mu_{theo} = 0.49$, $\mu_I = 0.52$. Using (68), we have $L_d = 42$ lattice sites.

Finally, we can compare the LGA automata with the developed QLGA and the theoretical solution using the mean velocity as (67). Figure 28 shows how the quantum version gives the same result as the classical one with small variations due to the random nature of the particles added in the initialization and in the inlet. It is important to notice that we need to use the first scheme given by Figure 21 as we are limited in the number of lattice sites we can emulate in a classical computer using the quantum algorithm. This is also the reason why we use a low number of lattice sites. However, despite these constraints, we obtain accurate results between both models, which are preserved when doing simulations with different initial conditions. We also tested that we obtain the exact same result if no particles are added and we begin with the same initial condition.



(a) Mean velocity profile respect y normalized.

(b) Mean pressure respect x normalized.

Figure 28: QLGA compared to LGA with scheme given by Figure 21 using a lattice of 16×32 sites which correspond to 13.8×16 scaling the lattice with $\Delta x = 0.5$ and $\Delta y = \sqrt{3}/2$. The simulation time was 64 time steps and probability of adding particles $p = 0.5$ in the first 30% of the lattice. The initial conditions are set with initial density $\rho_0 = 0.2$.

6.5 Further research and challenges

6.5.1 Monte-Carlo QLGA

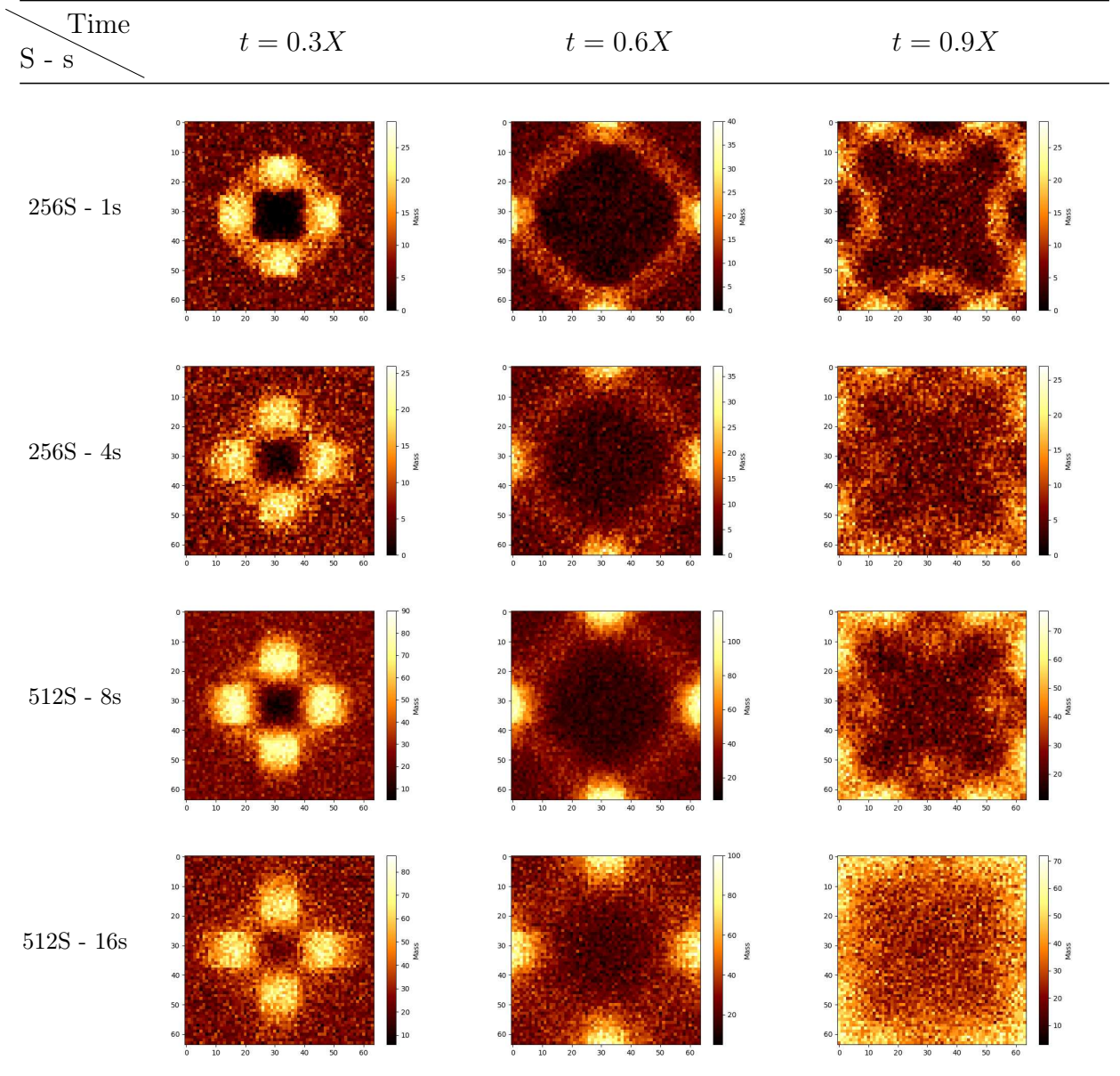
Lattice-gas automata (LGA) have been shown to generate nonlinear dynamics through the interaction of particles in a complex system. These interactions are facilitated by collision and propagation operators, which are consistent across all particles, thereby enabling parallelization. Quantum computers present an excellent opportunity to accelerate computational fluid dynamics (CFD) simulations using this method.

However, a major challenge in contemporary QLGA research is the difficulty in concatenating time steps. Once the state for the subsequent time step is determined, it becomes technically challenging to reapply the algorithm. This difficulty arises from the superposition used in the propagation, which changes the encoding to a superposition of channels. To continue the simulation, measurement is required, which incurs a high computational cost. The quantum algorithm has a computational complexity of $O(n^2)$, with $n = \log_2(N)$ being N the number of lattice sites. The simulation must be performed a ϵ number of times (shots) to extract accurate results. Mathematically, we express $\epsilon = N\rho\eta$ where $\rho \leq 1$ is the density and η is a factor related to how many times measurement is necessary per each particle occupied. Therefore the total complexity including measurement is $O(N\log_2^2(N))$. This increases further taking into account the initialization, which

makes QLGA to scale as $O(N^2 \log_2^2(N))$. In contrast, the complexity of the classical algorithm is $O(N)$. This makes the quantum algorithm less optimal and therefore not useful for acquiring a quantum advantage. There are three alternatives for faster simulations: Decreasing the number of measurements, reducing the complexity of the algorithm or concatenating time steps by adding additional ancilla qubits. Reducing the computational complexity would not solve the exponential scaling in terms of qubits and other approaches are necessary. On the other hand, we have not yet found a methodology to concatenate time steps without intermediary measurements. However fewer measurements is possible if we can generate the next time step with less information. Measuring using momentum and mass macroscopic variables by the expected value of an operator, and using them to generate the microscopic dynamic in every time step, could be useful for this purpose. This means, that we do not need to measure every particle, leading to fewer measurements.

The algorithm operates in a similar manner to LGA. Upon completion of one-time step, the mass and momentum of different sublattices are preserved in memory. For example, if we have a 32×32 lattice, we can divide this into 8×8 lattice with each 1×1 site composed by 4×4 subsites. Subsequently, the particles must be generated from the macroscopic variables, ensuring that we preserve mass and momentum in each sublattice. This is achieved using a Monte Carlo method [203]. For each sublattice, an m -array is created, where m represents the number of sites within the sublattice. The array contains l number of "ones", equal to the total absolute net momentum of the lattice in random positions (using random variables). The sublattice is then populated with particles of corresponding momentum at the randomly assigned positions. Once the net momentum is encoded, we need to add up to the total mass. This can be done using $1+D$ random variables, where D is the total of dimensions. For instance for a $2D$ -lattice, two random variables ranging from 0 to $m - 1$ are generated, determining where the next particles will be placed. Then a third random number (0 or 1) sets in which directions the particles will be generated (preserving zero net momentum). Notice that for an hexagonal grid, there are three possibilities to include the particles with zero-momentum. Therefore, we verify if a specific cell has two unoccupied particles in that specific direction (up-down or left-right for HPP). Then we populate the lattice sites again. This approach ensures that we end with the macroscopic mass and momentum, filling the final mass with pairs of particles with opposite momentum. Therefore, we indicate here a possible improvement to QLGA algorithm which can be developed by measuring less and therefore increasing its efficiency.

Table 5: Comparison between HPP model and Monte-Carlo HPP (MCHPP) for different lattice (S) and sublattice (s) sites. For example, for 256S - 1s, we refer to the HPP model with a 256x256 lattice as 1s corresponds to no averaging. Additionally, as the number of lattice sites changes, we note the time by its fraction respects the number of lattice sites X .



We can find next an overview of the algorithm using pseudocode.

Algorithm 2 Monte-carlo LGA

```
1: particles  $\leftarrow$  LGA(Initial state)
2: for each sublattice (i) do
3:   momentumx[m], momentumy[m], mass[m]  $\leftarrow$  Counting(particles[N, C])
4:   for each element of sublattice (j) do
5:     Generate random array particlesx, particlesy with momentumx[j] and
     momentumy[j] number of "1"
6:     while momentumx[j]  $\neq$  0 and momentumy[j]  $\neq$  0 do
7:       Lattice is filled depending on the sign of the momentum and the net
       momentum
8:       (x, y)  $\leftarrow$  obtainposition(i, j)           if momentumx[j] > 0 then
10:         lattice[x, y, 0] = 1
11:         momentumx[j] - = 1
12:         mass[j] - = 1
13:
14:         if momentumx[j] < 0 then
15:           lattice[x, y, 1] = 1
16:           momentumx[j] + = 1
17:           mass[j] - = 1
18:         end if
19:         if momentumy[j] > 0 then
20:           lattice[x, y, 2] = 1
21:           momentumy[j] - = 1
22:           mass[j] - = 1
23:         end if
24:         if momentumy[j] < 0 then
25:           lattice[x, y, 3] = 1
26:           momentumy[j] + = 1
27:           mass[j] - = 1
28:         end if
29:       end while
30:   end for
31:   while mass[j]  $\neq$  0 do
32:     randomx, randomy, randomd  $\leftarrow$  Random number generator
33:     (x, y)  $\leftarrow$  fobtainposition(i, randomx, randomy)
34:     if randomd == 0 and lattice[x, y, 0] == 0 and lattice[x, y, 1] == 0 then
35:       lattice[x, y, 0] = 1
36:       lattice[x, y, 1] = 1
37:       mass[j] - = 2
38:     end if
39:     if randomd == 1 and lattice[x, y, 2] == 0 and lattice[x, y, 3] == 0 then
40:       lattice[x, y, 2] = 1
41:       lattice[x, y, 3] = 1
42:       mass[j] - = 2
43:     end if
44:   end while
45: end for
```

To test the efficiency of the algorithm, we have compared HPP with Monte-Carlo HPP (MCHPP) for different numbers of lattice sites and sublattices as seen in Figure 3. If we compare the first row of figures with the second, we observe that CHPP with a sublattice of size four is comparable to the HPP. This indicates that the algorithm can be used to reduce the measurements as expected with low loss of accuracy. In the next row, we increase the lattice sites and sublattice sites by 2, having the same proportion between them. In this case, we obtain better results, closer to the expected behaviour. This means that the larger the lattice is, the better results we get, when increasing the sublattice in proportion to the total number of lattice sites. Therefore, we may think that we could maintain accuracy while increasing the sublattice at an higher rate than we increase the lattice sites. To confirm this hypothesis, we increase the sublattice sites for the last case (fourth row) with 512×512 using 8 and 16 sublattices. As we see in the figure, the results are accurate and comparable to the second and third row. However, as time passes, we lose accuracy and this is no longer the case. We must take into account, that we are reducing the number of measured lattice sites in this case from 262000 to 1000. This gives several conclusions. Firstly, the higher the number of sublattice sites is, the more noise you introduce to the model. Additionally, a longer simulation duration also contributes to an increase in noise. However, there is an additional advantage. By proportionally increasing the number of lattice and sublattice sites, we can decrease the accuracy resulting from this algorithmic approximation. This also implies that as we scale up the number of sites, we can effectively reduce the noise further by using the same sublattice-to-lattice ratio. This discovery holds promise for quantum lattice-gas automata (QLGA), as it suggests that as the lattice scales, so can the sublattice, reducing further the number of measurements in a quantum algorithm, potentially providing an advantage. Therefore, we propose the following steps to further explore this idea for QLGA:

- Create operators and extract momentum and mass with their expectation values for different sublattices in the quantum algorithm.
- Obtain the computational complexity of the postprocessing. Quantum advantage without considering initialization.
- Tackle state preparation: There are two possibilities to initialize states in a quantum computer using random variables. We can initialize probabilistically directly in the quantum state preparation circuit or generate them classically and use them in the quantum computer. While the first option could lead to a more efficient simulation, it seems challenging given the computational cost of initializing a quantum circuit.

6.5.2 QFHP with airfoil simulation

In this thesis so far we have tested the LGA model for a Poiseuille flow between parallel plates and confirmed that the most advanced quantum model designed here (QFHP-III with six particles) gives the same results as its classical counterpart. However, these are still laminar simple cases, where the analytical solution is known. Given the success and the advances developed in this thesis, a future research direction is testing this model under more complex flows. This has not been explored in the bibliography to the best of the author's knowledge. In particular, one of these research interests is airfoil simulation. This is not only interesting for academic purposes, but also for the industry, as stated in the recent quantum challenge by BMW and Airbus [204]. Therefore, we have tested LGA using this model with a NACA0012 aerofoil and compared with a similar simulation using Openfoam software [205] with simpleFoam solver for stationary solutions. The results from Figure 29 show that the final results are very similar. During the simulation we verified that when adding more lattice sites and time steps, the results improve as expected. However, it is important to clarify that the walls' boundary conditions are implemented with collisions and not with concrete conditions such as no-slip or slip walls. Additionally, the Reynolds number of both simulations could differ as the estimation of the first case's viscosity is unclear and we are using different units (time step and lattice sites distance for the LGA).

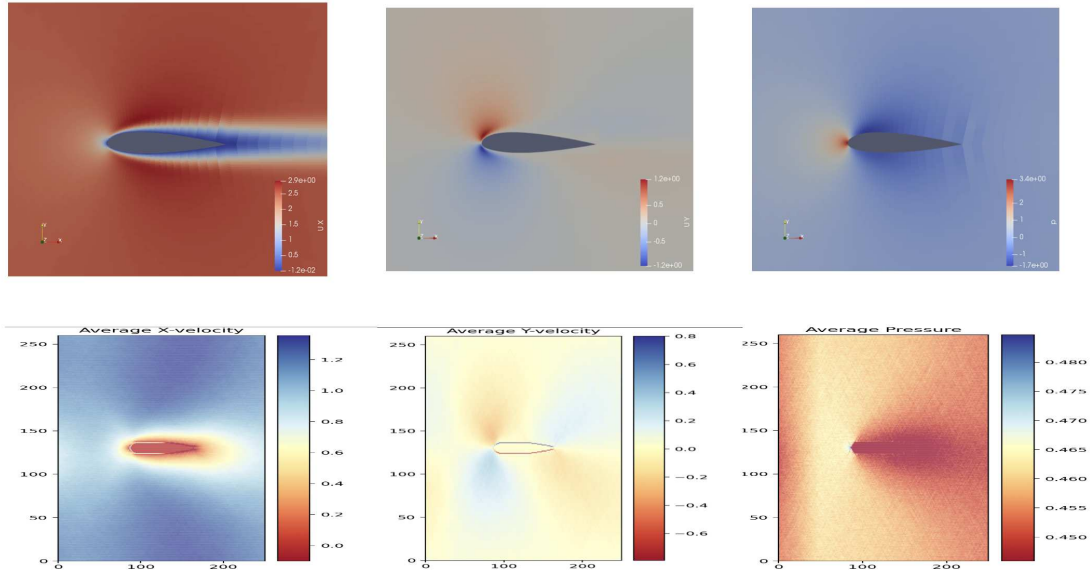


Figure 29: Airfoil simulation comparison between LGA and openfoam with airfoil NACA0012. A lattice of 500x300 was used, which translates to 250x260 after the scaling and 7500 time steps. For openfoam, simpleFoam solver was used with a mesh provided by openfoam airfoil tutorial [206].

After concluding that LGA can model an airfoil with a moderate number of lattice sites, we proceed to simulate this airfoil using QLGA. This demands the incorporation of a new step (generalize special collision rules). This method, has not yet been completely implemented successfully, as some particles do not bounce as expected in the walls. However, the model still gives similar results to LGA with a similar number of time steps, see Figure 30. The number of lattice sites is small as implementing boundary conditions with QLGA demands a marking procedure with Toffoli gates, similar to the one used previously during the QFHP collision circuit in Figure 20, making larger classical simulations of the quantum algorithm infeasible for now.

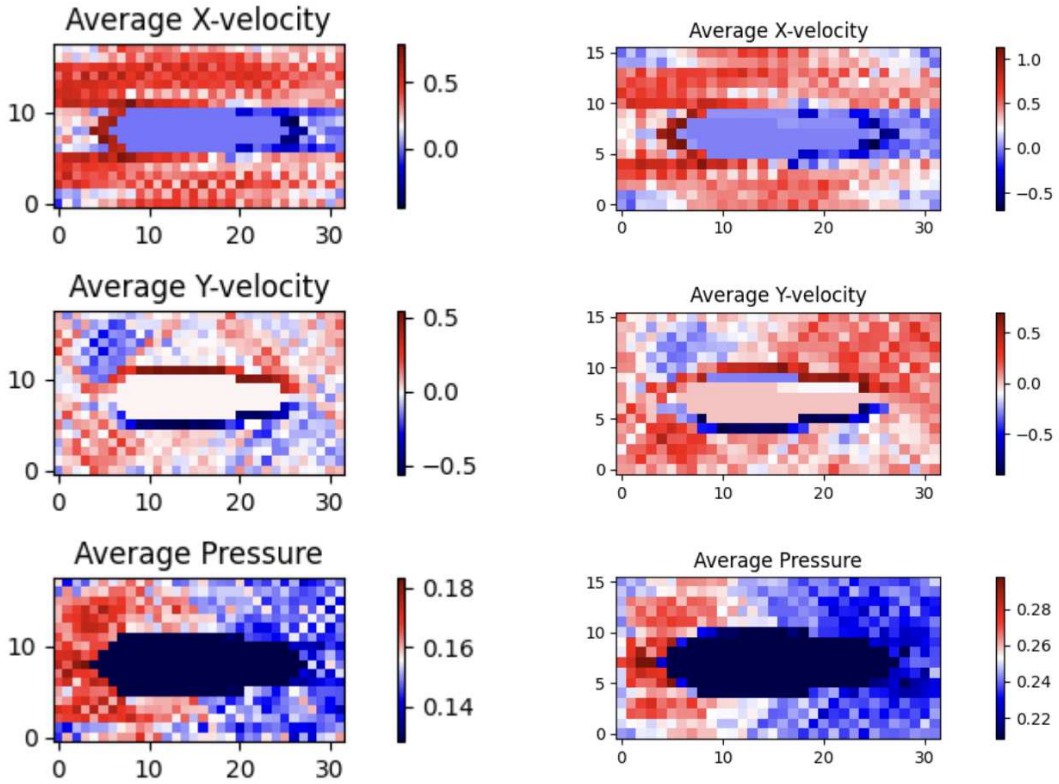


Figure 30: Airfoil simulation comparison between LGA (left) and QLGA (right) with 32x16 lattice sites. Velocity (u_x and u_y) and pressure is presented.

7 Conclusion

The goal of this work was to find the most relevant quantum algorithms for fluid dynamics and develop one of them further. In this regard, the most promising algorithms for CFD were found to be QLGA and QLBM. These lattice-based models can take advantage of the exponentially increasing dimensionality of the Hilbert space used in quantum computing, holding the potential for scaling en-

gineering problems beyond the limits of classical computing. However, other algorithms such as linear and nonlinear solvers (HHL among others), variational quantum algorithms, and a novel quantum finite volume method, were found to be good candidates for quantum CFD simulations, too. These algorithms could potentially lead quantum speedups. Nevertheless, when certain steps, such as the initialization or the measurement, are taken into account, no advantage is found at present. Additionally, these algorithms encounter limitations in replicating nonlinear differential equations due to the inherent linear nature of quantum computing. However, QLBM using a novel approach based on Carleman linearization, and the main topic of this thesis, QLGA, are not only scalable but also provide an intuitive and natural way to solve nonlinear terms in the equations.

We focused on developing novel QLGA algorithms, designing two new quantum algorithms. The first one is based on D1Q3 LGA (one-dimensional and three channels in each lattice) and the second is based on the FHP model with six particles (two-dimensional model). The algorithms follow the same steps as the classical version (collision and propagation) with additional steps to be adapted to quantum computing. These steps are initialization, mapping and measurements. In this work, we provided the quantum circuits of the different main steps of each algorithm, an analysis of the computational complexity and numerical simulations proving that these models can be used to solve CFD problems.

QLGA has the potential to provide an exponential advantage due to the large number of states that we can use in a quantum computer. Additionally, the algorithm decreases the complexity from $O(N)$ to $O(\log^2(N))$ without considering measurements and initialization. However, in their current stage, initialization and the inability to connect time steps (continue the simulation during several time steps without measurements) is a major obstacle to obtaining this final and pursued goal. On the other hand, we showed that QLGA is noise-resilient and can be simulated in current quantum devices with higher fidelity than similar quantum algorithms such as QLBM. This indicates how relevant this algorithm is but also that quantum computing could be used sooner rather than later for real applications. Specifically, we provided a comparison using a noise emulator with different noise levels and different shots on the evolution of the 1D algorithm. Finally, we tested QFHP using a well-known CFD benchmark (Flow between parallel plates) obtaining the same results as in LGA. We also demonstrated that the classical algorithm correctly models flows according to the analytical solution for the incompressible laminar Navier-Stokes equations.

To sum up, QLGA is an algorithm that can model nonlinear terms, is noise-resilient and can be used for simulations of real use cases, with a few remaining challenges still to be explored.

References

- [1] MeetIQM, “State of quantum report 2024.” <https://www.meetiqm.com/newsroom/press-releases/state-of-quantum-report-2024>, 2024.
- [2] M. Digital, “Quantum technology sees record investments, progress on talent gap,” 2024.
- [3] Wikipedia, “List of quantum processors — Wikipedia, the free encyclopedia,” 2023.
- [4] G. Buonaiuto, F. Gargiulo, G. De Pietro, *et al.*, “Best practices for portfolio optimization by quantum computing, experimented on real quantum devices,” *Scientific Reports*, vol. 13, p. 19434, 2023.
- [5] B. Lanyon, J. Whitfield, G. Gillett, *et al.*, “Towards quantum chemistry on a quantum computer,” *Nature Chemistry*, vol. 2, pp. 106–111, 2010.
- [6] D. Bernstein and T. Lange, “Post-quantum cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [7] S. Succi, W. Itani, K. Sreenivasan, and R. Steijl, “Quantum computing for fluids: Where do we stand?,” *Europhysics Letters*, vol. 144, p. 10001, oct 2023.
- [8] A. W. Harrow, A. Hassidim, and S. Lloyd, “Quantum algorithm for solving linear systems of equations,” *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.
- [9] S. Y.-C. Chen, C.-H. H. Yang, J. Qi, P.-Y. Chen, X. Ma, and H.-S. Goan, “Variational quantum circuits for deep reinforcement learning,” *IEEE Access*, vol. 8, pp. 141007–141024, 2019.
- [10] J. Hardy and Y. Pomeau, “Thermodynamics and hydrodynamics for a modeled fluid,” *Journal of Mathematical Physics*, vol. 13, pp. 1042–1051, 1972.
- [11] J. Hardy, Y. Pomeau, and O. De Pazzis, “Time evolution of a two-dimensional model system. i. invariant states and time correlation functions,” *Journal of Mathematical Physics*, vol. 14, no. 12, pp. 1746–1759, 1973.
- [12] J. Hardy, O. de Pazzis, and Y. Pomeau, “Molecular dynamics of a classical lattice gas: Transport properties and time correlation functions,” *Phys. Rev. A*, vol. 13, pp. 1949–1961, May 1976.
- [13] D. D’HUMIERES, P. LALLEMAND, and U. FRISCH, “Lattice gas models for 3d hydrodynamics,” *Europhysics Letters*, vol. 2, no. 4, pp. 291–297, 1986.

- [14] S. Chen and G. D. Doolen, “Lattice boltzmann method for fluid flows,” *Annual Review of Fluid Mechanics*, vol. 30, no. 1, pp. 329–364, 1998.
- [15] P. Muratore-Ginanneschi, J. Pekola, and B. Karimi, “TCM315 Fall 2022: Open Quantum Systems - Lecture 4,” 2022.
- [16] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.
- [17] F. Gieres, “Mathematical surprises and dirac’s formalism in quantum mechanics,” *Reports on Progress in Physics*, vol. 63, no. 12, p. 1893, 2000.
- [18] A. Aiello, “Spectral theorem for dummies: A pedagogical discussion on quantum probability and random variable theory,” 2022.
- [19] “Operator algebras and quantum statistical mechanics 1: C*- and w*-algebras. symmetry groups. decomposition of states,” vol. 13, no. 1, p. 500.
- [20] R. Jozsa, “Entanglement and quantum computation,” in *Geometric Issues in the Foundations of Science* (S. e. a. Huggett, ed.), 1997. arXiv:quant-ph/9707034.
- [21] E. J. Brändas and E. S. Kryachko, *Schrödinger’s Wave Equation – A Lie Algebra Treatment*. Springer Netherlands, Jan 2003.
- [22] J. Huang, “Lie groups and their applications to particle physics: A tutorial for undergraduate physics majors,” Dec 2020.
- [23] “A single quantum cannot be cloned,” vol. 299, p. 802–803.
- [24] A. K. Pati and S. L. Braunstein, “Impossibility of deleting an unknown quantum state,” *Nature*, vol. 404, pp. 164–165, 2000.
- [25] E. Johnston, N. Harrigan, and M. Gimeno-Segovia, *Programming Quantum Computers: Essential Algorithms and Code Samples*. O’Reilly Media, 2019.
- [26] M. Suneel, “Dynamical systems as logic gates,” 2005.
- [27] J. Long, T. Zhao, M. Bal, R. Zhao, G. S. Barron, H.-s. Ku, J. A. Howard, X. Wu, C. R. H. McRae, X.-H. Deng, G. J. Ribeill, M. Singh, T. A. Ohki, E. Barnes, S. E. Economou, and D. P. Pappas, “A universal quantum gate set for transmon qubits with strong zz interactions,” 2021.
- [28] B. Bauer, C. Levaillant, and M. Freedman, “Universality of single quantum gates,” 2014.
- [29] D. Deutsch, A. Barenco, and A. Ekert, “Universality in quantum computation,” 1995.
- [30] J.-L. Brylinski and R. Brylinski, “Universal quantum gates,” 2001.

- [31] L. He, D. Liu, J. Gao, W. Zhang, H. Zhang, X. Feng, Y. Huang, K. Cui, F. Liu, W. Zhang, and X. Zhang, “Super-compact universal quantum logic gates with inverse-designed elements,” *Science Advances*, vol. 9, no. 21, p. eadg6685, 2023.
- [32] Qiskit contributors, “Qiskit: An open-source framework for quantum computing,” 2023.
- [33] Q. A. team and collaborators, “Cirq,” Oct. 2020.
- [34] IBM, “Ibm quantum experience,” 2023.
- [35] Qiskit contributors, “Qiskit documentation: Ecrgate,” 2023.
- [36] Qiskit contributors, “Qiskit documentation: Sxgate,” 2023.
- [37] IONQ, “Getting started with native gates,” 2023.
- [38] Quantinuum, “System model h2 product data sheet,” 2023.
- [39] Qiskit contributors, “Qiskit documentation: Tgate,” 2023.
- [40] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, “Open quantum assembly language,” Jul 2017.
- [41] Q. D. Team, “Qiskit transpiler documentation.” <https://qiskit.org/documentation/apidoc/transpiler.html>, 2023.
- [42] M. Xu, Z. Li, O. Padon, S. Lin, J. Pointing, A. Hirth, H. Ma, J. Palsberg, A. Aiken, U. A. Acar, and Z. Jia, “Quartz: Superoptimization of quantum circuits (extended version),” 2022.
- [43] J. Pointing, O. Padon, Z. Jia, H. Ma, A. Hirth, J. Palsberg, and A. Aiken, “Quanto: Optimizing quantum circuits with automatic generation of circuit identities,” 2021.
- [44] Y. Nam, N. J. Ross, Y. Su, A. M. Childs, and D. Maslov, “Automated optimization of large quantum circuits with continuous parameters,” *npj Quantum Information*, vol. 4, p. 23, Nov 2018.
- [45] L. Moro, M. G. A. Paris, M. Restelli, and E. Prati, “Quantum compiling by deep reinforcement learning,” *Communications Physics*, vol. 4, p. 178, Nov 2021.
- [46] QuTech, “Quantum computing,” 2024.
- [47] J. Roffe, “Quantum error correction: an introductory guide,” *Contemporary Physics*, vol. 60, no. 3, pp. 226–245, 2019.

- [48] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Phys. Rev. A*, vol. 86, p. 032324, Sep 2012.
- [49] I. Staff, “The value of classical quantum simulators.” <https://ionq.com/resources/the-value-of-classical-quantum-simulators>, 2023.
- [50] O. Ezratty, “Is there a moore’s law for quantum computing?,” Mar 2023.
- [51] “A volumetric framework for quantum computer benchmarks,” *Quantum*, vol. 4, p. 362.
- [52] J. Preskill, “Quantum computing in the nisq era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [53] K. Bertels, T. Sarac, A. Sarkar, and I. Ashraf, “Quantum computing – from nisq to pirq,” 2021. arXiv:2106.11840 [quant-ph].
- [54] J. Krupansky, “Beyond nisq: Terms for quantum computers based on noisy, near-perfect, and fault-tolerant qubits,” 2021.
- [55] X.-C. Wu, S. Di, E. M. Dasgupta, F. Cappello, H. Finkel, Y. Alexeev, and F. T. Chong, “Full-state quantum circuit simulation by using data compression,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC ’19, (New York, NY, USA), Association for Computing Machinery, 2019.
- [56] H. Neven and J. Kelly, “Suppressing quantum errors by scaling a surface code logical qubit,” Feb 2023.
- [57] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Phys. Rev. A*, vol. 86, p. 032324, 2012.
- [58] M. Möttönen, J. Vartiainen, V. Bergholm, and M. Salomaa, “Transformation of quantum states using uniformly controlled rotations,” *Quantum Information & Computation*, vol. 5, p. 467–473, Sep 2005.
- [59] V. Shende, S. Bullock, and I. Markov, “Synthesis of quantum-logic circuits,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 6, pp. 1000–1010, 2006.
- [60] C. D. H. . L. C. L. H. Floyd M. Creevey, “Gasp: a genetic algorithm for state preparation on quantum computers,” *Scientific Reports*, vol. 13, no. 1, p. 11956, 2023.
- [61] “Efficient deterministic preparation of quantum states using decision diagrams,” *Physical Review A*, vol. 106, p. 022617.

- [62] F. P. . A. J. d. S. Israel F. Araujo, Daniel K. Park, “A divide-and-conquer algorithm for quantum state preparation,” *Scientific Reports*, vol. 11, p. 6329, Nov 2021.
- [63] X.-M. Zhang, T. Li, and X. Yuan, “Quantum state preparation with optimal circuit depth: Implementations and applications,” *Phys. Rev. Lett.*, vol. 129, p. 230504, Nov 2022.
- [64] N. Gleinig and T. Hoefler, “An efficient algorithm for sparse quantum state preparation,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 433–438, 2021.
- [65] “Quantum state preparation using tensor networks,” *Quantum Science and Technology*, vol. 8, no. 3, pp. 1–12, 2023.
- [66] J. Zylberman and F. Debbasch, “Efficient quantum state preparation with walsh series,” Nov 2023.
- [67] F. Mozafari, H. Riener, M. Soeken, and G. De Micheli, “Efficient boolean methods for preparing uniform quantum states,” *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–12, 2021.
- [68] F. Mozafari, M. Soeken, H. Riener, and G. De Micheli, “Automatic uniform quantum state preparation using decision diagrams,” in *2020 IEEE 50th International Symposium on Multiple-Valued Logic (ISMVL)*, pp. 170–175, 2020.
- [69] Q. Inspire, “Qubit Basis States.” <https://www.quantum-inspire.com/kbase/qubit-basis-states/>, 2024.
- [70] J. Cotler and F. Wilczek, “Quantum overlapping tomography,” *Phys. Rev. Lett.*, vol. 124, p. 100401, Mar 2020.
- [71] H.-Y. Huang, R. Kueng, R. Kueng, and J. Preskill, “Predicting many properties of a quantum system from very few measurements,” *Nature Physics*, vol. 16, p. 1050–1057, Jun 2020.
- [72] M. Kohda, R. Imai, K. Kanno, K. Mitarai, W. Mizukami, and Y. O. Nakagawa, “Quantum expectation-value estimation by computational basis sampling,” *Phys. Rev. Res.*, vol. 4, p. 033173, Sep 2022.
- [73] E. Knill, G. Ortiz, and R. D. Somma, “Optimal quantum measurements of expectation values of observables,” *Phys. Rev. A*, vol. 75, p. 012328, Jan 2007.
- [74] J.-P. Rivet and J. P. Boon, *Lattice Gas Hydrodynamics*. Cambridge University Press, 2001.
- [75] B. Chopard and M. Droz, *Cellular Automata Modeling of Physical Systems*. Cambridge University Press, 1998.

- [76] S. Wolfram, “Random sequence generation by cellular automata,” *Advances in Applied Mathematics*, vol. 7, no. 2, pp. 123–169, 1986.
- [77] B. Chopard, *Cellular Automata and Lattice Boltzmann Modeling of Physical Systems*. Springer, Berlin, Heidelberg, 2012.
- [78] V. Markova, “The lga models simulating sound wave propagation in a pipe,” *Bull. Nov. Comp. Center, Comp. Science*, vol. 25, pp. 25–36, 2006.
- [79] D. Cule, “Hpp lattice-gas automata for computational electromagnetics,” Master’s thesis, University of Manitoba, 1998.
- [80] V. Labarre, B. Dubrulle, and D. Paillard, “Sub-grid modelling for a diffusive lattice gas,” *Journal of Physics A: Mathematical and Theoretical*, vol. 53, p. 405006, sep 2020.
- [81] M. A. Mainster, “Cellular automata: Retinal cells, circulation and patterns,” *Eye*, vol. 6, pp. 420–427, 1992.
- [82] U. Frisch, B. Hasslacher, and Y. Pomeau, “Lattice-gas automata for the navier-stokes equation,” *Physical Review Letters*, vol. 56, no. 14, pp. 1505–1508, 1986.
- [83] J.-P. B. Alain Noullez, “Long-time correlations in a 2d lattice gas,” *Physica D: Nonlinear Phenomena*, vol. 47, no. 1–2, pp. 212–215, 1991.
- [84] P. Grosfils, J.-P. Boon, and P. Lallemand, “Spontaneous fluctuation correlations in thermal lattice-gas automata,” *Phys. Rev. Lett.*, vol. 68, pp. 1077–1080, Feb 1992.
- [85] A. Deutsch, J. M. Nava-Sedeño, S. Syga, and H. Hatzikirou, “Bio-lgca: A cellular automaton modelling class for analysing collective cell migration,” *PLOS Computational Biology*, vol. 17, pp. 1–22, 06 2021.
- [86] M. Fremling and J. Slingerland, “An investigation of pre-crystalline order, ruling out pauli crystals and introducing pauli anti-crystals,” *Scientific Reports*, vol. 10, p. 3710, 2020.
- [87] J. M. Buick, C. A. Greated, and W. J. Easson, “Investigation of a lattice gas model for surface gravity waves,” *Physics of Fluids*, vol. 9, no. 9, pp. 2585–2597, 1997.
- [88] S. Succi, *The Lattice Boltzmann Equation for Fluid Dynamics and Beyond (Numerical Mathematics and Scientific Computation)*, ch. 2, p. Section 2.3. Clarendon Press, 2001.
- [89] F. Hayot, “The effect of galilean non-invariance in lattice gas automaton one-dimensional flow,” *Complex Systems*, vol. 1, pp. 753–761, 1987.

- [90] D. Rothman, “Immiscible lattice gases: new results, new models,” vol. 46, pp. 232–238, 1989.
- [91] K. Molvig, P. Donis, R. Miller, J. Myczkowski, and G. Vichniac, “Multi-species lattice-gas automata for realistic fluid dynamics,” vol. 46, pp. 206–231, 1989.
- [92] S. Chen, H. Chen, and G. D. Doolen, “How the lattice gas model for the navier-stokes equation improves when a new speed is added,” *Complex Systems*, vol. 3, pp. 243–251, 1989.
- [93] A. Klales, D. Cianci, Z. Needell, D. A. Meyer, and P. J. Love, “Lattice gas simulations of dynamical geometry in two dimensions,” *Physical Review E*, vol. 82, p. 046705, 2010.
- [94] A. Koponen, *Simulations of fluid flow in porous media by lattice-gas and lattice-Boltzmann methods*. PhD thesis, University of Jyväskylä, 1998.
- [95] S. Chen, D. Martínez, W. Matthaeus, *et al.*, “Magnetohydrodynamics computations with lattice gas automata,” *Journal of Statistical Physics*, vol. 68, pp. 533–556, 1992.
- [96] M. D. Schuster, “Lattice gas cellular automata fluid dynamics: The model of frisch, hasslacher, and pomeau,” *Computing in Science & Engineering*, vol. 25, pp. 64–68, mar 2023.
- [97] Decker, Luc and Jeulin, Dominique, “Texture simulation by lattice gas from reaction-diffusion models,” *Microsc. Microanal. Microstruct.*, vol. 7, no. 5-6, pp. 565–571, 1996.
- [98] C. Kougias, G. Krause, and A. Rauh, “Simulation of river-discharge fronts with lattice gas automata,” *Discrete Models of Fluid Dynamics*, vol. 2, pp. 156–165, 1991.
- [99] M. da R. Ferreira, R. L. Cancian, A. H. da F. Klein, F. M. P. Tena, and B. Chopard, “Study of wind movement through the cellular automata and lattice boltzmann approach,” *Journal of Coastal Research*, pp. 1649–1653, 2006.
- [100] D. Kristanto and W. Paradhita, “Simulation study of fluid flow and estimation of a heterogeneous porous media properties using lattice gas automata method,” *Journal of Petroleum and Geothermal Technology*, vol. 1, no. 2, pp. 71–82, 2020.
- [101] L. Decker and D. Jeulin, “Texture simulation by lattice gas from reaction-diffusion models,” *Centre de Morphologie Mathématique, École des Mines de Paris*, vol. 7, pp. 565–571, 1996.

- [102] D. H. Rothman and S. Zaleski, “Lattice-gas models of phase separation: interfaces, phase transitions, and multiphase flow,” *Rev. Mod. Phys.*, vol. 66, p. 1417, 1994.
- [103] C. Borsani, G. Cattaneo, V. De Mattei, U. Jocher, and B. Zampini, *Backward Facing Step Validation of the FHP-III Lattice-Gas Model*. London: Springer London, 1998.
- [104] D. A. Meyer, “From quantum cellular automata to quantum lattice gases,” *Journal of Statistical Physics*, vol. 85, no. 5/6, pp. 551–574, 1996.
- [105] G. Grössing and A. Zeilinger, “Quantum cellular automata,” *Complex Systems*, vol. 2, pp. 197–208, 1988.
- [106] S. Fussy, G. Grössing, H. Schwabl, and A. Scrinzi, “Nonlocal computation in quantum cellular automata,” *Physical Review A*, vol. 48, pp. 3470–3477, 1993.
- [107] D. A. Meyer, “Quantum mechanics of lattice gas automata: One-particle plane waves and potentials,” *PHYSICAL REVIEW A*, vol. 55, no. 5, p. 551–574, 1997.
- [108] B. M. Boghosian and W. Taylor, “Simulating quantum mechanics on a quantum computer,” *Physica D: Nonlinear Phenomena*, vol. 120, no. 1-2, p. 30–42, 1998.
- [109] J. Yepez, “Quantum lattice-gas model for computational fluid dynamics,” *Phys. Rev. E*, vol. 63, p. 046702, Mar 2001.
- [110] J. Yepez, “Quantum lattice-gas model for the diffusion equation,” *International Journal of Modern Physics C*, vol. 12, no. 09, pp. 1285–1303, 2001.
- [111] J. Yepez, “Quantum lattice-gas model for the burgers equation,” *Journal of Statistical Physics*, vol. 107, pp. 203–224, 2002.
- [112] G. Vahala, J. Yepez, and L. Vahala, “Quantum lattice gas algorithm for quantum turbulence and vortex reconnection in the gross-pitaevskii equation,” in *Quantum Information and Computation VI*, vol. 6976, p. 69760U, SPIE Defense and Security Symposium, 2008.
- [113] G. Vahala, B. Zhang, J. Yepez, L. Vahala, and M. Soe, “Unitary qubit lattice gas representation of 2d and 3d quantum turbulence,” 2012.
- [114] J. S. Taylor, *Simulating Bose Einstein Condensates with Quantum Lattice Gas Algorithms*. PhD thesis, University of Hawai’i at Manoa, 2020.
- [115] T. Høfler, T. Häner, and M. Troyer, “Disentangling hype from practicality: On realistically achieving quantum advantage,” *Communications of the ACM*, vol. 66, no. 5, pp. 82–87, 2023.

- [116] F. E. Chrit, S. Kocherla, B. Gard, E. F. Dumitrescu, A. Alexeev, and S. H. Bryngelson, “Fully quantum algorithm for lattice boltzmann methods with application to partial differential equations,” 2023.
- [117] N. Fonio, P. Sagaut, and G. Di Molfetta, “A fully quantum algorithm for hydrodynamic lattice gas cellular automata,” 2023.
- [118] J. Kempe, “Quantum random walks: An introductory overview,” *Contemporary Physics*, vol. 44, no. 4, pp. 307–327, 2010.
- [119] A. Shakeel, D. A. Meyer, and P. J. Love, “History dependent quantum random walks as quantum lattice gas automata,” 2014.
- [120] P. Love, “On quantum extensions of hydrodynamic lattice gas automata,” *Condensed Matter*, vol. 4, no. 2, p. 48, 2019.
- [121] J. Yepez, “Type-ii quantum computers,” *International Journal of Modern Physics C*, vol. 12, no. 9, pp. 1273–1284, 2001.
- [122] D. Grinko, J. Gacon, C. Zoufal, *et al.*, “Iterative quantum amplitude estimation,” *npj Quantum Information*, vol. 7, 2021.
- [123] A. Kitaev, “Quantum measurements and the abelian stabilizer problem,” 1995.
- [124] X. Lu, N. Jiang, H. Hu, and Z. Ji, “Quantum adder for superposition states,” *International Journal of Theoretical Physics*, vol. 57, pp. 2575–2584, 2018.
- [125] D. Oliveira and R. Ramos, “Quantum bit string comparator: Circuits and applications,” *Quantum Computers and Computing*, vol. 7, 2007.
- [126] C. Körner, T. Pohl, U. Rüde, N. Thürey, and T. Zeiser, “Parallel lattice boltzmann methods for cfd applications,” pp. 439–466, 2006.
- [127] X. Zhang, J. Gomez-Paz, X. Chen, J. M. McDonough, M. M. Islam, Y. Andreopoulos, L. Zhu, and H. Yu, “Volumetric lattice boltzmann method for wall stresses of image-based pulsatile flows,” *Scientific Reports*, vol. 12, p. 1697, 2022.
- [128] M. Khan, N. Delbosc, C. Noakes, *et al.*, “Real-time flow simulation of indoor environments using lattice boltzmann method,” *Building Simulation*, vol. 8, pp. 405–414, 2015.
- [129] H. Faraji, M. Teggari, A. Arshad, M. Arıcı, E. Mehdi Berra, and K. Choukairy, “Lattice boltzmann simulation of natural convection heat transfer phenomenon for thermal management of multiple electronic components,” *Thermal Science and Engineering Progress*, vol. 45, p. 102126, 2023.

- [130] M. Babanezhad, A. Nakhjiri, A. Marjani, *et al.*, “Pattern recognition of the fluid flow in a 3d domain by combination of lattice boltzmann and anfis methods,” *Nature Scientific Reports*, vol. 10, p. 15908, 2020.
- [131] R. Khalili, E. Tavousi, R. Kazerooni, *et al.*, “Lattice boltzmann method simulation of nanofluid natural convection heat transfer in a square cavity with constant heat flux at walls,” *Iranian Journal of Science and Technology, Transactions of Mechanical Engineering*, 2024.
- [132] A. A. Mohamad, *Lattice Boltzmann Method: Fundamentals and Engineering Applications with Computer Codes*. Springer, 2019.
- [133] P. L. Bhatnagar, E. P. Gross, and M. Krook, “A model for collision processes in gases. i. small amplitude processes in charged and neutral one-component systems,” *Phys. Rev.*, vol. 94, pp. 511–525, May 1954.
- [134] J. Koelman, “A simple lattice boltzmann scheme for navier-stokes fluid flow,” *Europhysics Letters*, vol. 15, no. 6, pp. 603–607, 1991.
- [135] D. A. Wolf-Gladrow, *Lattice-Gas Cellular Automata and Lattice Boltzmann Models: An Introduction*, vol. 1725 of *Lecture Notes in Mathematics*. Springer Berlin Heidelberg, 2000.
- [136] Y. H. Qian, D. D’Humières, and P. Lallemand, “Lattice bgk models for navier-stokes equation,” *Europhysics Letters*, vol. 17, no. 6, p. 479, 1992.
- [137] B. Yan, J. Zhang, G. Yan, and T. Li, “A lattice boltzmann model for the eddy–stream equations in two-dimensional incompressible flows,” *Applied Mathematical Modelling*, vol. 35, no. 5, pp. 2358–2365, 2011.
- [138] B. N. Todorova and R. Steijl, “Quantum algorithm for the collisionless boltzmann equation,” *Journal of Computational Physics*, vol. 409, p. 109347, 2020.
- [139] W. Su, S. Lindsay, H. Liu, and L. Wu, “Comparative study of the discrete velocity and lattice boltzmann methods for rarefied gas flows through irregular channels,” *Phys. Rev. E*, vol. 96, p. 023309, Aug 2017.
- [140] F. Alouges, F. De Vuyst, G. Le Coq, and E. Lorin, “The reservoir technique: a way to make godunov-type schemes zero or very low diffuse. application to colella–glazsolver,” *European Journal of Mechanics B/Fluids*, vol. 27, no. 6, pp. 643–664, 2008.
- [141] S. Yamazaki, F. Uchida, K. Fujisawa, and N. Yoshida, “Quantum algorithm for collisionless boltzmann simulation of self-gravitating systems,” 2023.
- [142] L. Budinski, “Quantum algorithm for the advection–diffusion equation simulated with the lattice boltzmann method,” *Quantum Information Processing*, vol. 20, no. 57, 2021.

- [143] L. Budinski, “Quantum algorithm for the navier–stokes equations by using the streamfunction-vorticity formulation and the lattice boltzmann method,” *International Journal of Quantum Information*, vol. 20, no. 2, p. 2150039, 2022.
- [144] A. Salih, “Streamfunction-vorticity formulation,” tech. rep., Department of Aerospace Engineering, Indian Institute of Space Science and Technology, 2013.
- [145] G. H. Low and I. L. Chuang, “Hamiltonian simulation by qubitization,” *Quantum*, vol. 3, p. 163, 2019.
- [146] J. G. Zhou, “Macroscopic lattice boltzmann method,” *Water*, vol. 13, no. 1, 2021.
- [147] D. W. Berry, A. M. Childs, A. Ostrander, and G. Wang, “Quantum algorithm for linear differential equations with exponentially improved dependence on precision,” *Communications in Mathematical Physics*, vol. 356, pp. 1057–1081, 2017.
- [148] M. A. Schalkers and M. Möller, “Efficient and fail-safe collisionless quantum boltzmann method,” 2022.
- [149] T. G. Draper, “Addition on a quantum computer,” 2000.
- [150] M. A. Schalkers and M. Möller, “On the importance of data encoding in quantum boltzmann methods,” 2023.
- [151] T. Carleman, “Application of the theory of linear integration equations to nonlinear systems of differential equations,” *Acta Mathematica*, vol. 59, p. 63, 1932.
- [152] W. Itani and S. Succi, “Analysis of carleman linearization of lattice boltzmann,” *Fluids*, vol. 7, p. 24, 2022.
- [153] X. Li, X. Yin, N. Wiebe, J. Chun, G. K. Schenter, M. S. Cheung, and J. Mülmenstädt, “Potential quantum advantage for simulation of fluid dynamics,” 2023.
- [154] B. Bakker and T. Watts, “Quantum carleman linearization of the lattice boltzmann equation with boundary conditions,” 2023.
- [155] C. Sanavio and S. Succi, “Quantum lattice boltzmann-carleman algorithm,” 2023.
- [156] B. M. DeBlois, “Linearizing convection terms in the navier-stokes equations,” *Computer Methods in Applied Mechanics and Engineering*, vol. 143, no. 3, pp. 289–297, 1997.

- [157] M. Mohammadi, S. Vakilipour, and S. Ormiston, “Newton linearization of the navier–stokes equations for flow computations using a fully coupled finite volume method,” *Applied Mathematics and Computation*, vol. 397, p. 125916, 2021.
- [158] Z. Meng and Y. Yang, “Quantum computing of fluid dynamics using the hydrodynamic schrödinger equation,” *Phys. Rev. Research*, vol. 5, p. 033182, 2023.
- [159] L. Mouton, F. Reiter, Y. Chen, and P. Rebentrost, “Deep learning-based quantum algorithms for solving nonlinear partial differential equations,” 2023.
- [160] A. Gnanasekaran, A. Surana, and T. Sahai, “Efficient quantum algorithms for nonlinear stochastic dynamical systems,” 2023.
- [161] F. Flandoli, “Stochastic differential equations in fluid dynamics,” *Seminario Mat. e. Fis. di Milano*, vol. 66, pp. 121–148, 1996.
- [162] A. Luis and J. Peřina, “Optimum phase-shift estimation and the quantum description of the phase difference,” *Phys. Rev. A*, vol. 54, pp. 4564–4570, Nov 1996.
- [163] D. W. Berry, G. Ahokas, R. Cleve, *et al.*, “Efficient quantum algorithms for simulating sparse hamiltonians,” *Communications in Mathematical Physics*, vol. 270, pp. 359–371, mar 2007.
- [164] L. Lapworth, “A hybrid quantum-classical cfd methodology with benchmark hhl solutions,” 2022.
- [165] L. Lapworth, “Implicit hybrid quantum-classical cfd calculations using the hhl algorithm,” 2022.
- [166] B. D. Clader, B. C. Jacobs, and C. R. Sprouse, “Preconditioned quantum linear system algorithm,” *Phys. Rev. Lett.*, vol. 110, p. 250504, 2013.
- [167] M. Benzi, “Preconditioning techniques for large linear systems: A survey,” *Journal of Computational Physics*, vol. 182, no. 2, pp. 418–477, 2002.
- [168] L. Wossnig, Z. Zhao, and A. Prakash, “Quantum linear system algorithm for dense matrices,” *Phys. Rev. Lett.*, vol. 120, p. 050502, 2018.
- [169] Y. Saito, X. Lee, D. Cai, and N. Asai, “An iterative improvement method for hhl algorithm for solving linear system of equations,” 2021.
- [170] Y. Lee, J. Joo, and S. Lee, “Hybrid quantum linear equation algorithm and its experimental test on ibm quantum experience,” *Scientific Reports*, vol. 9, p. 4778, 2019.

- [171] F. Gao, G. Wu, M. Yang, *et al.*, “A hybrid algorithm to solve linear systems of equations with limited qubit resources,” *Quantum Information Processing*, vol. 21, p. 111, 2022.
- [172] B. Daribayev, A. Mukhanbet, and T. Imankulov, “Implementation of the hhl algorithm for solving the poisson equation on quantum simulators,” *Applied Sciences*, vol. 13, no. 20, 2023.
- [173] J. C. J. X. I. A. L. L. X. K. D. L. E. S. G. L. Tao Xin, Shijie Wei, “A quantum algorithm for solving linear differential equations: Theory and experiment,” *Phys. Rev. A*, vol. 101, p. 032307, 2020.
- [174] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, and P. J. Coles, “Variational quantum linear solver,” *Quantum*, vol. 7, p. 1188, 2023.
- [175] I. Griol-Barres, S. Milla, A. Cebrián, Y. Mansoori, and J. Millet, “Variational quantum circuits for machine learning. an application for the detection of weak signals,” *Applied Sciences*, vol. 11, no. 14, 2021.
- [176] J. Wu, H. Fu, M. Zhu, H. Zhang, W. Xie, and X.-Y. Li, “Quantum circuit autoencoder,” 2023.
- [177] D. An and L. Lin, “Quantum linear system solver based on time-optimal adiabatic quantum computing and quantum approximate optimization algorithm,” *ACM Transactions on Quantum Computing*, vol. 3, mar 2022.
- [178] X. Xu, J. Sun, S. Endo, Y. Li, S. C. Benjamin, and X. Yuan, “Variational algorithms for linear algebra,” *Science Bulletin*, vol. 66, no. 21, pp. 2181–2188, 2021.
- [179] H.-Y. Huang, K. Bharti, and P. Rebentrost, “Near-term quantum algorithms for linear systems of equations,” 2019.
- [180] Y. Liu, Z. Chen, C. Shu, P. Rebentrost, Y. Liu, S. C. Chew, B. C. Khoo, and Y. D. Cui, “A variational quantum algorithm-based numerical method for solving potential and stokes flows,” 2023.
- [181] C. Shu and B. E. Richards, “Application of generalized differential quadrature to solve two-dimensional incompressible navier-stokes equations,” *International Journal for Numerical Methods in Fluids*, vol. 15, no. 7, pp. 791–798, 1992.
- [182] Y. Y. Liu, Z. Chen, C. Shu, S. C. Chew, B. C. Khoo, X. Zhao, and Y. D. Cui, “Application of a variational hybrid quantum-classical algorithm to heat conduction equation and analysis of time complexity,” *Physics of Fluids*, vol. 34, no. 11, p. 117121, 2022.
- [183] E. Cappanera, “Variational quantum linear solver for finite element problems: a poisson equation test case,” Master’s thesis, Delft University of Technology, 2021.

- [184] F. Y. Leong, W.-B. Ewe, and D. E. Koh, “Variational quantum evolution equation solver,” *Scientific Reports*, vol. 12, no. 10817, 2022.
- [185] M. Lubasch, J. Joo, P. Moinier, M. Kiffner, and D. Jaksch, “Variational quantum algorithms for nonlinear problems,” *Phys. Rev. A*, vol. 101, p. 010301, 2020.
- [186] A. Sarma, T. W. Watts, M. Moosa, Y. Liu, and P. L. McMahon, “Quantum variational solving of nonlinear and multi-dimensional partial differential equations,” 2023.
- [187] O. Kyriienko, A. E. Paine, and V. E. Elfving, “Solving nonlinear differential equations with differentiable quantum circuits,” *Phys. Rev. A*, vol. 103, p. 052416, 2021.
- [188] Z. Chen, C. Xue, S. Chen, *et al.*, “Quantum approach to accelerate finite volume method on steady computational fluid dynamics problems,” *Quantum Information Processing*, vol. 21, p. 137, 2022.
- [189] K. Phalak, A. Chatterjee, and S. Ghosh, “Quantum random access memory for dummies,” *Sensors (Basel)*, vol. 23, no. 17, p. 7462, 2023.
- [190] S. K. Leyton and T. J. Osborne, “A quantum algorithm to solve nonlinear differential equations,” 2008.
- [191] J.-P. Liu, H. Kolden, H. K. Krovi, and A. M. Childs, “Efficient quantum algorithm for dissipative nonlinear differential equations,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 35, p. e2026805118, 2021.
- [192] D. An, D. Fang, S. Jordan, J.-P. Liu, G. H. Low, and J. Wang, “Efficient quantum algorithm for nonlinear reaction-diffusion equations and energy estimation,” 2023.
- [193] H. Krovi, “Improved quantum algorithms for linear and nonlinear differential equations,” *Quantum*, vol. 7, p. 913, 2023.
- [194] P. C. S. Costa, P. Schleich, M. E. S. Morales, and D. W. Berry, “Further improving quantum algorithms for nonlinear differential equations via higher-order methods and rescaling,” 2023.
- [195] S. Lloyd, G. De Palma, C. Gokler, B. Kiani, Z.-W. Liu, M. Marvian, F. Tennie, and T. Palmer, “Quantum algorithm for nonlinear differential equations,” 2020.
- [196] N. Karjanto, “The nonlinear schrödinger equation: A mathematical model with its wide-ranging applications,” 2019.
- [197] K. Georgopoulos, C. Emary, and P. Zuliani, “Comparison of quantum-walk implementations on noisy intermediate-scale quantum computers,” *Phys. Rev. A*, vol. 103, p. 022408, Feb 2021.

- [198] A. Shakeel, “Efficient and scalable quantum walk algorithms via the quantum fourier transform,” *Quantum Information Processing*, vol. 19, p. 323, 2020.
- [199] L. Budinski, O. Niemimäki, R. Zamora-Zamora, and V. Lahtinen, “Efficient parallelization of quantum basis state shift,” *Quantum Science and Technology*, vol. 8, no. 4, p. 045031, 2023.
- [200] P. K. Kundu, I. M. Cohen, and D. R. Dowling, *Fluid Mechanics*. Academic Press, 2011.
- [201] J. Vovrosh, K. E. Khosla, S. Greenaway, C. Self, M. S. Kim, and J. Knolle, “Simple mitigation of global depolarizing errors in quantum simulations,” *Physical Review E*, vol. 104, p. 035309, 2021.
- [202] L. P. Kadanoff, G. R. McNamara, and G. Zanetti, “A poiseuille viscometer for lattice gas automata,” *Complex Systems*, vol. 1, no. 4, pp. 791–803, 1987.
- [203] N. Metropolis and S. Ulam, “The monte carlo method,” *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949.
- [204] Airbus and B. Group, “Quantum computing challenge: Quantum solutions for optimization in logistics and vehicle routing,” 2023.
- [205] “Openfoam.”
- [206] “Turbulent flow over naca0012 airfoil (2d).” <https://www.openfoam.com/documentation/guides/latest/doc/verification-validation-naca0012-airfoil-2d.html>. Accessed: yyyy-mm-dd.
- [207] K. Mattila, *Implementation techniques for the lattice Boltzmann method*. PhD thesis, University of Jyväskylä, 2010.
- [208] P. J. Dellar, “An interpretation and derivation of the lattice boltzmann method using strang splitting,” *Computers Mathematics with Applications*, vol. 65, no. 2, pp. 129–141, 2013. Special Issue on Mesoscopic Methods in Engineering and Science (ICMMES-2010, Edmonton, Canada).
- [209] T. Inamuro, M. Yoshino, and K. Suzuki, *An Introduction to the Lattice Boltzmann Method: A Numerical Method for Complex Boundary and Moving Boundary Flows*. World Scientific, 2021.
- [210] B. E. Rapp, *Microfluidics: Modelling, Mechanics and Mathematics*, ch. 9.9.2 Knudsen Number, pp. 257–259. William Andrew, 2016.
- [211] V. Vougalter and V. Volpert, “Solvability relations for some diffusion equations with convection terms,” *The interdisciplinary journal of Discontinuity Nonlinearity and Complexity*, vol. 3, no. 4, pp. 457–465, 2014.

- [212] R. Herman, *Introduction to Partial Differential Equations*, ch. Appendix-The Fredholm Alternative Theorem, p. 145. Open Education Resource (OER) LibreTexts Project, 2023.
- [213] F. Bösch, *Entropic Lattice Boltzmann Models for Fluid Dynamics*. PhD thesis, ETH Zurich, 2017.
- [214] D. d. Irina Ginzburg, Frederik Verhaeghe, “Two-relaxation-time lattice boltzmann scheme: About parametrization, velocity, pressure and mixed boundary conditions,” *Communications in Computational Physics*, vol. 3, no. 2, pp. 427–478, 2008.
- [215] J. Latt and B. Chopard, “Lattice boltzmann method with regularized pre-collision distribution functions,” *Mathematics and Computers in Simulation*, vol. 72, no. 2-6, pp. 165–168, 2006.
- [216] M. K. P. L. L.-S. L. Dominique d’Humières, Irina Ginzburg, “Multiple-relaxation-time lattice boltzmann models in three dimensions,” *Philosophical Transactions of the Royal Society of London A: Mathematical, Physical and Engineering Sciences*, vol. 360, no. 1792, pp. 437–451, 2002.
- [217] M. Haussmann, *Lattice Boltzmann Methods for Turbulent Flows: Application to Coriolis Mass Flowmeter*. PhD thesis, Karlsruher Institut für Technologie (KIT), 2020.
- [218] P. Nathen, *Towards an Improved Lattice-Boltzmann Methodology for the Simulation of Turbulent Flows*. PhD thesis, Technische Universität München, Lehrstuhl für Aerodynamik und Strömungsmechanik, 2019.
- [219] M. Gehrke and T. Rung, “Scale-resolving turbulent channel flow simulations using a dynamic cumulant lattice boltzmann method,” *Physics of Fluids*, vol. 34, no. 7, p. 075129, 2022.
- [220] P. Sagaut, “Subgrid modelling for lbm-based large-eddy simulation,” in *CFM 2009 - 19ème Congrès Français de Mécanique*, (Marseille, France), 2009.
- [221] R. Benzi, S. Succi, and M. Vergassola, “The lattice boltzmann equation: Theory and applications,” *SpringerLink*, pp. 187–203, 1992.

A From LGA to LBM using a multiscale expansion

In performing a simulation, our goal is to model the temporal progression of specific variables. This involves determining the evolution of an initial state based on certain differential equations, which are often not analytically solvable. The procedure consists of several assumptions to obtain a discrete equation which can be computationally simulated. Usually, an error analysis follows due to space-time discretization. This is the case for the finite differences method, where differential equations are discretized using Taylor expansions on space and time. One of the significant problems of LGA is that reproducing a specific differential equation is generally challenging as the link between the collision and propagation operators and the final macroscopic variables evolution is not straightforward. In this chapter we will see how to connect both interpretations. The study of these methodologies originated an exact transport equation for LGA at a mesoscopic level¹⁶ called lattice Boltzmann equation (LBE). We will use [74, 135] as main references for writing this subsection.

Using probability distribution functions, the LBE defines ensemble-averaged (over space, time or realizations) quantities per channel and their time evolution. It is essential to highlight that other statistical descriptions are possible. Specifically, the Liouville equation, describes the probability of the different configurations in the phase space. A homogeneous equilibrium distribution function can also be obtained following the variational principle using the functional H from the H -theorem. This means that following constraints related to the conservation laws, the maximization of H yields an equilibrium distribution for the ensemble-average boolean quantities of LGA. However, in this appendix, we will focus on the probability distribution functions of the specific channels of every lattice site (Boltzmann description). The general microscopic evolution equation for LGA can be written as

$$n_i(r + c_i, t + \Delta t) = \sum_{s': (n, s') \in \gamma^2} s'_i(r, t) \xi(n(r, t) \rightarrow s'(r, t)) \quad (73)$$

where we sum over all possible local states s' that may be outcome from the local collision of n expressed by ξ . If we average this equation over an ensemble, we obtain

$$f_i(r + c_i, t + \Delta t) = \left\langle \sum_{s': (n, s') \in \gamma^2} s'_i(r, t) \xi(n(r, t) \rightarrow s'(r, t)) \right\rangle \quad (74)$$

where f_i is the probability distribution function, product of the average ensemble of the boolean equation. In other words, f_i expresses the probability of finding the

¹⁶Physical systems can be described at different levels of abstraction. A detailed particle interaction is described at the microscopic level, while for the macroscopic level, the description is general and continuum (like Navier-Stokes equations). The intermediate compromised between these two is called mesoscopic (statistical mechanics) [207]

state n_i in a given occupation level at a particular time and lattice site. In order to simplify the previous equation, we need to use the Boltzmann decorrelation hypothesis. This is because particles entering a collision may be correlated, which means that they share common information produced by the successive collision and propagation, which sets the rules of microscopic evolution. The Boltzmann hypothesis supposes that propagation suppresses all possible correlations between particles, which allows us to write the above equation as

$$f_i(r + c_i, t + \Delta t) = \sum_{s': (n, s') \in \gamma^2} \langle s'_i(r, t) \rangle \mathbf{A}(n(r, t) \rightarrow s'(r, t)) \quad (75)$$

with $\mathbf{A}$ the probability of going from a state n to state s' . The equation can be rewritten as

$$f_i(r + c_i, t + \Delta t) - f_i(r, t) = \sum_{s': (n, s') \in \gamma^2} \langle (s'_i(r, t) - n_i(r, t)) \rangle \mathbf{A}(\mathbf{n}(\mathbf{r}, \mathbf{t}) \rightarrow \mathbf{s}'(\mathbf{r}, \mathbf{t})) \quad (76)$$

or same wise

$$f_i(r + c_i, t + \Delta t) - f_i(r, t) = \langle \Omega \rangle \quad (77)$$

with

$$\langle \Omega \rangle = \sum_{s': (n, s') \in \gamma^2} \langle (s'_i(r, t) - n_i(r, t)) \rangle \mathbf{A}(n(r, t) \rightarrow s'(r, t)) \quad (78)$$

This is the same expression we mentioned in (19) for LGA macroscopics. Using the H-theorem and the variational principle, it can be proved that the global equilibrium follows a Fermi-Dirac probability distribution. As it is homogeneous, it doesn't depend on space and time. It is essential to notice that it is possible to write f in terms of the macroscopic variables using the symmetries of the lattice and collisional invariances as presented in Rivet's book [74]. The connection between microscopic and macroscopic variables can be made using several methods such as Chapman-Enskog expansion, the projectors approach [74], strang splitting [208] and the S-expansion [209]. Regarding the method, there is an unavoidable step: the multiple-scale hypothesis, which consists of the separation of microscopic dynamics and macroscopic phenomena of different scales. For instance, the difference between the Chapman-Enskog expansion (a method that we will briefly describe) and the S-expansion, is the latter applies a multiscale expansion not only to the probability distribution function but also to the macroscopic variables such as mass, momentum and energy densities. First, to use a similar notation as in Rivet's work, we will rewrite (76) as

$$f_i(r + c_i, t + \Delta t) - f_i(r, t) = \sum_{S \in \Gamma : n_i(r, t) \in S} \mathcal{P}(S) \times \sum_{s': (n, s') \in \gamma^2} (s'_i(r, t) - n_i(r, t)) \mathbf{A}(n(r, t) \rightarrow s'(r, t)) \quad (79)$$

with Γ the phasic space and $\mathcal{P}$ the probability of having the phase state S with $n_i(r, t)$ the occupation of the channel i at space r and time t . Therefore, we can

rewrite it as

$$f_i(r + c_i, t + \Delta t) - f_i(r, t) = \sum_{S \in \Gamma : n_i(r, t) \in S} \mathcal{P}(S) L_{n_i}(r) \quad (80)$$

with L_{n_i}

$$L_{n_i}(r) = \sum_{s' : (n, s') \in \gamma^2} (s'_i(r, t) - n_i(r, t)) \mathbf{A}(n(r, t) \rightarrow s'(r, t)) \quad (81)$$

Now, we can introduce the concept of the scale separation parameter. We will use ϵ as the ratio between the spatial microscopic scale and the smallest macroscopic scale, which is related to the Knudsen number for fluid dynamics [210]. This means we assume a description of the channel's probability distribution at different scales. We can expand $f(r, t)$

$$f_i = f_i^{(0)} + \epsilon f_i^{(1)} + \epsilon^2 f_i^{(2)} + O(\epsilon^3) \quad (82)$$

We will also assume that the flow is weakly inhomogeneous and, therefore, the local equilibrium distribution function $f_i^{local} = f_i^{(0)}$, as $f_i = f_i^{local}$ when the scale separation vanishes, and the macroscopic field is at equilibrium ($\epsilon = 0$). A significant result is that given that the local equilibrium distribution defines the macroscopic variables and these must be conserved, higher orders of the equilibrium distribution function $f_i^{(0)}$ do not contribute to them. This means that

$$\begin{aligned} \sum_i q_i f_i^{(1)}(r, t) &= 0 \\ \sum_i q_i f_i^{(2)}(r, t) &= 0 \end{aligned} \quad (83)$$

with q_i a macroscopic variable invariant under the collision operator and

$$\begin{aligned} \sum_i q_i f_i^{(0)}(r, t) &= \rho \\ \sum_i q_i c_i f_i^{(0)}(r, t) &= j \end{aligned} \quad (84)$$

being ρ and j the mass and momentum density.

To acknowledge the noticeable space variations of the macroscopic variables, it is used $r = \epsilon^{-1} r_1$, using ϵ^{-1} as the new space scale. Given that Navier-Stokes equations have two different generic kinds of macroscopic phenomena, we also expect two different scales for time. Specifically, non-linear and pressure contributions involve first-order space derivatives, while linear diffusion effects such as viscosity involve second-order space derivatives. This translates into damping over different

time scales. Therefore we will also define $t = \epsilon^{-1}t_1 + \epsilon^{-2}t_2$, using in the future

$$\begin{aligned}\frac{\partial}{\partial r} &= \epsilon \frac{\partial}{\partial r_1} \\ \frac{\partial}{\partial t} &= \epsilon \frac{\partial}{\partial t_1} + \epsilon^2 \frac{\partial}{\partial t_2}\end{aligned}\tag{85}$$

Solving (80) by making use of the Chapman-Enskog expansion requires of using the properties of collision invariants Q . They are observables whose value measured at any node is left unchanged by the collision phase. Mathematically, we can express it as

$$\forall (s, s') \in \gamma^2, \quad \sum_{i=1}^b Q_i(s'_i - s_i) \mathbf{A}(\mathbf{s} \rightarrow \mathbf{s}') = 0 \tag{86}$$

with Q a specific variable or property associated with the channels, such as mass or velocity. These conserved quantities constrain the collision term L_{n_i} . As explained in Lattice Gas Hydrodynamics, the important consequence is that the conservation laws in a lattice gas model have a crucial incidence on its ability to exhibit global statistical equilibria, and thus macroscopic collective motions. Using (86), we can deduce that the set of collisional invariances form a subspace $\mathbb{R}^b$. In this basis, the number of linearly independent vectors is $\delta \in [1, b]$, with b the number of channels. To sum up, any collisional invariant will be generated according to

$$Q_i(f_i) = \sum_{\kappa=1}^b \lambda^{[\kappa]} q_i^{[\kappa]} \tag{87}$$

with $\lambda^{[\kappa]}$ the scalar associated with the linear combination of the collisional invariance basis $q^{[\kappa]}$. Using equations (41) and (86) and considering the elements of the collisional invariant basis are invariants, we can deduce that

$$\sum_{i=1}^b q_i^{[\kappa]} L_{n_i}(r) = 0 \quad \forall \kappa = 1, \dots, \delta \quad \forall S : n_i \in S \quad \forall r \in L \tag{88}$$

This property is important as it will allow us to use Fredholm's criterion of solvability later on. More information about it and its use cases can be found in [211]. To solve (80), we need to expand the first term with a Taylor expansion and relate the equilibrium distribution functions with their order according to Chapman-Enskog expansion. This means that the first order must have terms $f^{(1)}$, second order $f^{(2)}$ and so on. As a result, we obtain the first order

$$\sum_{S \in \Gamma} L_{n_i} \mathcal{P}(f_i^{(1)}, S) = \partial_{t_1} f_i^{(0)} + \partial_{1,\beta} c_{i,\beta} f_i^{(0)} \tag{89}$$

and second order

$$\begin{aligned}\sum_{S \in \Gamma} L_{n_i} \mathcal{P}(f_i^{(2)}, S) &= \partial_{t_1} f_i^{(1)} + \partial_{1,\beta} c_{i,\beta} f_i^{(1)} + \partial_{t_1} f_i^{(0)} + \\ &\quad \frac{1}{2} \partial_{1\beta} \partial_{1\gamma} c_{i\beta} c_{i\gamma} f_i^{(0)} + \frac{1}{2} \partial_{t_1}^2 f_i^{(0)} + \partial_{1\beta} \partial_{t_1} c_{i\beta} f_i^{(0)}\end{aligned}\tag{90}$$

equations. Notice that the first order can be calculated using the zero-order distribution function, and the second order is obtained from the first. This allows us to solve every scale by an iterative algorithm. Now, using (86), (89) and (90), we can make use of the first Fredholm alternative theorem [212] that reads as follows

Theorem 3. *Let $\mathbf{A}$ be a $n \times n$ matrix and $b, x \in \mathbb{R}^n$, then the equation $\mathbf{A}x = b$ has a solution if and only if $b^T v = 0 \quad \forall v \in \mathbb{R}^n : \mathbf{A}^T v = 0$*

This means that the solvability conditions are

$$\partial_{t_1} \sum_{i=1}^b q_i^{[\kappa]} f_i^{(0)} + \partial_{i\beta} \sum_{i=1}^b b q_i^{[\kappa]} c_{i\beta} f_i^{(0)} = 0 \quad \kappa = 1, \dots, \delta \quad (91)$$

for the first-order equation and

$$\begin{aligned} 0 = \partial_{t_1} \sum_{i=1}^b q_i^{[\kappa]} f_i^{(1)} + \partial_{1\beta} \sum_{i=1}^b q_i^{[\kappa]} f_i^{(0)} + \frac{1}{2} \partial_{1\beta} \partial_{1\gamma} \sum_{i=1}^b q_i^{[\kappa]} c_{i\beta} c_{i\gamma} f_i^{(0)} \\ + \frac{1}{2} \partial_{t_1}^2 \sum_{i=1}^b q_i^{[\kappa]} f_i^{(0)} + \partial_{1\beta} \partial_{t_1} \sum_{i=1}^b q_i^{[\kappa]} c_{i\beta} f_i^{(0)} \end{aligned} \quad (92)$$

the second-order equation.

To solve these equations, we need to write the equilibrium distribution functions in terms of the macroscopic variables (mass, momentum and energy density) and use them to reformulate the gradients. A fundamental assumption in this respect is that the model has crystallographic isotropy up to the fourth order. This isotropic property is also used in classical Navier-Stokes equations derivation for non-turbulent flows in Newtonian fluids. As pointed out in Fluid Dynamics by Pijush K. Kundu and Ira M. Cohen [200], turbulent flows at all scales must be anisotropic for small scale eddies to extract energy from the mean field. The derivation of the final equations is out of the scope of this thesis, but it can be found in J.-P. Rivet's book (Lattice Gas Hydrodynamics). We obtain the mass, momentum and energy density equations for the first-order time scales

$$\partial_{t_1} \rho + \partial_{1\beta} (\rho u_\beta) = 0 \quad (93)$$

$$\partial_{t_1} j_\alpha + \partial_{1\beta} (g(\rho) j_\alpha u_\beta) + \partial_\alpha P(\rho, j^2, w) = 0 \quad (94)$$

$$\partial_{t_1} w + \partial_{1\beta} (g'(\rho) w u_\beta) + \partial_{1\beta} \left(\frac{2\xi_4}{D\xi_2} j_\beta \right) = 0 \quad (95)$$

It is crucial to notice that these equations are only approximations for weakly homogeneous equilibriums where $f^{(0)}$ is constant. As we will later see, this is why the initial conditions for simulating LBM must be the local equilibrium distribution functions. As the Chapman-Enskog expansion is done around a nearly equally distributed probability for each channel and node, we assume the fluid

velocity is small. This and the isotropic assumption make LBM unsuitable for turbulent flow simulation at all scales. Nonetheless, turbulent flow simulations can be solved using LBM with different collision rules such as Bhatnagar–Gross–Krook (BKG), Entropic Lattice Boltzmann (ELB) [213], Two-Relaxation-Time (TRT) [214], Regularized Lattice Boltzmann (RLB) [215] and Multiple-Relaxation-Time (MRT) [216]. These approaches are compared and analyzed for turbulent flows in two recent PhD thesis [217, 218]. Other methods based on sublattice grids to resolve more scales have also been explored [219, 220]. Continuing our derivation, for the second order, we obtain the following solvability conditions

$$\partial_{t_2}\rho = 0 \quad (96)$$

$$\partial_{t_2}j_\alpha = \partial_{1\beta} \left(\nu \left(\partial_{1\alpha}j_\beta + \partial_{1\beta}j_\alpha - \frac{2}{D}\partial_{1\gamma}j_\gamma\delta_{\alpha\beta} \right) \right) \quad (97)$$

$$\partial_{t_1}w = \partial_{1\beta}(\zeta\partial_{1\beta}w) \quad (98)$$

Using first and second-order conditions, we obtain

$$\partial_t\rho + \partial_\beta(\rho u_\beta) = 0 \quad (99)$$

$$\partial_tj_\alpha + \partial_\beta(g(\rho)j_\alpha u_\beta) + \partial_\alpha P = \partial_{1\beta} \left(\nu \left(\partial_{1\alpha}j_\beta + \partial_{1\beta}j_\alpha - \frac{2}{D}\partial_{1\gamma}j_\gamma\delta_{\alpha\beta} \right) \right) \quad (100)$$

$$\partial_tw + \partial_\beta(g'(\rho)wu_\beta) + \partial_\beta \left(\frac{2\xi_4}{D\xi_2}j_\beta \right) = \partial_\beta(\zeta\partial_\beta w) \quad (101)$$

where we have recovered the time and space derivatives instead of the variables associated with different time and space scales. In the previous equation, ζ is the thermal conductivity coefficient, and ν is the shear viscosity coefficient. Additionally, various ξ factors appear, related to the geometry of the lattice and g' non-galilean factor depending on the mass density. Now, considering the limit of small Mach number (low velocity) and grouping the terms according to [221], we obtain the continuity and Navier-Stokes equations for incompressible flow

$$\nabla u = 0 \quad (102)$$

$$\partial_tJ_\alpha + \partial_\beta P_{\alpha\beta} = \partial_\beta S_{\alpha\beta} \quad (103)$$

Finally, in the derivation, the Boltzmann approximation is used to compute the unknown transport coefficients implicit in P and S terms and, with them, the first and second-order distribution functions. As these final derivations and expressions are long and cumbersome and our goal was to introduce the connection between LGA and LBM, we refer the reader to read more details on Lattice Gas Hydrodynamics.

B Classical LBM example using D1Q3 diffusion equation

The diffusion equation is a partial differential equation (PDE) that can be written as

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (104)$$

In order to obtain its time evolution, we can use the LBE (41). To use it, we need to derive the equilibrium distribution function to simulate the diffusion equation. In order to do this, we need an ansatz, which is an initial educated guess of the expression of the distribution function. For example for the diffusion equation we will assume them to be constant as

$$f_i^{eq} = A_i \quad (105)$$

and imposing the conservation of mass and momentum, we obtain

$$\begin{aligned} \sum_{i=1}^3 f_i^{eq} &= 1 \\ \sum_{i=1}^3 c_i f_i^{eq} &= 0 \\ \sum_{i=1}^3 c_i^2 f_i^{eq} &= c_s^2 \end{aligned} \quad (106)$$

where $c_s = \frac{1}{3}$ for lattices. Now, we derive the weights of the specific lattice that we will use. In this case we are using 1D lattice site with three channels. Relating f_i^{eq} with W_i (42), we can derive the weights using previous equations (106).

$W_0 = \frac{2}{3}$ for $c_0 = 0$ and $W_{1,2} = \frac{1}{6}$ for $c_1 = 1$ and $c_2 = -1$. Additionally, we have that

$$T = \sum_{i=1}^3 f_i^{eq} \quad (107)$$

and therefore

$$f_i^{eq} = W_i T_i \quad (108)$$

To ensure we will obtain the desire solution, we need to begin with a weekly inhomogenous initial condition. Therefore we will use the weights for $x = 0$ and the channels for the rest of the lattice (adding 0 as net result).

$x = -1$	$x = 0$	$x = 1$
$f_0 = 0$	$f_0 = 4/6$	$f_0 = 0$
$f_1 = 1$	$f_1 = 1/6$	$f_1 = -1$
$f_2 = -1$	$f_2 = 1/6$	$f_2 = 1$

Table 6: Initial conditions for the probability distribution of the diffusion D1Q3 LBM at time $t = 0$

and the local equilibrium distribution as

$x = -1$	$x = 0$	$x = 1$
$f_0^{eq} = 0$	$f_0^{eq} = 4/6$	$f_0^{eq} = 0$
$f_1^{eq} = 0$	$f_1^{eq} = 1/6$	$f_1^{eq} = 0$
$f_2^{eq} = 0$	$f_2^{eq} = 1/6$	$f_2^{eq} = 0$

Table 7: Initial conditions for the local equilibrium distribution functions of the diffusion D1Q3 LBM at time $t = 0$

Utilizing (41), we obtain the probability distributions after the collision. The relation between ω and the parameter α to set the diffusion rate can be obtained from a multiscale expansion such as Chapman-Kolmogorov. Following [132], we know that

$$\alpha = \frac{\Delta x^2}{\delta t D} \left(\frac{1}{\omega} - \frac{1}{2} \right) \quad (109)$$

with D the dimension of the lattice. In this case, setting $\Delta x = \Delta t = 1$

$$\alpha = \left(\frac{1}{\omega} - \frac{1}{2} \right) \quad (110)$$

Setting $\alpha = 0.25$, we get $\omega = 4/3$ and therefore after the collision we obtain

$x = -1$	$x = 0$	$x = 1$
$\widetilde{f}_0 = 0$	$\widetilde{f}_0 = 4/6$	$\widetilde{f}_0 = 0$
$\widetilde{f}_1 = -1/3$	$\widetilde{f}_1 = 1/6$	$\widetilde{f}_1 = 1/3$
$\widetilde{f}_2 = 1/3$	$\widetilde{f}_2 = 1/6$	$\widetilde{f}_2 = -1/3$

Table 8: Local distribution functions after the collision step

Again, from (41), we know that

$$f_i(x + c_i \Delta t, t + \Delta t) = \widetilde{f_i(x, t)} \quad (111)$$

Therefore, after the streaming operation, we get

$x = -1$	$x = 0$	$x = 1$
$f_0 = 0$	$f_0 = 4/6$	$f_0 = 0$
$f_1 = 1/3$	$f_1 = -1/3$	$f_1 = 1/6$
$f_2 = 1/6$	$f_2 = -1/3$	$f_2 = 1/3$

Table 9: Local distribution functions after the collision and streaming at time at time $t = 1$

Notice that if we set boundary conditions, the streaming would be modified and local distributions near the walls would not stream in the direction of the propagation of the channel. Finally, we can calculate the macroscopics and update the local equilibrium distribution functions for the next step. Using (107), we can calculate the new values for the temperature T .

$$\begin{aligned}
T_0(-1, 1) &= f_0(-1, 1) + f_1(-1, 1) + f_2(-1, 1) = 1/2 \\
T_0(0, 1) &= f_0(0, 1) + f_1(0, 1) + f_2(0, 1) = 0 \\
T_0(1, 1) &= f_0(1, 1) + f_1(1, 1) + f_2(1, 1) = 1/2
\end{aligned} \tag{112}$$

Utilizing (108), we can recalculate the local equilibrium distribution function f_i^{eq} to obtain the distribution functions f_i for next time step.

$x = -1$	$x = 0$	$x = 1$
$f_0 = 2/6$	$f_0 = 0$	$f_0 = 2/6$
$f_1 = 1/12$	$f_1 = 0$	$f_1 = 1/12$
$f_2 = 1/12$	$f_2 = 0$	$f_2 = 1/12$

Table 10: Local equilibrium distribution functions of the diffusion D1Q3 LBM at time $t = 1$

The procedure is repeated several time steps to obtain the time evolution of the local distribution functions modelling the partial differential equation.